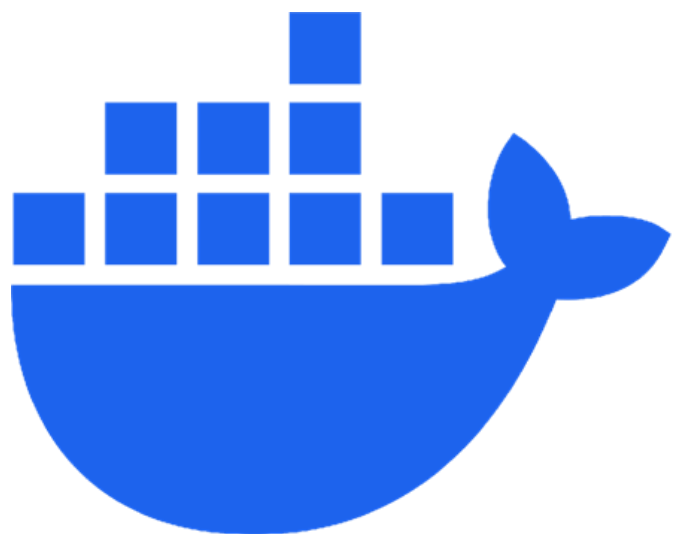


Mauro Casadei

Tecnologie e Linguaggi per il Back-End



docker®

INTRODUZIONE A DOCKER

COS'È DOCKER

Docker è una piattaforma open che permette di **sviluppare, distribuire ed eseguire applicazioni**.

È diventato uno strumento quasi indispensabile nello sviluppo e gestione del software moderno.

Consente di **separare applicazioni ed infrastruttura** su cui esse girano.

Le applicazioni vengono infatti "impacchettate" ed eseguite all'interno di ambienti isolati ed autonomi chiamati **CONTAINER**.

Isolamento e sicurezza danno la possibilità di eseguire più container contemporaneamente su un singolo host.

I container sono leggeri e **contengono tutto ciò che serve affinché l'applicazione giri** (non importa quindi cosa è installato sull'host).

La condivisione di container assicura che chi li riceve, avrà lo stesso container, funzionante allo stesso modo.

PERCHÉ USARE DOCKER

Docker semplifica la vita di sviluppatori ed aziende. Supponiamo di voler spedire un prodotto fragile e complesso a tanti clienti in luoghi diversi, usando mezzi di trasporto diversi.

Senza un imballaggio standard, ogni spedizione sarebbe un problema: un cliente non ha il camion giusto, un altro non ha la rampa, un altro ancora non ha lo spazio in magazzino.

Docker risolve questo tipo di problemi offrendo benefici come EFFICIENZA, AFFIDABILITÀ E COSTI RIDOTTI.

1. "Funziona sulla mia macchina!"... e anche sulla tua, e ovunque!

Docker serve ad evitare il problema di applicazioni che girano su alcune macchine ma non su altre. Come?

- **AMBIENTI STANDARDIZZATI E ISOLATI (CONTAINER)** contenenti tutto ciò che serve all'applicazione per funzionare (codice, librerie, dipendenze e configurazioni).
Tradotto: funzionerà esattamente allo stesso modo su laptop dello sviluppatore, sul server di test, su quello di produzione dimenticandosi dei problemi di compatibilità
- **SVILUPPO PIÙ RAPIDO E COLLABORATIVO:** i container sono facilmente condivisibili consentendo a più sviluppatori di lavorare su ambienti identici ed eliminando errori dovuti a differenze nelle configurazioni locali.

2. Velocità ed agilità nel rilascio del software.

- **CICLO DI SVILUPPO ACCELERATO (CI/CD – Integrazione Continua e Consegna Continua)**
Ogni volta che il codice viene modificato, può essere automaticamente testato in un container e, se supera i test, inviato in produzione in un altro container. Questo porta una grossa riduzione dei tempi tra scrittura del codice ed utilizzo da parte degli utenti finali.
- **DEPLOYMENT RAPIDI ED AFFIDABILI:** Dipendenze e server non devono essere configurati manualmente. Sarà sufficiente "avviare" il container e l'applicazione sarà pronta per essere eseguita. La messa in produzione risulterà quindi molto più veloce e meno soggetta ad errori umani.

3. Efficienza ed ottimizzazione delle risorse

- **LEGGERO ED EFFICIENTE**
I container di Docker sono molto più leggeri delle tradizionali Virtual Machine (VM). Si traduce nella possibilità di far girare molte più applicazioni sullo stesso server fisico, ottimizzando utilizzo dell'hardware e riducendo i costi (meno server, meno energia)
- **SCALABILITÀ IMMEDIATA**
Se l'applicazione ha un picco di traffico, si possono rapidamente avviare nuove copie del container per gestire il carico aggiuntivo. Quando il traffico diminuisce, possono essere spente con altrettanta facilità. Questa scalabilità dinamica consente l'adattamento quasi real time, evitando sprechi di risorse.

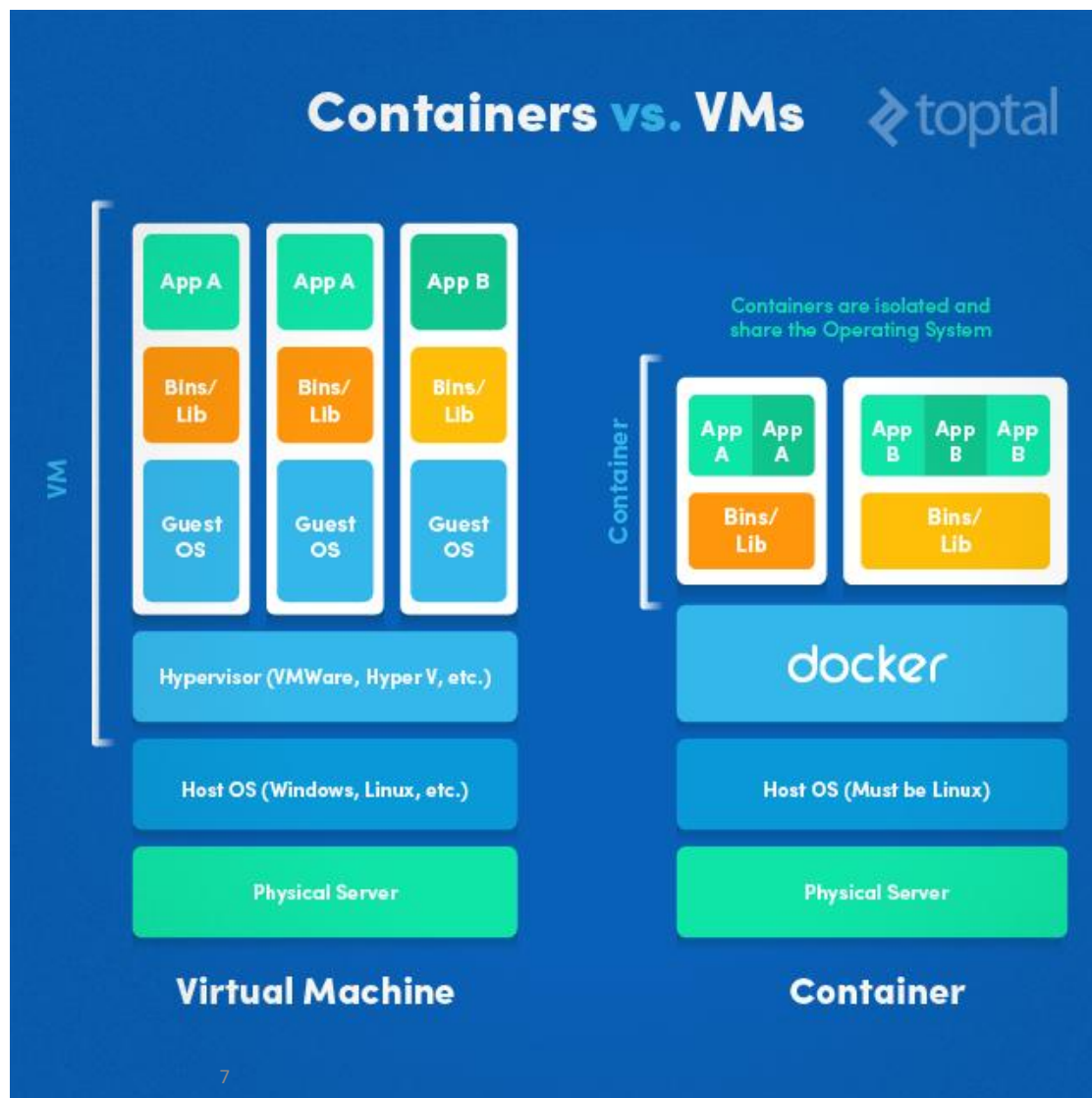
4. Portabilità su ogni tipo di infrastruttura

- **BUILD ONCE, RUN ANYWHERE**

una volta creata l'applicazione ed averla inserita in un container, questa girerà su qualsiasi ambiente che supporti Docker. Si è svincolati da specifici hardware o provider.

Potrà girare su pc, sui server dell'azienda, su servizi cloud come AWS, Google Cloud o Azure

DIFFERENZA TRA VM E CONTAINER



1. ARCHITETTURA

Una VM emula l'intero hardware di un computer, e sopra vi installa un sistema operativo completo (**Guest OS**). L'hypervisor gestisce e isola più VM sullo stesso host fisico. Ogni VM ha il proprio kernel, filesystem e librerie.

I container, invece, non virtualizzano l'hardware, ma **condividono il kernel del sistema operativo host**. Ogni container è un processo isolato con tutto ciò che serve per far girare l'applicazione. I container **condividono OS e kernel**, ma hanno file e librerie differenti

2. PESO E PRESTAZIONI

ASPETTO	VM	CONTAINER
Peso	Centinaia di MB o GB (OS completo)	Decine o centinaia di MB
Velocità	Avvio lento (avvia tutto OS)	Avvio rapidissimo (è solo un processo)
Overhead (costo extra)	Maggiore (kernel e driver duplicati)	Minimo (condivisione del kernel)

2. ISOLAMENTO

TIPO	VM	CONTAINER
Kernel	Isolato: ogni VM ha il suo	Condiviso
File System	Totalmente separato	Separato, ma non così profondo come VM
Sicurezza	Maggiore isolamento nativo	Isolamento più leggero, ma migliorabile

4. PORTABILITÀ

Le VM sono meno portabili: il formato dell'immagine dipende dall'hypervisor (es. vmdk, .vdi, ecc.)

I container, invece, sono ultra-portabili: l'immagine Docker gira allo stesso modo su Linux, Windows, cloud, server, laptop

5. CASI D'USO IDEALI

USO COMUNE	VIRTUAL MACHINE	CONTAINER
Testare OS diversi	✓ (puoi testare Windows, Linux, ecc.)	✗ (i container condividono l'OS host)
Microservizi	✗ (troppo pesante)	✓ (ogni microservizio in un container)
Legacy applications	✓ (alcuni software girano solo in ambienti completi)	✗ (potrebbero non funzionare)
Continuous Deployment	✗ (lento e difficile da orchestrare)	✓ (velocissimo, perfetto per CI/CD)

ARCHITETTURA DI DOCKER

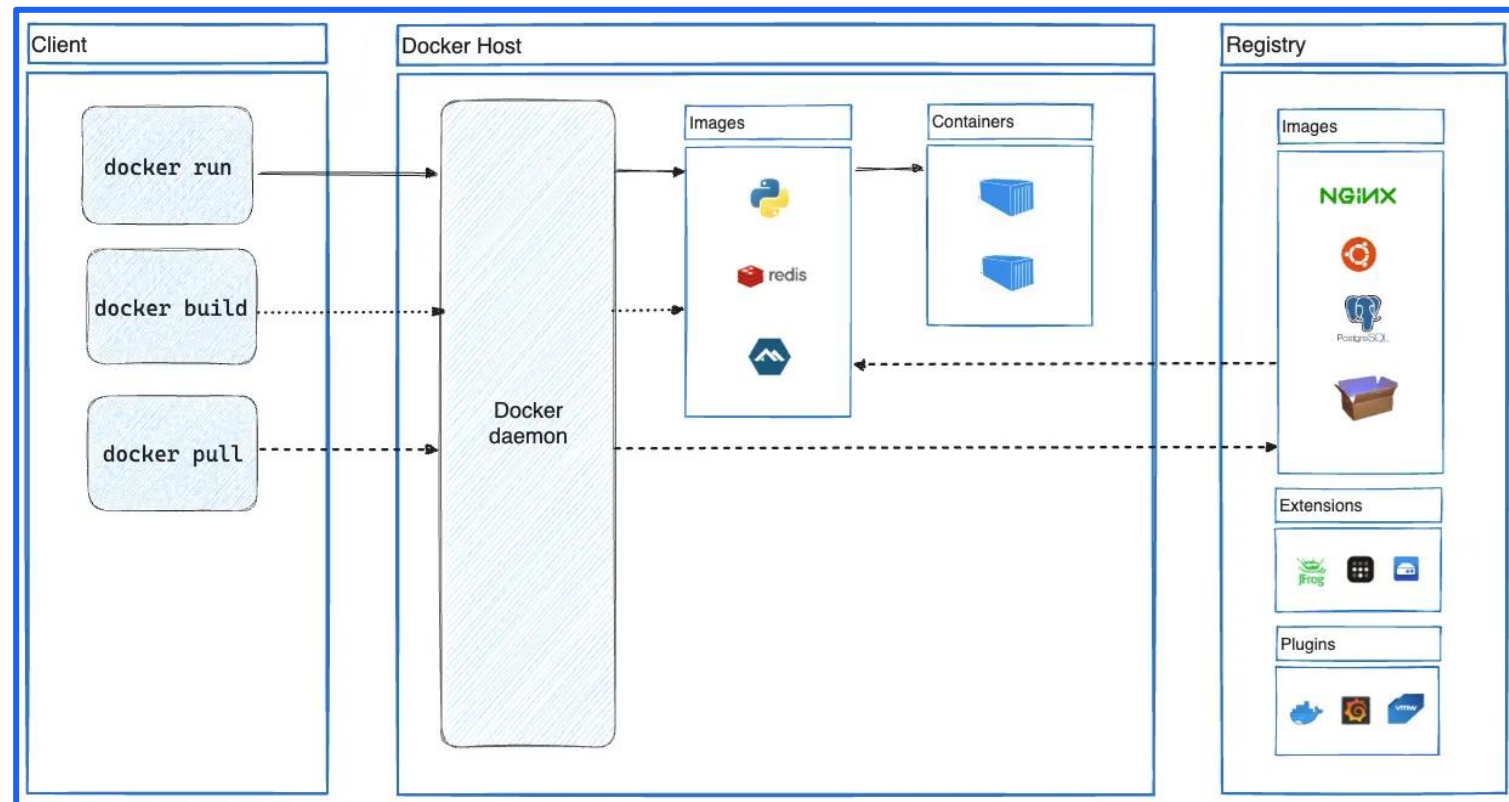
Funziona con un'**architettura client-server**.

Il client Docker comunica con il daemon Docker (dockerd), che si occupa di **creare, eseguire e distribuire i container**.

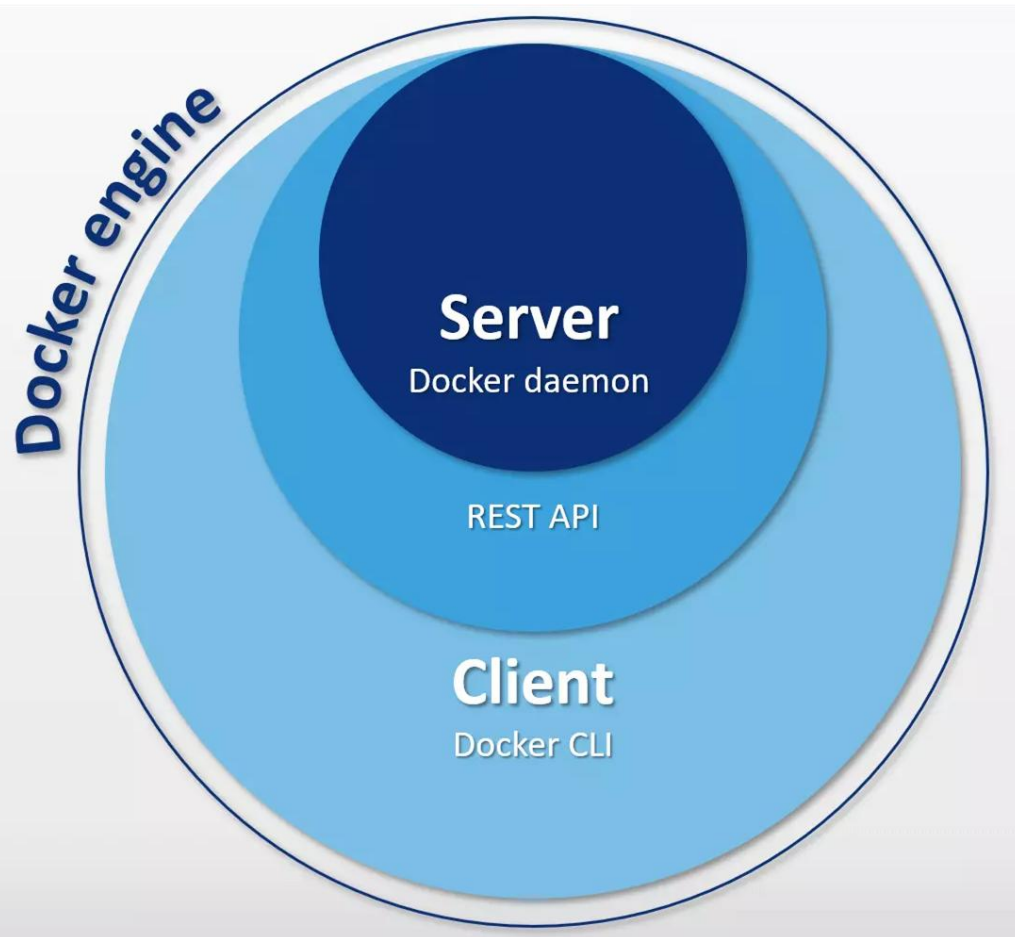
Client e daemon possono essere sullo stesso sistema oppure si può connettere un client Docker ad un daemon che si trova su un computer remoto.

Comunicheranno tra loro usando una API REST.

Altro strumento client di Docker è **Docker Compose** che permette di lavorare con applicazioni composte da un insieme di container.



COMPONENTI CHIAVE



• DOCKER ENGINE

Motore centrale di Docker che consente di **abilitare tutti i componenti e le funzionalità** di Docker:

- Creare le immagini Docker
- Eseguire i container basati su quelle immagini
- Gestire il ciclo di vita dei container (avvio, stop, riavvio)
- Dirigere la comunicazione tra container e con il sistema host.

Si compone principalmente in 3 parti:

- Docker Daemon – **il server del Docker Engine**: è il processo dockerd che gira in background;
- Docker CLI: non parla direttamente al kernel, ma **invia richieste alla REST API del daemon**;
- REST API :interfaccia di programmazione che permette al **Docker Client e ad altri programmi di comunicare con il Docker Daemon**. ESEMPIO: ogni comando digitato in CLI viene tradotto in una chiamata API che il daemon interpreta ed esegue

- **DOCKER DAEMON (dockerd)**

È il vero e proprio motore che gestisce tutte le operazioni di Docker.

- **COSA FA?** Resta sempre in ascolto di eventuali comandi: "costruire" un'applicazione, "avviare" un container, "connettere" due container tra loro. Gestisce tutto ciò che riguarda le immagini (i "modelli" della nostra app), i container (le app in esecuzione), le reti (come i container comunicano tra loro) e i volumi (dove le app salvano i dati)
- **ESEMPIO:** Quando digitiamo `docker run la-mia-app`, il daemon prende in carico la richiesta, cerca l'immagine "la-mia-app", la avvia come un container isolato e si assicura che abbia le risorse necessarie.

- **DOCKER CLIENT (docker)**

È il punto di contatto con il daemon attraverso terminale o riga di comando.

- **COSA FA?** Quando digitiamo un comando (es. `docker build` per creare un'immagine o `docker ps` per vedere i container in esecuzione), il client prende il nostro comando e lo invia al daemon. Client e daemon possono essere sullo stesso computer oppure il client può inviare comandi ad un daemon su un altro server dall'altra parte del mondo.
- **ESEMPIO:** se digitiamo `docker pull ubuntu`, il client invia la richiesta al daemon che, a sua volta, contatterà un registry (come **Docker Hub**) per scaricare l'immagine base di Ubuntu.

• DOCKER DESKTOP

Soluzione all-in-one per usare Docker su PC (Mac, Windows o Linux). Fornisce un ambiente Docker "pronto all'uso".

- **COSA INCLUDE?** Ha Docker Daemon, Docker Client, Docker Compose (per gestire più container contemporaneamente), e anche Kubernetes (strumento più avanzato per gestire numeri elevati di container su larga scala)
- **A COSA SERVE?** Semplifica l'interazione con Docker consentendo di iniziare subito a costruire e testare applicazioni containerizzate senza dover configurare i singoli componenti.

• DOCKER REGISTRIES

Sono "luoghi" dove vengono archiviate le immagini Docker. Sono come librerie di ricette (le immagini) che descrivono esattamente come preparare un certo tipo di "pizza" (la nostra applicazione)

- **DOCKER HUB:** "magazzino" pubblico e più famoso. Chiunque può caricare le proprie immagini e chiunque può scaricarle. Quando si scarica un'immagine base come Ubuntu o Node.js, Docker la prende da qui (a meno che non gli venga specificato diversamente)
- **REGISTRI PRIVATI** ai quali possono accedere solo le persone autorizzate
- **COME SI USANO?** Quando si esegue `docker pull nome-immagine`, si scarica l'immagine; quando si usa `docker push nome-immagine`, la si sta caricando.

INSTALLAZIONE E REQUISITI DI DOCKER

- Si può scaricare ed installare Docker su più piattaforme. Scaricare Docker Desktop per il proprio sistema operativo:
 - [Per Mac](#)
 - [Per Windows](#)
 - [Per Linux](#)

SISTEMA OPERATIVO	VERSIONE MINIMA	ARCHITETTURA	VIRTUALIZZAZIONE	RAM CONSIGLIATA	ALTRI REQUISITI
WINDOWS	Windows 10 64-bit Windows 11 64-bit	X86_64 / AMD 64	WSL 2 attivo + Hyper-V abilitato	Min 4GB (8 GB consigliata)	BIOS con Virtualization abilitata File system NTFS
macOS (Intel)	macOS 12 (Monterey) o superiore	X86_64 / AMD 64	Hypervisor Framework (nativo macOS)	Min 4GB (8 GB consigliata)	File system APFS
macOS (Apple Silicon)	macOS 12 o superiore	Arm64 (M1, M2, M3)	Rosetta 2 (per container x86), Hypervisor Framework	Min 4GB (8 GB consigliata)	Supporto nativo ARM + Rosetta installato
Linux (Ubuntu, Debian, Fedora, Arch)	Ubuntu 22.04, Debian 11, Fedora 37, Arch Linux (rolling)	X86_64, ARM64 (varia per distro)	Kernel con cgroups e namespaces attivi	Min 4GB (8 GB consigliata)	Supporto a systemd, cgroups v2, user con permessi sudo

2. INSTALLAZIONE SU WINDOWS

Dopo aver scaricato Docker Desktop da [questo link](#), **eseguire il file .exe**

Di default, Docker verrà installato in
C:\Program Files\Docker\Docker

Durante l'installazione assicurarsi che sia
selezionata l'opzione
Use WSL 2 instead of Hyper-V

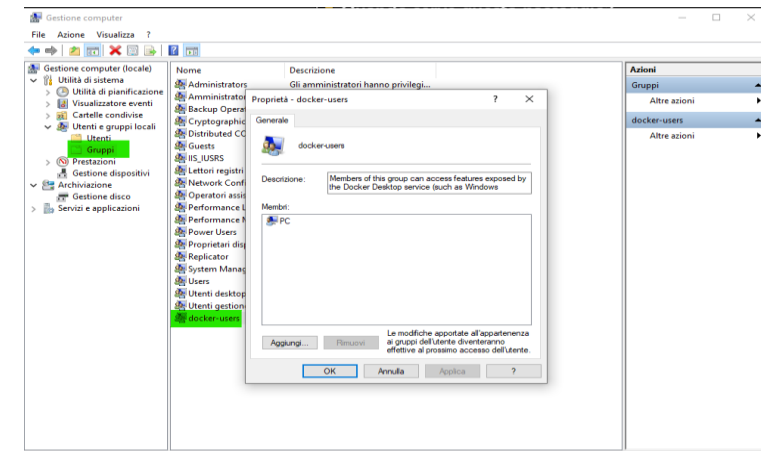
Cliccare su **install**. Al termine dell'installazione,
riavviare il pc se necessario.

Avviare Docker Desktop e completare il
processo di setup

* Se l'account utente è differente da quello
dell'amministratore del PC, al fine di evitare problemi con
l'utilizzo dei container con l'accesso a funzionalità che
richiedono privilegi elevati, **aggiungere l'utente al gruppo
docker-users.**

- I. Aprire **Gestione computer** come amministratore
- II. Andare su **Utenti e gruppi locali** → **Gruppi** →
doppio clic su **docker-users**
- III. Clic destro su docker-users → "**Aggiungi al
gruppo**"
- IV. "**Aggiungi**"
- V. Inserire nome utente Windows e premere OK

Uscire da sessione utente e rientrare per applicare i
cambiamenti.



DOCKER DESKTOP vs DOCKER ENGINE

- Docker Desktop è una **suite completa** che offre tutti gli strumenti per usare Docker in modo immediato:

- Docker engine,
- interfaccia grafica,
- supporto a Kubernetes integrato,
- gestione visuale dei container,
- possibilità di installare estensioni.

Tutto è preconfigurato: configurazione motore Docker, file di sistema, avvio dei servizi.

- Docker Desktop **utilizza però una macchina virtuale interna**, basata su **WSL2**, Hyper-V o simili. Questo si traduce in **un maggior impiego di risorse rispetto al semplice motore Docker su Linux e nella possibilità di prestazioni inferiori.**

- Docker Engine è **versione base e open source** di Docker ed è quello che viene **eseguito nativamente su Linux**.
- È il motore di Docker
- **Non ha interfaccia grafica**, non include Kubernetes integrato, né una VM.
- Molto potente e leggero, non consuma risorse inutili e **funziona direttamente sul kernel Linux** (grazie ai cgroups e namespaces)
- **Su Windows o macOS, non può essere installato direttamente**, richiede per forza una VM o Docker Desktop perché ha bisogno di kernel Linux per funzionare.

Caratteristica	Docker Desktop	Docker Engine
Cos'è	Applicazione completa con GUI, CLI, Kubernetes, estensioni e VM integrate	Motore runtime per container, installato direttamente sul sistema operativo
Interfaccia grafica (GUI)	✅ Sì (Dashboard per gestire container, immagini, volumi, ecc.)	❌ No (solo CLI)
Sistema operativo host	Windows, macOS, Linux	Linux nativo (Windows/macOS solo via WSL2 o VM)
Architettura	Utilizza una VM interna per eseguire il motore Docker (Hyper-V, WSL2, QEMU)	Esegue direttamente sul kernel Linux con cgroups e namespaces
Contesti Docker (contexts)	Crea un contesto dedicato (desktop-linux) per isolarsi dal motore nativo	Usa il contesto di default (default)
Supporto Kubernetes	✅ Integrato e attivabile dalle impostazioni	❌ Non incluso – va installato manualmente (es. kubeadm)
Aggiornamenti	✅ Aggiornamenti automatici e interfaccia per upgrade	❌ Manuali (via package manager come apt, dnf, ecc.)
Estensioni Docker	✅ Sì (Docker Extensions Marketplace)	❌ No
Performance	📉 Meno performante su Windows/macOS (usa una VM)	✅ Più performante su Linux nativo
Consumo risorse	⚠️ Più alto (VM + interfaccia + servizi)	✅ Minimo

WSL (Windows Subsystem for Linux)

- **WSL** è una funzionalità di Windows che ti permette di usare un **ambiente Linux completo** direttamente su Windows, **senza macchina virtuale tradizionale**
- Puoi usare comandi Linux (ls, cd, grep, ecc.)
- Puoi installare distribuzioni come Ubuntu, Debian, ecc.
- Docker Desktop usa WSL2 per far girare i container in modo più efficiente.
- wsl --version
- wsl --install [installerebbe ubuntu nel sistema windows]
- Per accedere:
 - Da cmd o powershell
 - Wsl
 - ls # Elenca i file
 - cd nome # Entra nella cartella "nome"
 - cd .. # Torna indietro
 - pwd # Mostra il percorso corrente
 - PS C:\Users\HP> wsl --list --verbose
 - NAME STATE VERSION
 - * docker-desktop Running 2
 - docker-desktop-data Running 2
 - **docker-desktop-data è una distro WSL speciale, non eseguibile come terminale interattivo.**
Serve **solo a Docker Desktop** per conservare i **dati persistenti** (volumi, immagini, layer)
non ha una shell e non si possono inserire comandi

CONCETTI BASE

Quando si usa Docker, si interagisce costantemente con "oggetti" come immagini e container.

1. IMAGES (immagini)

Modello immutabile ed in sola lettura.

Un'immagine Docker è come una ricetta di cucina.

Contiene tutte le istruzioni ed i componenti necessari per creare un container.

- **COSA CONTENGONO?**

Un'immagine è un pacchetto che include il **codice** della nostra applicazione, tutte le **librerie** di cui ha bisogno, le **dipendenze**, gli **strumenti** di sistema e le **configurazioni**. È un'istanza completa e definita del nostro **ambiente applicativo**.

- **BASATE SU ALTRE IMMAGINI**

Spesso un'immagine è costruita partendo da un'altra immagine "base". Ad esempio, si potrebbe partire da un'immagine di Ubuntu e poi aggiungere il server web Apache, il codice dell'applicazione e tutte le altre configurazioni necessarie.

Dockerfile

```
FROM python:3.11-slim      # 1° layer: base image
COPY . /app                # 2° layer: copia il codice
RUN pip install -r requirements.txt  # 3° layer: installa librerie
CMD ["python", "app.py"]   # 4° layer: comando di avvio
```

- **COME SI CREANO?**

Si possono usare immagini scaricate da un **Registry** (come **Docker Hub**) oppure crearne di nuove scrivendo un **Dockerfile**. Questo è un file di testo con istruzioni passo-passo. **Ogni istruzione crea uno strato (layer) nell'immagine**. Se viene cambiato il Dockerfile e si ricostruisce l'immagine, Docker ricostruirà **solo gli strati che sono cambiati**, rendendo le immagini più leggere, piccole e veloci rispetto ad altre tecniche di virtualizzazione

2. CONTAINERS

Un container è **un'istanza eseguibile** (cioè funzionante) di una Docker Image.

Con i container è possibile:

- **Crearli, avviarli, fermarli, spostarli o cancellarli**
- Connetterli a reti
- Aggiungere **storage** (memoria)
- Creare una nuova immagine a partire dal loro stato attuale

- **ISOLAMENTO DEI CONTAINER**

Per impostazione predefinita, **ogni container è isolato:**

- **Dagli altri container** (non vede gli altri container)
- **Dal computer che li ospita** (host): non ha accesso diretto al sistema operativo

Si può comunque decidere di aumentare o ridurre l'isolamento.

ASPETTO DEL SISTEMA	COSA SIGNIFICA "ISOLARLO"	COSA SI PUÒ FARE
RETE	Ogni container ha la propria rete privata (network)	Può essere connesso ad un network condiviso, gli si può assegnare un IP, farlo comunicare con altri container
MEMORIA (RAM)	Usa solo parte della RAM del sistema	Si può limitare la quantità di memoria che può usare (es. 512MB)
FILE/STORAGE	Ha un file system separato	Si può montare una cartella condivisa tra container o con il computer host
CPU	Usa tutta o parte della potenza disponibile	Si può limitare a 1 core o una certa percentuale di utilizzo
PROCESSI	Vede solo i suoi processi	Si può permettere (o impedire) di vedere o interagire con altri
PERMESSI	Accesso limitato al sistema	Si può consentire l'accesso a dispositivi, porte, file speciali, ecc.

• COM'È FATTO UN CONTAINER

Un container è basato su:

- Un'immagine di partenza
- Le opzioni di configurazione che gli si attribuiscono in fase di creazione o avvio.

Quando un container viene eliminato, tutte le modifiche fatte spariscono, a meno che non siano state salvate in uno storage persistente (es. volumi Docker)

ESEMPIO: COMANDO `docker run`

```
docker run -i -t ubuntu /bin/bash
```

Cosa fa questo comando?

- Docker **scarica l'immagine di Ubuntu** (se non abbiamo già quell'immagine). È come usare `docker pull Ubuntu`
- Docker **crea un nuovo container** (come se facessimo `docker container create`)
- Viene assegnato al container un **file system scrivibile** in modo che il container possa creare/modificare file

- Docker collega al container una rete predefinita e gli dà un indirizzo IP, così può accedere ad internet.
- Il container viene avviato ed esegue `bin/bash`, cioè apre un terminale Linux interattivo.
 - i = interattivo (riceve input da tastiera)
 - t = apre un terminale virtuale (TTY)
- Quando si lancia il comando `exit`, il comando `/bin/bash` termina, il container si ferma (ma non viene cancellato) e può essere riavviato oppure eliminato.
- più container possono **condividere la stessa immagine di base**, ma restano indipendenti nello stato di esecuzione

3. VOLUMES

- **Archivi di dati persistenti** creati e gestiti da Docker. Consentono di **salvare dati al di fuori del layer scrivibile del container**, garantendo che le informazioni **persistano anche se il container viene rimosso o ricreato**.
- I volumi vengono **archiviati in una directory sul sistema host** e possono essere montati in uno o più container.
- Servono a:
 - **condividere dati** tra container,
 - **conservare** dati importanti come **database, file caricati dagli utenti o file di configurazione**.
 - **Mantenere i dati anche dopo la rimozione di un container**
 - **Effettuare backup, ripristino o migrazione dei dati**
 - **Migliorare le prestazioni** rispetto alla scrittura sul layer scrivibile del container
 - **Gestire i dati in modo indipendente** dal sistema operativo host e dalla struttura delle cartelle

- I volumi sono gestiti direttamente da Docker e isolati dal file system dell'host, il che li rende più sicuri e portabili rispetto ai **bind mount**, nella maggior parte dei casi.

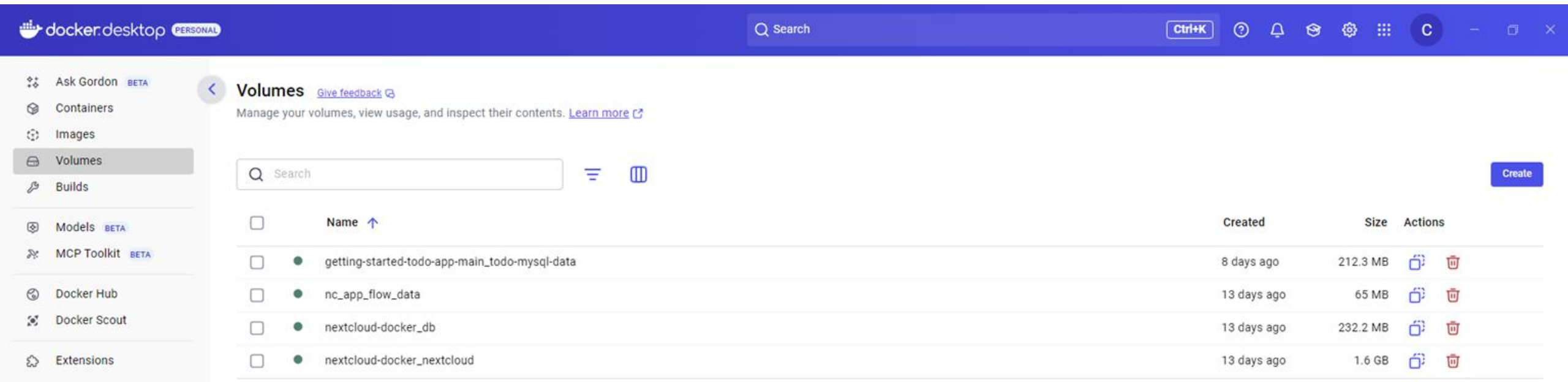
COMANDI CLI PER GESTIONE VOLUMI -

COMANDO	FUNZIONE
docker volume create mio-volume	Crea volume
docker volume ls	List volumi
docker volume inspect mio-volume	Dettagli e path volume
docker volume rm mio-volume	Rimozione
docker volume prune	Rimuovi tutti volumi inutilizzati

GESTIONE VOLUMI TRAMITE DOCKER DESKTOP - [doc](#)

Docker Desktop offre una **interfaccia grafica (GUI)** per gestire i volumi in modo semplice e visivo:

- **Visualizzare i volumi:** elenco completo con stato, data di creazione e dimensione.
- **Creare un volume:** clic su **Crea**, inserisci un nome e conferma.
- **Ispezionare un volume:** clicca su un volume per vedere quali container lo usano e sfogliare i file presenti.
- **Clonare un volume:** crea una copia completa di un volume con tutti i suoi dati.
- **Eliminare un volume:** clic sull'icona del cestino (il volume non deve essere in uso).
- **Esportare/Importare un volume:** utile per **backup** o **migrazione dei dati**.
- Puoi anche **filtrare, ordinare e personalizzare** le colonne nella vista volumi.



4. DOCKERFILE

- È da un dockerfile che tutto ha inizio. Docker crea infatti le immagini leggendo le istruzioni da un Dockerfile.
- È un documento di testo contenente **tutte le istruzioni necessarie** per costruire il codice sorgente.
- **Istruzioni più comuni**
 - **FROM** <image> : definisce l'immagine base su cui costruire immagine finale
 - **RUN** <comando>: Esegue comandi in un nuovo layer sopra l'immagine corrente, e ne registra il risultato come layer. Può usare la forma shell.
 - **WORKDIR** <directory>: Imposta la cartella di lavoro per le istruzioni successive (RUN, CMD, COPY, ecc.).
 - **COPY** <src> <dest>: Copia file o directory da <src> nel filesystem dell'immagine a <dest>.
 - **CMD** <comando>: Definisce il programma di default da eseguire quando il container parte. Solo l'ultima CMD è applicata se ve ne sono più.
- **NOME DEL FILE**: il nome predefinito è **Dockerfile**, senza estensione. IN questo modo si evita l'uso dell'opzione - -file al comando docker build. Se ci sono più Dockerfile nel progetto, si possono usare nomi come <nome>.Dockerfile e specificarli con - -file durante la build

5. REGISTRY

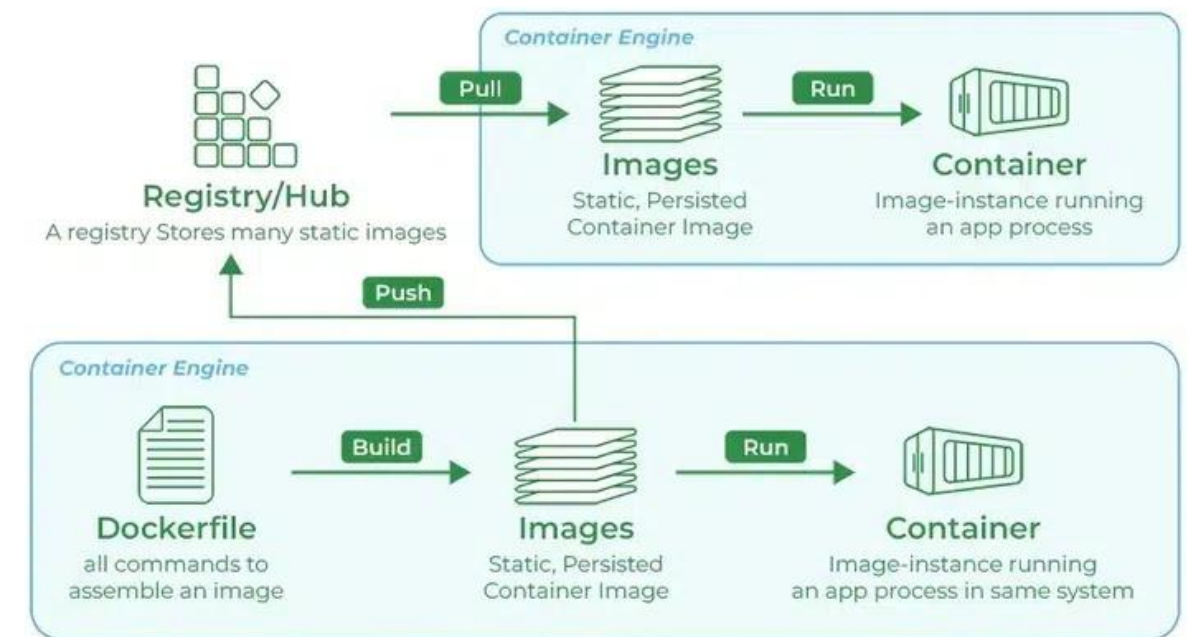
- Un registry è un luogo personalizzato dove si possono **archiviare e condividere immagini di container**.
- Può essere **pubblico o privato**
- Il Registry predefinito di Docker è **Docker Hub**, registro pubblico.
- Altri registri pubblici sono:
 - Amazon Elastic Container Registry (ECR)
 - Azure Container Registry (ACR)
 - Google Container Registry (GCR)
- È possibile creare un **registry privato locale** usando, ad esempio, GitLab Container Registry, Harbor o JFrog Artifactory



DIFFERENZA TRA REGISTRY E REPOSITORY

Simili ma non sono esattamente la stessa cosa.

- Registry è il sistema/server che gestisce le immagini
- Repository è come una cartella all'interno del registry che raccoglie immagini correlate tra loro (ad esempio, immagini di diverse versioni dello stesso progetto). Quindi un registry contiene un repository. Ogni repository contiene una o più immagini versionate.





Comandi Principali

- **Immagini**

- docker image ls
- docker images
- docker pull
- docker build
- docker tag
- docker push
- docker rmi
- docker image inspect
- docker history
- docker save
- docker load
- docker image prune

- **Container**

- docker run
- docker create
- docker start
- docker stop
- docker restart
- docker kill
- docker pause
- docker unpause
- docker rm
- docker ps
- docker ps -a
- docker exec
- docker logs
- docker inspect
- docker stats
- docker top
- docker rename
- docker update
- docker wait
- docker cp
- docker attach

- **Volumi**

- docker volume ls
- docker volume create
- docker volume inspect
- docker volume rm
- docker volume prune

- **Network**

- docker network ls
- docker network create
- docker network inspect
- docker network connect
- docker network disconnect
- docker network rm
- docker network prune

- **Sistema / Pulizia**

- docker info
- docker version
- docker system df
- docker system prune
- docker events

- **Docker Compose**

- docker compose up
- docker compose down
- docker compose build
- docker compose ps
- docker compose logs
- docker compose exec
- docker compose run
- docker compose stop
- docker compose start
- docker compose restart
- docker compose pull
- docker compose config

docker image ls (docker image list)

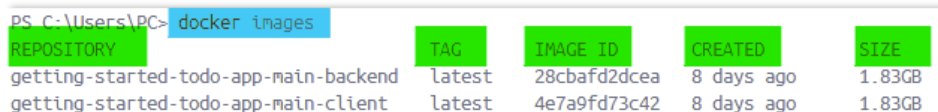
docker image ls [OPTIONS] [REPOSITORY[:TAG]]

docker image list

docker images

Per impostazione predefinita mostra:

- Tutte le immagini di primo livello (cioè non quelle intermedie)
- Il loro repository
- Il loro tag
- L'id
- Data creazione
- Dimensione complessiva (SIZE): include tutti i layer, anche quelli ereditati dai parent (base image)



REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
getting-started-todo-app-main-backend	latest	28cbafd2dcea	8 days ago	1.83GB
getting-started-todo-app-main-client	latest	4e7a9fd73c42	8 days ago	1.83GB

```
PS C:\Users\HP> docker images --filter  
"reference=php*"
```

REPOSITORY	TAG	IMAGE ID	CREATED
SIZE	28		

- Un'unica immagine può essere etichettata più volte (es. myapp:latest, myapp:v1, ecc.). In questo caso apparirà più volte nell'elenco ma il suo ID sarà lo stesso e lo spazio su disco viene usato una sola volta, anche se appare più volte.

OPZIONI

-a, --all : mostra tutte immagini

--digests: show digests

-f, --filter: filtra i risultati in base alle condizioni fornite.

--format: formatta output usando un custom template. table è il formato tabellare di default. Accetta json e Go Template

--no-trunc: non tronca i valori lunghi (es. mostra l'intero IMAGE_ID o COMMAND)

-q, --quiet: mostra solo ID delle immagini.

--tree: mostra le immagini multi-piattaforma in una struttura ad albero (sperimentale)

```
PS C:\Users\HP> docker images --format json|
```


Esempio creare immagine alpine da Dockerfile o da Immagine scaricata

- Da una immagine su repository
- `docker build -t ss-alpine-linux:1.0 .`
- `-t` = tag (cioè nome + versione)
- `ss-alpine-linux` = nome dell'immagine
- `:1.0` = tag/versione (default è latest se non lo metti)
- `.` = contesto di build (cartella corrente dove sta il Dockerfile)
- Dockerfile
- `FROM alpine:latest`
- `WORKDIR /app`
- Da una immagine presente su docker hub e **senza Dockerfile**
- `docker run -it --name mio_container ss-alpine-linux:1.0`
- Per usare l'immagine:
- Creare il container e Eseguire il container (in questo caso apre la shell)
- `docker run -ti soluzionisoftware/alpine-test:dev sh`

Controllare e ripulire lo spazio impegnato

- Solo immagini inutili (dangling)
 - `docker image prune`
- Tutto il non usato (immagini, volumi, container, reti)
 - `docker system prune -a`
- Vedere lo spazio occupato
 - `docker system df`
- Ripulire container non utilizzati
 - `docker container prune`

Ask Gordon

BETA

Containers

Images

Volumes

Builds

Models

BETA

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

[Give feedback](#)

View and manage your local and Docker Hub images. [Learn more](#)

Search

☐	Name	Tag	Image ID	Created	Size	Actions
☐	getting-started-todo-app-main-backend	latest	28cbafd2dcea	8 days ago	1.82 GB	
☐	getting-started-todo-app-main-client	latest	4e7a9fd73c42	8 days ago	1.83 GB	
☐	traefik	v3.4	06ddf61ee653	13 days ago	288.29 MB	
☐	nextcloud	latest	f89541cd2f09	26 days ago	2.03 GB	
☐	ghcr.io/nextcloud/flow	1.2.0	e62da728d9d0	3 months ago	3.85 GB	
☐	mariadb	10.6	7589b386fa66	3 months ago	539.13 MB	
☐	mysql	9.3	b9d8b7ec6e6a	4 months ago	1.17 GB	
☐	phpmyadmin	latest	c23c54bd48ac	6 months ago	813.31 MB	
☐	tecnativa/docker-socket-proxy	latest	9e4b9e7517a6	11 months ago	30.63 MB	

docker rmi(docker image remove)

docker image rm [OPTIONS] IMAGE [IMAGE...]

docker image remove

docker rmi

Rimuove una o più immagini Docker dalla macchina locale. Docker rimuove il tag specificato e, se non ci sono più tag che fanno riferimento a quell'immagine, elimina anche l'immagine vera e propria.

Non rimuove immagine da un registry.

Non si può rimuovere un'immagine da un container in esecuzione a meno che non si usi l'opzione **-f**.

OPZIONI

-f, --force: forza la rimozione di un'immagine

--no-prune: non eliminare parents senza tag

--platform: permette di rimuovere solo una specifica variante di piattaforma di un'immagine multi-platform (cioè immagini disponibili per più sistemi operativi o architetture).

```
docker rmi fd484f19954f
```

```
Error: Conflict, cannot delete image fd484f19954f because it is tagged in multiple repositories, use -f to force
2013/12/11 05:47:16 Error: failed to remove one or more images
```

```
docker rmi test1:latest
```

```
Untagged: test1:latest
```

```
docker rmi test2:latest
```

```
Untagged: test2:latest
```

```
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
test	latest	fd484f19954f	23 seconds ago	7 B (virtual 4.964 MB)

```
docker rmi test:latest
```

```
Untagged: test:latest
```

```
Deleted: fd484f19954f4920da7ff372b5067f5b7ddb2fd3830cecd17b96ea9e286ba5b8
```

Container

docker run

crea ed avvia un nuovo container a partire da una specifica immagine docker. Accetta parametri opzionali per la configurazione del container.

docker run [OPTIONS] IMAGE[:TAG|@DIGEST] [COMMAND] [ARG...]

Il riferimento all'immagine è il modo in cui si indica **quale immagine usare** per creare o avviare un container. Include:

- **Nome immagine** – IMAGE (es. ubuntu, nginx)
- Un **:TAG (versione)** – facoltativo (es. :24.04, :latest)
- Un **@DIGEST (identificativo univoco ed immutabile)** basato su SHA256 e che rappresenta il contenuto) – facoltativo.

Utile per garantire che l'immagine non sia stata modificata.

SINTASSI

```
docker run IMAGE[:TAG][@DIGEST]
docker create IMAGE[:TAG][@DIGEST]
```

ESEMPI:

```
docker run ubuntu:24.04
docker run alpine@sha256:9cacb7... Date
```

OPTIONS

Esistono diverse opzioni per **personalizzare l'avvio** del container. Tra queste:

- **--name nome_container**: assegna un nome al container
- **-d** o **--detach**: esegue il container in background
- **-p 8080:80** : mappa le porte
- **--memory, --cpus**: limiti di risorse
- **--network**: configura la rete
- **--rm**: rimuove automaticamente il container alla fine

COMANDI E ARGOMENTI

Per specificare quale comando eseguire all'avvio.

ESEMPIO: avviare una shell interattiva nel container ubuntu, dove poter scrivere comandi

```
docker run -it ubuntu sh
```

-i: mantiene aperto lo standard input, cioè permette di interagire con il container scrivendo comandi anche dopo l'avvio

-t: alloca un terminale (tty), cioè fornisce un'interfaccia simile ad un terminale vero e proprio, utile per formattare output e rendere leggibile interazione

Sh: comando eseguito all'avvio del container. IN questo caso la shell terminale sh, per eseguire comandi all'interno del container stesso

EXIT STATUS

Quando esegui un container con docker run, questo termina restituendo un codice di uscita (exit code) che indica come è terminato: 0: tutto ok, esecuzione terminata con successo. ≠ 0: qualcosa è andato storto.

- **125** – Errore di Docker stesso (prima di avviare il container). Il container non è neanche partito perché c'è stato un errore a livello di Docker (es. opzione errata, permessi, ecc.).
- **126** – Il comando esiste, ma non può essere eseguito. Significa che il file è presente, ma non è un comando eseguibile (es. una directory, permessi insufficienti, ecc.).
- **127** – Il comando non esiste. Significa che Docker ha cercato di eseguire un comando che non è presente nel container.
- Altri codici: Se il container viene avviato correttamente, qualsiasi altro codice di uscita è quello del **comando eseguito all'interno del container**

docker ps

docker ps

docker ps -a

fornisce una **lista di tutti i container in esecuzione** (di default, quelli in esecuzione nel momento in cui si lancia il comando).

-a, --all per vedere tutti i containers, anche quelli fermati.

-q, --quiet per vedere solo l'id dei container attivi.

docker ps -q

--filter, -f : filtrare per stato, nome, immagine

docker ps --filter "name=nginx"

docker ps --filter "status=exited"

docker ps --filter "ancestor=ubuntu"

docker ps -f "label=com.example.version=1.0"

Filtro per name che accetta corrispondenze parziali (senza wildcard) > docker ps -a --filter "name=web«

docker ps -a --filter ancestor=wordpress

-n <num> : mostra solo gli ultimi <num> container creati

docker ps -n 3

-l, --latest: mostra **ultimo container creato** (in qualsiasi stato)

--no-trunc: mostra le informazioni complete (non troncate)

--format personalizza l'output (solo alcune colonne). Campi disponibili:

- .ID
- .Image
- .Command
- .CreatedAt
- .RunningFor
- .Status
- .Ports
- .Names
- .Size.Labels

docker ps --format "table {{.ID}}\t{{.Image}}\t{{.Status}}"

COMBINAZIONI AVANZATE

- Ultimi due container fermati
`docker ps -a -f "status=exited" -n 2`
- ID dei container NGINX attivi:
`docker ps -q -f "ancestor=nginx"`

OUTPUT DEL COMANDO

- **CONTAINER ID**: identificativo univoco
- **IMAGE**: immagine da cui è stato creato il container
- **COMMAND**: comando che è stato eseguito all'avvio del container
- **STATUS**: stato attuale del container
- **PORTS**: mappatura delle porte (es. porta 80 del container <-> porta 80 del sistema host)
- **NAMES**: nome automatico o personalizzato del container

Run 'docker --help' for more information

PS C:\Users\PC> `docker ps`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
04aa202510d3	ghcr.io/nextcloud/flow:1.2.0	"/bin/sh /ex_app_scr..."	12 days ago	Up 4 hours	8000/tcp	nc_app_flow
cbdec51cf9de	nextcloud:latest	"/entrypoint.sh apac..."	12 days ago	Up 4 hours	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	nextcloud-docker-app-1
34853153c819	tecnativa/docker-socket-proxy	"docker-entrypoint.s..."	12 days ago	Up 4 hours	0.0.0.0:2375->2375/tcp, [::]:2375->2375/tcp	docker-socket-proxy
00174b657cf1	mariadb:10.6	"docker-entrypoint.s..."	12 days ago	Up 4 hours	3306/tcp	nextcloud-docker-db-1

PS C:\Users\PC>

docker stop

docker stop [OPTIONS] CONTAINER [CONTAINER..]

Ferma uno o più container in esecuzione specificandone l'ID o il nome.

- Fermare **un singolo container** indicandone il **nome**

docker stop my_container

- Fermare un singolo container indicandone **l'ID**

docker stop 5f3c13b4a8e2

- Fermare **più container** insieme

docker stop web1 web2 db1

- Fermare **tutti i container** in esecuzione

docker stop \$(docker ps -q)

docker stop (docker ps -q)

In powershell non mettere \$

DIFFERENZA TRA docker stop e docker kill

docker stop invia prima il **SIGTERM** al processo principale del container, se dopo il timeout il processo non si ferma, invia **SIGKILL** per forzare interruzione.

docker kill **invia direttamente SIGKILL**, chiudendo immediatamente il processo.

- Specificare un **timeout personalizzato** (in secondi)
Per impostazione predefinita, docker stop aspetta **10 secondi prima di forzare l'arresto** di un container, dandogli il tempo di chiudere i processi in modo pulito.

docker stop -t 5 my_container

Container CPU usage ⓘ

2.96% / 400% (4 CPUs available)

Container memory usage ⓘ

317.28MB / 5.57GB

Show charts

Search



Only show running containers

Delete



	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☒	nc_app_flow	04aa202510d3	nextcloud/flow:1.2.0		2.92%	5 hours ago	
☐	getting-started-todo-app-main	-	-	-	0%	8 days ago	
☐	nextcloud-docker	-	-	-	0.04%	5 hours ago	
☐	app-1	cbdec51cf9de	nextcloud:latest	8080:80	0%	5 hours ago	

Selected 1 of 6

- Su Docker Desktop, cliccare direttamente il pulsante stop in corrispondenza del container per fermarlo
- Per fermare più container contemporaneamente, selezionare i container che si desiderano arrestare e cliccare sul pulsante stop in alto a destra.

docker rm

`docker rm [OPTIONS] CONTAINER [CONTAINER..]`

Rimuove uno o più container.

OPZIONI

- f, --force:** forza la rimozione di un container in esecuzione (usa **SIGKILL**)
- l, --link:** rimuove il link specificato
- v, --volumes:** rimuove volumi anonimi associati al container

ESEMPI

- Rimuovere un container
Rimuove il container referenziato sotto al link /redis

`docker rm /redis`

- Forzare la rimozione di un container in esecuzione

`docker rm --force redis`

- Rimuovere un container ed i suoi volumi. Docker rimuove solo i volumi anonimi, non quelli nominati.

`docker rm --volumes redis`

Container CPU usage ⓘ
7.64% / 400% (4 CPUs available)

Container memory usage ⓘ
313.08MB / 5.57GB

Show charts

Q Search

☰

Only show running containers

Delete

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☒	nc_app_flow	04aa202510d3	nextcloud/flow:1.2.0		7.59%	6 hours ago	
☐	> getting-started-todo-app-main	-	-	-	0%	8 days ago	
☐	▼ nextcloud-docker	-	-	-	0.06%	6 hours ago	
☐	app-1	cbdec51cf9de	nextcloud:latest	8080:80	0.01%	6 hours ago	

39

Selected 1 of 6

Creare un wordpress

- Senza docker compose
- # Crea rete per collegare container
- `docker network create wpnet`
- # Avvia MySQL
- `docker run --name wordpresddb --network wpnet -e MYSQL_DATABASE=wordpress -e MYSQL_USER=wp_user -e MYSQL_PASSWORD=wp_pass -e MYSQL_ROOT_PASSWORD=root_pass -d mysql:5.7`
- # Avvia WordPress
- `docker run --name wordpresssite --network wpnet -p 8000:80 -e WORDPRESS_DB_HOST=wordpresddb:3306 -e WORDPRESS_DB_NAME=wordpress -e WORDPRESS_DB_USER=wp_user -e WORDPRESS_DB_PASSWORD=wp_pass -d wordpress`

```
docker run --name sswp_myphpmyadmin --network wpnet -d -e PMA_HOST=wordpresddb -p 8080:80 phpmyadmin
```

```
http://localhost:8080/  
u: wp_user  
p: wp_pass
```



Blog

Ciao mondo!

Ti diamo il benvenuto in WordPress. Questo è il tuo primo articolo. Modificalo o eliminalo e quindi inizia a scrivere!

31 Agosto 2025

```
PS C:\Users\HP> docker exec -it 601e9155e691 sh
```

```
# ls
```

index.php	wp-admin	wp-config-sample.php	wp-includes	wp-mail.php	xmlrpc.php
license.txt	wp-blog-header.php	wp-config.php	wp-links-opml.php	wp-settings.php	
readme.html	wp-comments-post.php	wp-content	wp-load.php	wp-signup.php	
wp-activate.php	wp-config-docker.php	wp-cron.php	wp-login.php	wp-trackback.php	

```
#
```

docker exec

docker container exec [OPTIONS] CONTAINER COMMAND [ARG...]

docker exec

Lancia un nuovo comando in un container in esecuzione. Il comando viene eseguito solo mentre è attivo il processo principale del container (PID 1). Non viene riavviato se il container viene riavviato.

Il comando viene eseguito nella directory di lavoro predefinita del container e deve essere un eseguibile valido:

✓ Funziona:

```
docker exec -it my_container sh -c "echo a && echo b"
```

Viene avviato sh con -c per permettere di eseguire comandi concatenati.

✗ Non funziona:

```
docker exec -it my_container "echo a && echo b"
```

Questo non funziona perché "echo a && echo b" non è un eseguibile.

Utile per:

- lanciare comandi o shell in container attivi
- Diagnosticare, modificare o controllare l'ambiente di un container senza riavviarlo
- Sperimentare comandi senza modificare il processo principale.

OPZIONI PRINCIPALI

-d, --detach: esegue comando in background

--detach-keys: permette di modificare la combinazione di tasti per staccarsi dal container

-e, --env: imposta variabili d'ambiente per il comando eseguito

--env-file: legge un file contenente variabili di ambiente

-i, --interactive: mantiene lo STDIN aperto

--privileged: concede privilegi estesi al comando

-t, --tty: alloca una pseudo-TTY (terminale virtuale)

-u, --user: specifica utente/UID e gruppo/GID

-w, --workdir: imposta una directory di lavoro per il comando

ESEMPI

1. PER PRIMA COSA, AVVIARE UN CONTAINER

```
docker run --name mycontainer -d -i -t alpine /bin/sh
```

Crea un container chiamato **mycontainer** con immagine **alpine**.

Se non specificato un nome docker sceglie come nome un aggettivo, seguito dal nome di uno scienziato famoso.

Avvia **/bin/sh** come **processo principale**.

Opzioni:

- d = in background
- i = mantiene STDIN aperto
- t = terminale TTY

2. ESEGUIRE UN COMANDO NEL CONTAINER

```
docker exec -d mycontainer touch /tmp/execWorks
```

Crea un file **/tmp/execWorks** nel container, in background.

3. AVVIARE UNA SHELL INTERATTIVA

```
docker exec -it mycontainer sh
```

Apri una **nuova sessione sh** nel container, interattivamente.

4. IMPOSTARE VARIABILI D'AMBIENTE

Il comando docker exec eredita le variabili d'ambiente impostate quando il container è stato creato. Si usa **--env** o **-e** per sovrascrivere variabili d'ambiente globali o per impostare variabili d'ambiente aggiuntive per il processo iniziato da docker exec.

```
docker exec -e VAR_A=1 -e VAR_B=2 mycontainer env
```

Crea una nuova sessione shell nel container specificato, con le variabili d'ambiente VAR_A e VAR_B settate solo per quel comando, non per l'intero container. Non sono quindi visibili o accessibili dagli altri processi già in esecuzione all'interno del container.

Questo comando va eseguito sul sistema host, e non dentro il container.

Si otterrà il seguente output:

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
HOSTNAME=ac54b11631b0
```

```
VAR_A=1
```

```
VAR_B=2
```

```
HOME=/root
```

```
apk add htop  
htop
```

5. IMPOSTARE DIRECTORY DI LAVORO

Di default, docker exec esegue i comandi nella stessa directory impostata quando è stato creato il container

```
docker exec -it mycontainer pwd
```

OUTPUT:

/ (root, directory di default).

Per specificare la directory in cui eseguire il comando, usare opzione `--workdir` (o shorthand `-w`)

```
docker exec -it -w /root mycontainer pwd
```

OUTPUT:

/root (directory impostata con `-w`). Pwd è un comando Linux che sta per "print working directory"

6. ESECUZIONE DI EXEC SU CONTAINER IN PAUSA

```
docker pause mycontainer
```

```
docker exec mycontainer sh
```

OUTPUT:

Error response from daemon: Container mycontainer is paused, unpause the container before exec

Se il container è in pausa, il comando exec fallirà con un errore.

```
echo $?
```

OUTPUT: 1 (comando fallito)

docker logs (docker container logs)

docker container logs [OPTIONS] CONTAINER

docker logs

Serve per recuperare i log prodotti da un container Docker, cioè tutto ciò che è stato stampato su STDOUT (standard output) e STDERR(standard error).

Il comando legge i log già presenti al momento dell'esecuzione. Per vedere in tempo reale i nuovi log, usare `--follow`.

OPZIONI

--details: dettagli extra (es. variabili d'ambiente, etichette) fornite con `--log-opt`

-f, --follow: output dei log in tempo reale

--since: solo log generati dopo un certo orario (RFC3339, UNIX timestamp, durata Go tipo 5m)

-n, --tail: solo ultime N righe di log

-t, --timestamp: aggiunge un timestamp RFC3339Nano a ogni riga di log

--until: solo log generati prima di un certo orario.

ESEMPIO – RECUPERARE LOGS FINO AD UNA CERTO LIMITE TEMPORALE (--until)

```
$ docker run --name test -d busybox sh -c "while true; do $(echo date); sleep 1; done"
$ date
Tue 14 Nov 2017 16:40:00 CET
$ docker logs -f --until=2s test
Tue 14 Nov 2017 16:40:00 CET
Tue 14 Nov 2017 16:40:01 CET
Tue 14 Nov 2017 16:40:02 CET
```

Viene avviato un container che stampa la data ogni secondo.

Il comando `docker logs` visualizza in tempo reale i log, ma solo fino a 2 secondi.

Dockerfile

CREAZIONE DI IMMAGINI PERSONALIZZATE

Il Dockerfile è come uno snapshot della nostra applicazione. È un file di testo che contiene le istruzioni per costruire un'immagine Docker. Ogni istruzione costituisce un layer dell'immagine.

Le istruzioni più comuni sono:

- **FROM**: definisce l'immagine base (es. python:3.12). È obbligatoria e deve essere la prima istruzione valida (dopo eventuali ARG)
- **WORKDIR**: imposta la directory di lavoro per i comandi successivi (RUN, COPY, ecc.). Se non esiste, viene creata automaticamente
- **COPY / ADD**: copia file dall'host al container. Sintassi: COPY <src> <dest>
- **RUN**: esegue comandi sul sistema operativo del container durante la build
- **ENV**: imposta variabili d'ambiente permanenti che saranno disponibili in runtime nel container

- **EXPOSE**: documenta le porte esposte dal container
- **USER**: cambia utente con cui verranno eseguiti i comandi successivi ed il processo finale. Utile per non agire come root
- **CMD**: comando eseguito al runtime del container. IN caso di più CMD, solo l'ultimo conta

Altre istruzioni utili sono:

- **ARG**: definisce variabili usate solo durante la build
- **ENTRYPOINT**: imposta il comando principale del container. CMD può fungere da argomento di default per ENTRYPOINT
- **LABEL**: aggiunge metadati all'immagine
- **ADD**: simile a COPY, ma supporta URL remoti e file tar, con decompressione automatica
- **ONBUILD**: definisce istruzioni che saranno eseguite quando l'immagine diventa base di un'altra build
- **HEALTHCHECK**: verifica la salute del container eseguendo un comando periodico
- **SHELL**: imposta la shell predefinita usata dalle istruzioni RUN, CMD, ENTRYPOINT in forma shell
- **VOLUME**: dichiara una cartella dove Docker può montare volumi persistenti
- **STOPSIGNAL**: definisce il segnale che Docker invia per fermare il container (es. SIGTERM)

- delle immagini Docker. **Ogni istruzione del Dockerfile genera un layer** Ogni layer rappresenta le modifiche rispetto al precedente.

Esempio di Dockerfile Python+Flask:

```
# syntax=docker/dockerfile:1 - DIRETTIVA PARSER

FROM ubuntu:22.04

# install app dependencies
RUN apt-get update && apt-get install -y python3 python3-pip
RUN pip install flask==3.0.*

#install app
COPY hello.py /

# final configuration
ENV FLASK_APP=hello
EXPOSE 8000
CMD ["flask", "run", "--host=0.0.0.0", "--port=8000"]
```

Nell'esempio:

- Si utilizza **Ubuntu come base (FROM)**: tutte le istruzioni che seguono sono eseguite in questa immagine di base. Esistono molte immagini pubbliche sulle quali sviluppare progetti: DOCKER HUB
- Si **installano dipendenze (RUN)**: esegue una shell in Ubuntu che aggiorna l'indice del pacchetto APT e installa Python nel container. Il secondo RUN installa la dipendenza flask richiesta dall'applicazione Python
- Si **copia il file dell'app (COPY)**: copia il file hello.py dalla directory locale (build context) nella root (/) del file system del container in fase di build. Il **build context** è la **cartella (ed il suo contenuto) inviato a Docker quando si esegue il comando build. Docker può accedere solo ai file presenti all'interno del build context**. Se un file è fuori, il comando COPY fallirà.
- Si setta una **variabile d'ambiente (ENV)**
- Si **espone la porta dell'app (EXPOSE)**: indica che l'immagine finale ha servizi in ascolto su porta 8000.

- Si definisce il comando di default all'avvio (CMD): in questo caso viene avviato il server di sviluppo flask in attesa su tutti gli indirizzi sulla porta 8000. Viene usata la versione "exec form" di CMD. Si può usare anche la "shell form":

CMD flask run --host 0.0.0.0 --port 8000

- La direttiva parser specifica quale sintassi usare e consente compatibilità retroattiva con versioni precedenti.
- #commenti: i commenti sono preceduti da #, così come direttiva di sintassi. Viene interpretato come direttiva solo se il pattern corrisponde ad una direttiva ed appare all'inizio del file, altrimenti viene considerato come commento.

IL FILE DOCKERIGNORE - doc

Il file dockerignore può essere usato per escludere file e cartelle dal contesto del build. Questo permette di evitare di inviare file e cartelle indesiderate o non necessarie al builder, migliorando la velocità del build.

```
# .dockerignore
node_modules
bar
```

Quando si lancia il comando build, il client cerca il file dockerignore nella root. Se esiste, i files e le cartelle che corrispondono ai percorsi indicati nel file, sono rimossi prima che l'immagine sia mandata al build.

SCRIVERE UN DOCKERFILE

I. STRUTTURA DEL PROGETTO

Serve innanzitutto definire una struttura minima del progetto. Supponiamo di voler costruire una semplice applicazione che usa Python per stampare un messaggio in console.

```
myapp/  
├─ Dockerfile  
├─ requirements.txt  
└─ src/  
    └─ main.py
```

I. CREAZIONE CARTELLA DEL PROGETTO E FILE

Creare una cartella **myapp** ed i file specificati nel progetto.

III. SCRIVERE IL CONTENUTO DEL DOCKERFILE

```
# 1. IMMAGINE BASE  
FROM python:3.13-slim  
  
# 2. CARTELLA DI LAVORO DEL CONTAINER  
WORKDIR /usr/local/app  
  
#install the application dependencies  
COPY requirements.txt ./  
RUN pip install --no-cache-dir -r requirements.txt  
  
#Copy in the source code  
COPY src ./src  
EXPOSE 5000  
  
#setup an app user so the container doesn't run as the root user  
RUN useradd app  
USER app  
CMD ["python", "src/main.py"]
```

IV. SCRIVERE IL CONTENUTO DI requirements.txt PER LE DIPENDENZE

Contiene elenco dei pacchetti da installare.

```
flask==2.3.3
```

V. SCRIVERE IL FILE SRC/MAIN.PY

Contenente una semplice app Flask di esempio.

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "Ciao dal container Flask!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

VI. BUILD DELL'IMMAGINE

Una volta che tutti i file sono pronti, possiamo costruire l'immagine.

Aprire il terminale nella cartella myapp/ e lanciare il comando:

```
docker build -t simple-flask-app .
```

```
[+] Building 24.4s (12/12) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 447B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.13-slim 2.9s
=> [auth] library/python:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                       0.0s
=> [2/6] WORKDIR /usr/local/app                                    0.7s
=> [3/6] COPY requirements.txt ./                                  0.2s
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt        6.3s
=> [5/6] COPY src ./src                                           0.3s
=> [6/6] RUN useradd app                                           1.0s
=> exporting to image                                              2.7s
=> => exporting layers                                              1.7s
=> => exporting manifest sha256:42ecf771a990ec1bbef204f57fe91daeae7ec9f09a5901b2b2922ad3b990e457 0.0s
=> => exporting config sha256:f5f2054e6023782b87a78022c24eb97cae30c2bdc1766524bd1d543f2881fb86 0.0s
=> => exporting attestation manifest sha256:ee8b6039338ac958148f6fd640502f5200b4e8d77fefc8aa36ff37f0da 0.0s
=> => exporting manifest list sha256:beec28808035c9c41e23a96241d03e0b23dce923600e8311dd95a197fe8e6b8a 0.0s
=> => naming to docker.io/library/simple-flask-app:latest          0.0s
=> => unpacking to docker.io/library/simple-flask-app:latest       0.7s
```

VII. AVVIARE IL CONTAINER

```
docker run -d -p 5000:5000 --name flask-container simple-flask-app
```

Crea ed avvia il container flask-container

VIII. APRIRE IL BROWSER

☐		tecnativa/docker-socket-proxy	latest	9e4b9e7517a6	11 months ago	30.63 MB			
☐		alpine	latest	4bcff63911fc	22 days ago	12.77 MB			
☐		simple-flask-app	latest	beec28808035	2 minutes ago	201.25 MB			

☐		flask-container	4bbd6a0d8cf2	simple-flask-app	5000:5000 
--------------------------	---	-----------------	--------------	----------------------------------	---

Esempio Dockerfile per progetto node

```
# Usa un'immagine Node.js ufficiale
FROM node:20-alpine

# Imposta la cartella di lavoro
WORKDIR /app

# Copia i file package.json e package-lock.json
COPY package*.json ./

# Installa le dipendenze
RUN npm install --production

# Copia il resto del codice
COPY . .

# Espone la porta 5511
EXPOSE 5511

# Avvia il server
CMD ["npm", "start"]
```

```
#NestJs
FROM node:22-alpine

WORKDIR /usr/src/app

COPY package*.json ./

RUN npm install --legacy-peer-deps

COPY . .

RUN npm run build

EXPOSE 3000

CMD ["npm", "run", "start:prod"]
```

`docker build -t app-gid .`

`docker run --name app-g -p 3000:3000 app-gid`

docker build

Il comando `docker build` **costruisce un'immagine docker a partire da un Dockerfile e dal contesto** (file nella cartella corrente o specificata. Se nel Dockerfile c'è un `COPY..`, Docker copierà tutto il contenuto del contesto nel container di build).

Legge le istruzioni presenti nel Dockerfile, le esegue in sequenza e produce un'immagine pronta da eseguire con **docker run**. Per ogni istruzione viene creato un nuovo layer.

Ogni layer è cacheabile: **se nulla cambia in quel passaggio, Docker riusa il layer precedente per velocizzare**. Alla fine viene generata un'immagine composta da tutti i layer.

Sintassi di base:

docker build [opzioni] **PATH | URL | -**

PATH: cartella dove si trova il Dockerfile (di default `./Dockerfile`).

Si può passare un **URL** o usare **-** per leggere il Dockerfile da stdin

docker build -t nome-immagine[:tag] .

- **-t: assegna un nome** ed, opzionalmente, una versione (tag) all'immagine. Il nome può includere anche un namespace (es. `utente/mia-immagine`). Se la versione viene omessa, Docker usa `:latest` come default
- **.** : indica il contesto di build (in questo caso directory corrente dove si trova il Dockerfile).

ESEMPIO:

Dockerfile

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "index.js"]
```

```
docker build -t mia-app-node .
```

Verrà creata un'immagine chiamata `mia-app-node` con Node.js installato e il nostro codice copiato.

Il risultato è dunque un'immagine che può essere eseguita, versionata e distribuita.

ESEMPIO CON ARGOMENTI:

Dockerfile

```
FROM node:20
ARG PORT=3000
ENV PORT=$PORT
WORKDIR /app
COPY . .
RUN npm install
EXPOSE $PORT
CMD ["npm", "start"]
```

```
docker build -t mia-app --build-arg PORT=8080 .
```

OPZIONE	DESCRIZIONE
<u>--annotation</u>	Aggiunge note all'immagine
<u>--build-arg</u>	Imposta variabili per il build
<u>--build-context</u>	Build context aggiuntivi (e.g., name=path)
<u>-f, --file</u>	Nome del Dockerfile (default: PATH/Dockerfile)
<u>--label</u>	Imposta metadata per una immagine
<u>--network</u>	Imposta la modalità di rete per le istruzioni RUN durante la build
<u>--no-cache</u>	Non usare la cache quando si costruisce l'immagine
<u>-o, --output</u>	Destinazione di output (format: type=local,dest=path)
<u>--platform</u>	Imposta la piattaforma di destinazione per la build per costruire immagini per architettura diversa da quella della macchina locale
<u>--progress</u>	Imposta lo stile dell'output (auto, quiet, plain, tty, rawjson). Usare plain per mostrare il contenuto dell'output
<u>--pull</u>	Cerca sempre di scaricare tutte le immagini referenziate da remote, anche se presenti localmente
<u>--push</u>	Shorthand per --output=type=registry
<u>-t, --tag</u>	Nome e tag opzionale (format: name:tag)
<u>--target</u>	Imposta lo stage di build di destinazione – per build multistage

DEBUG

Usare `--progress=plain` per vedere log dettagliati

Aggiungere `RUN echo "step"` nei punti chiave del Dockerfile (ad esempio il FROM, COPY/ADD, RUN, ecc.)

ESEMPIO:

`RUN echo "Arrivato qui: dopo installazione"`

Oppure, in fase di test

`RUN ls -l /app`

Eseguire questo comando dentro il container in fase di build per mostrare l'elenco dei file e delle cartelle nella directory corrente, in formato dettagliato.

```
-rw-r--r-- 1 root root 1200 Aug 7 09:12 index.js
drwxr-xr-x 2 root root 4096 Aug 7 09:12 node_modules
```

COSA FARE CON IMMAGINE CREATA?

Dopo la build è possibile:

- eseguirla: `docker run mia-app`
- salvarla: `docker save`
Esporta un'immagine Docker in un file tar (es. immagine.tar) che può essere trasferito o archiviato.
- caricarla: `docker load`
Importa un'immagine Docker da un file .tar
- Per il push:
 - `docker tag` mia_immagine:latest
mio_username/mia_immagine:latest
 - `docker push` mio_username/mia_immagine:latest

docker inspect Ispezionare un container / volume / network / image

- Serve per **vedere tutti i dettagli tecnici** di un oggetto Docker in formato JSON.
- Può ispezionare:
- container
- immagini
- volumi
- network
- `docker inspect NOME_O_ID`

```
> docker inspect mycontainer
[
  {
    "Id":
      "fc723f18f20b583f8a6fb61f8ba15309e86adad
      dde11e2bba388e43b89a7c7dc",
    "Created": "2026-04-
      25T08:25:48.917565712Z",
    "Path": "sh",
    "Args": [],
    "State": {
      "Status": "exited",
      "Running": false,
      "Paused": false,
```

VOLUMI

Quando si lavora con Docker, uno degli aspetti fondamentali è capire come gestire i dati generati o utilizzati dai container. I volumi Docker sono una delle soluzioni principali per garantire che questi **dati persistano anche quando il container viene arrestato o eliminato**.

COSA SONO

Un volume è uno **spazio di archiviazione gestito direttamente da Docker**, separato dal filesystem interno del container.

Questo significa che i dati memorizzati in un volume **non vengono cancellati quando il container viene rimosso**, a differenza dei dati scritti direttamente nel filesystem del container, che invece sono temporanei e scompaiono con il container.

Docker salva i volumi in una cartella dedicata sul sistema host, solitamente sotto il percorso:

```
/var/lib/docker/volumes/
```

Questa separazione è fondamentale perché permette ai dati di sopravvivere al ciclo di vita del container, dando così una vera **persistenza**.

PERCHÉ USARE I VOLUMI DOCKER?

- **Persistenza indipendente dal container**: anche se il container viene eliminato o aggiornato, i dati rimangono intatti nel volume, pronti per essere riutilizzati da un altro container.
- **Condivisione tra più container**: più container possono montare lo stesso volume e quindi accedere agli stessi dati. Comodo in scenari di lavoro distribuito o quando si vuole separare i dati dalla logica applicativa.
- **Portabilità e isolamento**: i volumi sono gestiti da Docker, che si occupa di mantenerli isolati dal filesystem dell'host, rendendo più semplice spostare, fare backup o sincronizzare i dati senza rischiare di toccare altre parti del sistema operativo.
- Non è legato direttamente ad un percorso dell'host

I volumi possono essere:

- **Volume nominato**: ha un nome scelto
- **Volume anonimo**: nome generato automaticamente da Docker (casuale e unico sul sistema).

Non vengono eliminati se il container viene rimosso. Il flag **--rm** nel comando **docker run**, però, **elimina il volume anonimo insieme al container**.

Se si creano più container con volumi anonimi, ognuno avrà il suo volume diverso. **Non vengono condivisi in automatico**. Per condividere un volume anonimo, usare il suo ID.

GESTIONE DEI VOLUMI – COMANDI PRINCIPALI [doc](#)

- **docker volume create nome_volume**
crea un nuovo volume con il nome specificato.
Quando si crea un volume, Docker lo salva in una cartella dedicata all'interno del filesystem della macchina host. Quando monti il volume in un container, questa cartella viene "condivisa" con il container, permettendo così di leggere e scrivere dati in modo persistente.

- **docker volume ls**
mostra la **lista** di tutti i volumi presenti nel sistema
- **docker volume inspect nome_volume**
fornisce informazioni dettagliate su un volume, come il percorso sul filesystem host e altre proprietà
- **docker volume rm nome_volume**
elimina un volume, purché non sia montato da alcun container
- **docker volume prune**
elimina tutti i volumi che non sono più usati da nessun container, utile per fare pulizia.

```
$ docker volume inspect my-vol
[
  {
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/my-vol/_data",
    "Name": "my-vol",
    "Options": {},
    "Scope": "local"
  }
]
```

Per vedere il contenuto di un volume

- Se non esiste un container che lo utilizza:
- `docker run --rm -it -v ss_storage:/data --name=check_storage_volume alpine sh`
- `--rm` 🖐️ il container viene rimosso automaticamente alla sua chiusura (quando esci dalla shell).
- `-it` 🖐️ modalità interattiva con TTY (per aprire sh).
- `-v ss_storage:/data` 🖐️ monta il volume `ss_storage` dentro `/data`.
- `--name=check_storage_volume` 🖐️ assegna un nome al container (`check_storage_volume`).
- `alpine sh` 🖐️ esegue l'immagine `alpine` con la shell `sh`.

- Per sapere se lo utilizza un container:
- `> docker inspect nome_container`

```
PS C:\Users\HP> docker inspect test_node_app2_container
```

Usare --volume

Il flag `--volume` o `-v`, consiste in **tre campi**, separati da due punti :

I campi devono essere **nell'ordine corretto**.

```
$ docker run -v [<volume-name>:]<mount-path>[:opts]
```

In caso di volumi nominati, il primo campo è occupato dal nome del volume. Per i volumi anonimi, il primo campo è omissivo.

Il secondo campo è il path dove il file o la directory vengono montati nel container.

Il terzo campo è opzionale ed è una lista di opzioni separate da virgola. Opzioni disponibili sono:

- Readonly, ro: il volume è montato nel container in sola lettura
- Volume-nocopy: i dati della destinazione non sono copiati nel volume se il volume è vuoto. Di default, il contenuto nella destinazione finale, viene copiato all'interno di un volume montato, se vuoto.

```
$ docker run -v myvolume:/data:ro
```

BIND MOUNTS

COSA È UN MOUNT

In generale, il termine **mount** indica l'operazione di **collegare (montare) uno spazio di archiviazione** (come una cartella o un disco) **all'interno del filesystem** di un sistema operativo o di un container.

In Docker, **mount** si riferisce a **qualsiasi tipo di collegamento di dati esterni dentro un container**.

BIND MOUNTS

Un **bind mount** è un tipo di mount in cui **una directory o file esistente nel filesystem della macchina host viene montato direttamente all'interno del container**, mantenendo la stessa struttura e contenuto.

Al contrario, quando usi un volume, **Docker crea una nuova cartella** nella sua directory di archiviazione e ne gestisce direttamente i contenuti.

Caratteristiche principali:

- La directory **esiste già** sulla macchina host.
- Il **container accede direttamente** a quella stessa cartella o file.
- Qualsiasi **modifica fatta dal container** si riflette immediatamente **sulla macchina host (e viceversa)**.
- Dipende dalla struttura e permessi del filesystem host.
- Veloce e semplice da configurare.
- Utile **per:**
 - **lavorare direttamente su file presenti sull'host** (ad esempio codice sorgente, file di configurazione).
 - **Condividere file di configurazione** dalla macchina host ai container. Ad esempio, Docker fornisce la risoluzione DNS ai container montando `/etc/resolv.conf` dall'host all'interno di ogni container.
 - le build perché permettono di **montare il codice sorgente dell'host** nel container di build per testare, eseguire linting o compilare un progetto.

BIND MOUNT SU DATI ESISTENTI

Se si monta una cartella o un file in un punto del container dove ci sono già dati, questi vengono nascosti.

È come quando su Linux monti una chiavetta USB su una cartella: i file originali spariscono finché non smonti la chiavetta.

Nei container, per vedere di nuovo i file nascosti, devi ricreare il container senza il mount.

CONSIDERAZIONI E VINCOLI

I bind mount per impostazione predefinita **hanno accesso in scrittura ai file sull'host**.

È quindi possibile modificare il filesystem dell'host tramite processi che girano nel container, inclusa la possibilità di **creare, modificare o eliminare file o cartelle di sistema**.

Questo può avere implicazioni sulla sicurezza, ad esempio può interferire con altri processi non Docker presenti sull'host.

È possibile usare l'opzione **readonly** o **ro** per **impedire la scrittura** da parte del container sul mount.

ARCHITETTURE

I bind mount sono creati **sull'host in cui gira il Docker daemon, non sulla macchina da cui lanci il comando (client)**.

Se Docker gira su una macchina remota, non si possono montare file della macchina locale nel container.

Con Docker Desktop, il demone Docker gira all'interno di una VM Linux, ma gestisce automaticamente i bind mount, rendendo visibili i file dell'host nei container.

LEGAME STRETTO CON L'HOST

- I container che usano bind mount sono **fortemente legati alla macchina host**.
- I bind mount **dipendono dalla struttura del filesystem dell'host**. Questo significa che i container potrebbero non funzionare correttamente su host diversi, se la struttura delle cartelle non è identica.

Differenza tra VOLUME e BIND_MOUNT

- ◆ Bind Mount

- Cosa fa: Collega una cartella del filesystem host (es. Windows) a una cartella dentro il container.
- Percorso reale: È una cartella normale su Windows (es. C:\Users\HP\progetto).
- Visibile in Esplora Risorse 
- Uso tipico: sviluppo, così puoi modificare i file su Windows e vederli subito aggiornati nel container.
-  Esempio:
 - `docker run -it --rm -v C:\Users\HP\progetto:/app ubuntu bash`
 - Dentro il container, /app punta alla cartella C:\Users\HP\progetto
 - Se modifichi file da Windows → li vedi nel container
 - Se modifichi file dal container → li vedi in Windows

- Volume

- Cosa fa: Collega una cartella gestita da Docker a una cartella nel container.
- Percorso reale su Windows: dentro la distro WSL docker-desktop-data, quindi non lo trovi direttamente in C:\.
- (Esplorabile solo via `\\wsl$\docker-desktop-data\version-pack-data\community\docker\volumes\...`)
- Non visibile in Esplora Risorse di default 
- Uso tipico: dati persistenti di database o app (MySQL, PostgreSQL, Redis...), dove non ti interessa accedere da Windows, ma vuoi che i dati rimangano anche se il container muore.
-  Esempio:
 - `docker run -it --rm -v mio_volume:/app ubuntu bash`
 - Docker crea (se non esiste) un volume chiamato mio_volume
 - Dentro il container /app punta a quel volume
 - I dati rimangono anche se il container viene cancellato
 - Ma su Windows non vedi subito i file, a meno di entrare via WSL o usare un container di ispezione

```
docker run --rm -it -v C:\Software\Docker\shared_volume:/data alpine sh
```

OPZIONI PER --volume (-v)

Composto da **tre campi separati da due punti** ..

I campi **devono essere in ordine corretto**:

```
docker run -v <percorso-host>:<percorso-container>[:opzioni]
```

- I. percorso sull'host da montare.
- II. percorso nel container
- III. (opzionale) elenco di opzioni separate da virgole

Opzioni valide per --volume con un bind mount:

- **readonly, ro**: monta in sola lettura
- **z, Z**: configura le etichette SELinux
- **rprivate**: propagazione bind impostata su rprivate (default)
- **private**: propagazione impostata su private
- **rshared**: propagazione impostata su rshared
- **shared**: propagazione impostata su shared
- **rslave**: propagazione impostata su rslave
- **slave**: propagazione impostata su slave.

PROPAGAZIONE: come i cambiamenti in un filesystem montato si propagano tra i vari mount condivisi tra container e host, o tra container diversi

AVVIA UN CONTAINER CON UN BIND MOUNT

Abbiamo una cartella source che, dopo la compilazione del codice, genera file nella sottocartella source/target/. Si vogliono rendere questi file accessibili dal container nella cartella **/app/**, così ogni volta che si ricompila, il container vede i file aggiornati.

Usare uno dei due comandi (da dentro la cartella source):

```
docker run -d \
  -it \
  --name devtest \
  --mount type=bind,source="$(pwd)"/target,target=/app \
  nginx:latest
```

```
docker run -d \
  -it \
  --name devtest \
  -v "$(pwd)"/target:/app \
  nginx:latest
```

Per controllare che il mount sia corretto:

```
docker inspect devtest
```

```
"Mounts": [  
  {  
    "Type": "bind",  
    "Source": "/percorso/host/target",  
    "Destination": "/app",  
    "RW": true,  
    "Propagation": "rprivate"  
  }  
]
```

BIND MOUNT IN SOLA LETTURA

Affinché il container legga i file ma non li modifichi, montare la cartella come sola lettura con readonly (oppure ro).

```
docker run -d \  
-it \  
--name devtest \  
--mount type=bind,source="$(pwd)"/target,target=/app,readonly \  
nginx:latest
```

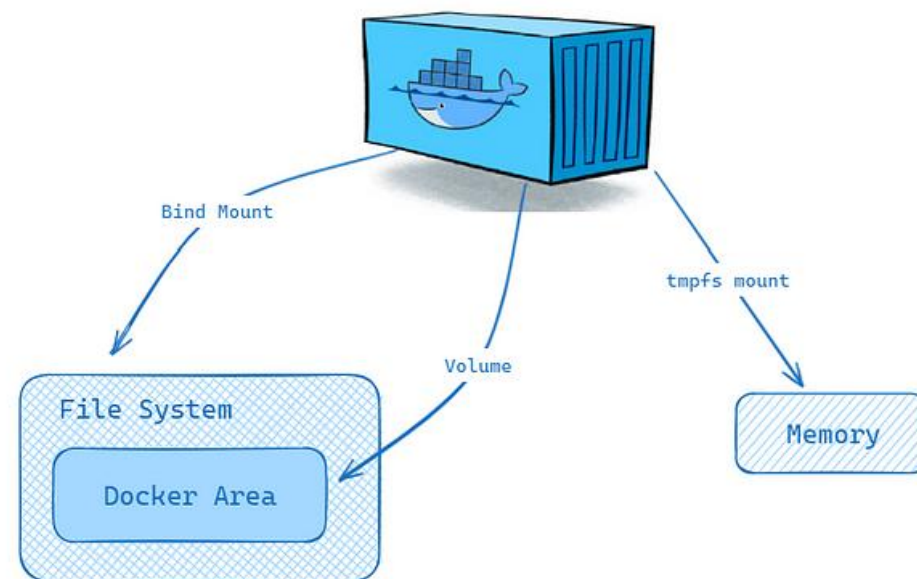
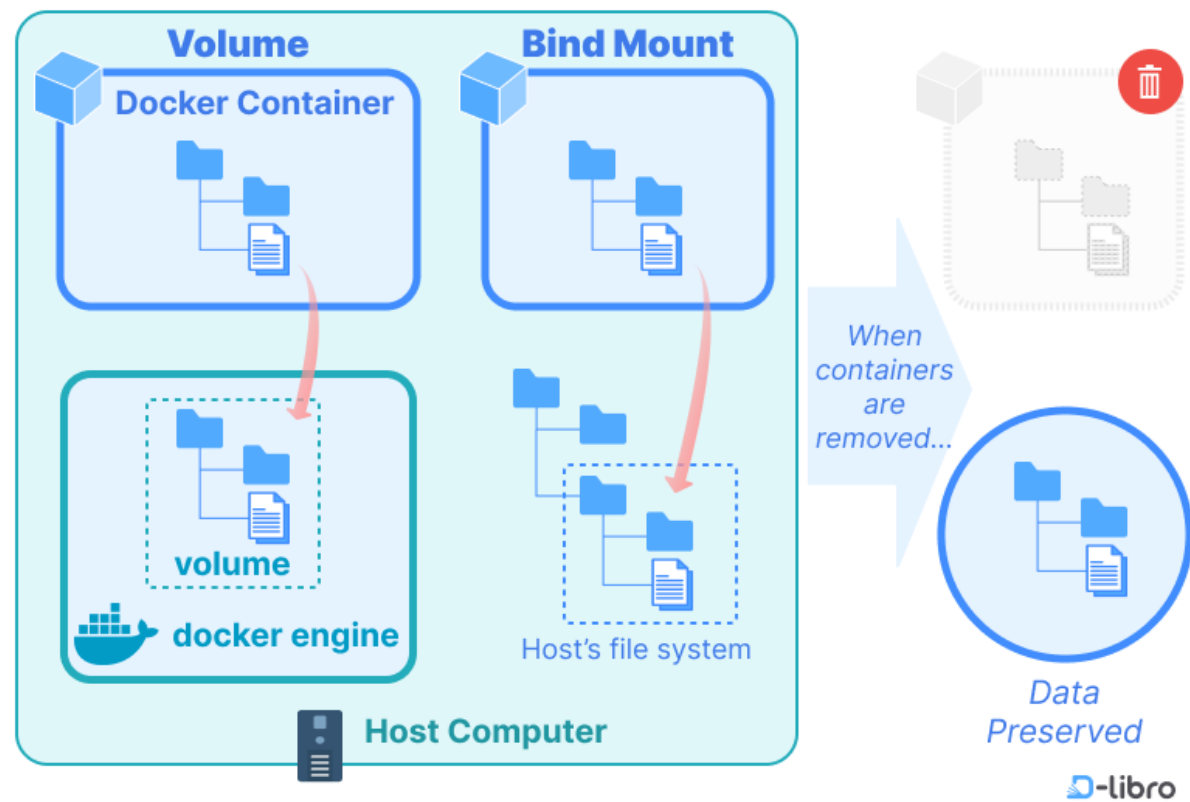
```
docker run -d \  
-it \  
--name devtest \  
-v "$(pwd)"/target:/app:ro \  
nginx:latest
```

DIFFERENZE TRA VOLUMI E BIND MOUNTS

CARATTERISTICA	BIND MOUNTS	VOLUMI DOCKER
GESTIONE	Manuale , dipende dal filesystem host	Gestito da Docker
POSIZIONE DATI	Cartella/ file esistente sull'host	Cartella isolata gestita da Docker
PERSISTENZA	Sì, ma legata alla cartella host	Sì, indipendente dai container
CONDIVISIONE TRA CONTAINER	Sì	Sì
BACKUP / MIGRAZIONE	Più complesso	Più semplice
PORTABILITÀ	Limitata al filesystem host	Alta, può essere facilmente spostato
ACCESSO HOST	Diretto	No, isolato da Docker
USO CONSIGLIATO	Sviluppo , accesso a file host	Produzione , dati persistenti, backup

In entrambi i casi si parla di **mounting** di dati nel container, ma **cambia come e dove sono gestiti i dati**

Docker Persistent Storage



Esempio di bind mount su cartella locale con dockerfile

- Dockerfile
 - FROM alpine:latest
 - WORKDIR /app
 - VOLUME /miovol
 - CMD ["sh"]
- `docker build -t ss_test_volume .`
- `docker run --name ss_test_volume -v c:\Software\Docker\dati_condivisi:/miovol -it ss_test_volume`
- Se avessi aggiunto come parametri `-d` avrei poi dovuto fare:
`docker exec -it ss_test_volume sh`
- In questo caso viene scaricata immagine alpine, creato container `ss_test_volume`, create cartella `/app` e `/dati` se eseguo
- `cd miovol`
- `touch miofile.txt` (oppure `cat` oppure `gc`)
- viene creato nella cartella `c:\Software\Docker\dati_condivisi`

BACKUP E RESTORE

Cosa può essere sottoposto a backup in Docker?

ELEMENTO	DESCRIZIONE	SI PUÒ FARE BACKUP?	COME?
◆ VOLUMI	Dati persistenti dei container (es. DB, file)	✓	docker run + tar, docker cp, snapshot
◆ CONTAINER	I container in esecuzione (stato attuale, filesystem live)	⚠ Parziale	Meglio ricrearli da immagini
◆ IMMAGINI	Tutte le immagini scaricate o create	✓	docker save e docker load
◆ CONFIGURAZIONI	File Compose (docker-compose.yml), .env, script	✓	Backup dei file in Git o ZIP
◆ BIND MOUNTS	Cartelle mappate direttamente dall'host	✓	Backup classico dal filesystem host
◆ REGISTRY LOCALE	Registry Docker privato con immagini custom	✓	docker save, backup del volume del registry

DOCKER COMPOSE

Strumento per **definire ed eseguire applicazioni multi-container**.

Permette di **gestire l'intero stack della tua applicazione** (servizi, reti, volumi, variabili d'ambiente, ecc.) in un **unico file di configurazione YAML** chiamato **docker-compose.yml**.

Già incluso in Docker Desktop

Perché usare Docker Compose?

Semplifica sviluppo, distribuzione e gestione delle applicazioni containerizzate.

- **CONTROLLO SEMPLIFICATO:** Gestione applicazioni multi-container in un unico file YAML, facilitando la configurazione e la replica degli ambienti.
- **COLLABORAZIONE EFFICIENTE:** I file YAML sono **condivisibili** e leggibili, quindi facilitano la collaborazione tra sviluppatori e team DevOps, migliorando il lavoro di squadra e la risoluzione dei problemi.

- **SVILUPPO RAPIDO:** **riutilizza i container** già creati se non ci sono modifiche. Questo accelera il riavvio dei servizi e l'applicazione delle modifiche all'ambiente.
- **PORTABILITÀ TRA AMBIENTI:** Puoi usare **variabili** nel file Compose per adattare la configurazione a diversi ambienti (es. sviluppo, test, produzione) o utenti.

Funziona in **qualsiasi ambiente**: produzione, staging (pre-produzione), sviluppo, test e anche nei flussi di lavoro CI (integrazione continua).

Inoltre, dispone di **comandi per gestire l'intero ciclo di vita della tua applicazione**, tra cui:

- **Avviare, fermare e ricostruire** i servizi
- **Visualizzare lo stato** dei servizi in esecuzione
- **Mostrare in tempo reale i log** dei servizi
- **Eeguire comandi una tantum** su un servizio (es. migrazioni, test, script)

Vecchio e nuovo comando

- Differenza fra docker-compose e docker compose
- ♦ docker-compose (con il trattino -)
- È il vecchio comando che si installava come binario separato (docker-compose.exe su Windows).
- Per anni è stato lo standard:
- docker-compose up
- docker-compose down
- ♦ docker compose (senza trattino, sottocomando di docker)
- È la nuova integrazione ufficiale di Docker Compose nella Docker CLI.
- Arrivata con Docker Desktop e Docker Engine più recenti.

```
PS C:\Software\Docker\test_volume> docker compose version
>>
Docker Compose version v2.23.3-desktop.2
PS C:\Software\Docker\test_volume> docker-compose version
>>
Docker Compose version v2.23.3-desktop.2
PS C:\Software\Docker\test_volume>
```

USI COMUNI DI DOCKER COMPOSE

- **AMBIENTI DI SVILUPPO**

Permette di **avviare facilmente l'intero ambiente di sviluppo** (es. app + database + cache + API) in modo isolato, con un solo comando. Tutto è infatti documentato nel file YAML

docker compose up

- **AMBIENTI DI TEST AUTOMATICI**

Nel CI/CD, è utile creare ambienti temporanei per eseguire i test. Con Compose si possono creare e distruggere ambienti di test facilmente ed in modo ripetibile:

```
docker compose up -d
./run_tests
docker compose down
```

- **AMBIENTI DI PRODUZIONE**

Anche se Compose è nato per sviluppo e test, oggi supporta sempre più **funzionalità orientate alla produzione**.

COME FUNZIONA COMPOSE

Viene usato un **file YAML** (chiamato Compose file) per **configurare i servizi dell'applicazione**.

Una volta definito, si crea e avvia tutto con un singolo comando della CLI di Docker Compose.

- **IL FILE COMPOSE (compose.yaml)**

Il file compose.yaml segue le regole della [Compose Specification](#), lo standard ufficiale per definire applicazioni multi-container.

Questo file è usato per definire e configurare servizi, reti e volumi in una forma dichiarativa.

Il nome predefinito è compose.yaml (preferito) o compose.yml.

Sono ancora supportati i vecchi nomi **docker-compose.yaml** e **docker-compose.yml** per compatibilità.

Se ci sono più file, Compose preferisce **compose.yaml**. Puoi mantenere il file ordinato e riutilizzabile con frammenti, estensioni, o più file Compose combinati insieme.

SINTASSI DEL FILE `compose.yaml`

- **TOP LEVEL ELEMENTS: NAME E VERSION** (obsoleta)
Il campo **version** in cima al file `docker-compose.yaml` / `compose.yaml` è **obsoleto** ed è stato **mantenuto solo per compatibilità**. Se usato, Docker Compose mostrerà avviso che è obsoleto.
Docker Compose ignora il valore di `version` e **utilizza sempre l'ultima versione dello schema disponibile** per validare il file.

Il campo **name** in cima al file serve per **impostare il nome del progetto**, nel caso non sia impostato esplicitamente da riga di comando o tramite variabile d'ambiente. Se non specificato, Compose userà il nome della cartella come nome di progetto per default.

Il nome del progetto diventa disponibile automaticamente come **variabile** chiamata **COMPOSE_PROJECT_NAME**, che puoi usare nel file stesso.

```
name: myapp

services:
  foo:
    image: busybox
    command: echo "I'm running ${COMPOSE_PROJECT_NAME}"
```

Verrà avviato un container che stampa a video:

```
I'm running myapp
```

- **SERVICES TOP LEVEL ELEMENTS**

Un servizio è una componente di un'applicazione, che può essere scalata (più container dello stesso tipo) o sostituita indipendentemente da altri componenti.

Poiché i servizi sono basati su container, sono definiti da una Docker image e da un insieme di argomenti di runtime.

Un file Compose **DEVE DICHIARARE UN ELEMENTO** di livello superiore chiamato **services**, che è una mappa in cui:

- Le chiavi sono rappresentazioni testuali dei nomi dei servizi
- I valori sono le definizioni dei servizi. Una definizione di servizio contiene la configurazione che viene applicata a ciascun container del servizio.

TUTTI i container di un servizio sono IDENTICI (stessa image e config)

Si possono avere **più file Compose per ambienti diversi** (es. compose.production.yaml)

Nginx: web server, serve file statici più velocemente e con ssl

```

> Run All Services
services:
  > Run Service
  proxy:
    # il servizio proxy usa l'immagine nginx
    image: nginx
    # monta un file locale di configurazione Nginx nel container
    volumes:
      - type: bind
        source: ./proxy/nginx.conf
        target: /etc/nginx/conf.d/default.conf
        read_only: true
    # espone la porta 80
    ports:
      - 80:80
    # dipende dal servizio backend
    depends_on:
      - backend

# STRUTTURA BASE DOCKER-COMPOSE.YAML
> Run All Services
services:
  > Run Service
  nome_servizio:
    image: nome_immagine:tag
    ports:
      - "8080:80"
    environment:
      - VAR=valore
    volumes:
      - ./dati:/app/dati
    networks:
      - rete-interna
  > Run Service
  backend:
    build:
      context: backend
      target: builder
```

ELEMENTI PRINCIPALI DI UN SERVIZIO

[lista completa](#)

Sezione	Descrizione
image	Specifica l'immagine da usare per creare il container
build	(Opzionale) Invece di image, puoi definire come costruire l'immagine
ports	Mappa porte host:container per l'accesso esterno
volumes	Monta volumi (persistenza dati, codice, ecc.)
environment	Definisce variabili d'ambiente per il container
networks	Specifica a quali reti appartiene il servizio
deploy	(Opzionale) Requisiti di esecuzione per ambienti in produzione (es. Swarm)
Restart	Definisce le policy di riavvio (always, ecc.)
depends_on	Stabilisce l'ordine di avvio tra i servizi

ATTRIBUTI AVANZATI UTILI IN PRODUZIONE

logging	Configura driver (es. syslog) e opzioni di log
network_mode	Definisce come container accede alla rete (es. host, none, service:<name>)
mac_address	Imposta manualmente l'indirizzo MAC del container
mem_limit, mem_reservation, mem_swappiness, memswap_limit	Consentono di gestire risorse, swap e priorità di memoria

• NETWORKS TOP LEVEL ELEMENTS

permettono ai container dei vari **servizi** di comunicare tra loro. Compose crea **una rete di default** se non ne specifichi una.

Ogni container di un servizio si connette alla rete di default ed è

- sia raggiungibile dagli altri container su quella rete,
- sia individuabile tramite il nome del servizio.

L'elemento top level networks permette di configurare reti nominate che possono essere riutilizzate da più servizi.

Per usare una rete condivisa tra più servizi, è necessario **concedere esplicitamente a ciascun servizio l'accesso tramite l'attributo networks all'interno della sezione dei servizi.**

```
# Le reti front-tier e back-tier sono create a runtime
# Il servizio frontend è collegato ad entrambe
▶ Run All Services
services:
  ▶ Run Service
  frontend:
    image: example/webapp
    networks:
      - front-tier
      - back-tier

networks:
  front-tier:
76 back-tier:
```

PRINCIPALI ATTRIBUTI DI NETWORKS

driver	Tipo di driver della rete (es. bridge, overlay, custom-driver)
driver_opts	Opzioni specifiche del driver come coppie chiave/valore
attachable	Se true, permette a container non gestiti da Compose di collegarsi alla rete
enable_ipv4	Se false, disattiva l'uso di IPv4 (<i>richiede Docker Compose ≥ 2.33.1</i>)
enable_ipv6	Se true, abilita l'indirizzamento IPv6
external	Se true, Docker non crea la rete, ma usa una già esistente
ipam	Specifica una configurazione IPAM personalizzata (subnet, gateway, range IP, opzioni...) Se true, la rete è isolata dall'esterno (nessun accesso a/da internet o altri container fuori dalla rete)
internal	
labels	Aggiunge metadati alla rete sotto forma di etichette (es. com.example.description: "...")
name	Assegna un nome personalizzato alla rete (può essere usato con external)

- **VOLUME TOP LEVEL ELEMENTS**

Permette di configurare volumi nominati che possono essere riutilizzati da più servizi.

Per utilizzare un volume condiviso da più servizi, è necessario concedere esplicitamente a ciascun servizio l'accesso tramite attributo volumes all'interno della sezione dei servizi.

Quando si esegue il comando docker compose up, il volume db-data viene creato se non esiste già.

Se il volume esiste, viene usato tale volume; se viene cancellato manualmente al di fuori di Compose, allora viene ricreato automaticamente alla prossima esecuzione di docker compose up.

```
# la directory dati di un database viene condivisa
# con un altro servizio tramite un volume chiamato db-data,
# in modo che possa essere periodicamente eseguito un backup.
▷ Run All Services
services:
  ▷ Run Service
  backend:
    image: example/database
    volumes:
      - db-data:/etc/data

  ▷ Run Service
  backup:
    image: backup-service
    volumes:
      - db-data:/var/lib/backup/data

volumes:
  db-data:

# Il volume db-data viene montato nel percorso /etc/data nel
# container del servizio backend e nel percorso /var/lib/backup/data
# nel container del servizio backup.
```


ESEMPIO ILLUSTRATIVO

Pensiamo ad un'applicazione composta da:

- Un **frontend** (web app): configurato a runtime tramite un file di configurazione HTTP gestito da infrastruttura, che fornisce un nome di dominio esterno ed un certificato HTTPS
- Un **backend** (database) che memorizza i dati in un volume persistente
- Una configurazione HTTP (file)
- Un certificato HTTPS (secret)
- Un volume persistente per i dati
- Due reti Docker

Front e back comunicano tra loro tramite una rete isolata chiamata back-tier, mentre il frontend è connesso anche ad una rete front-tier ed espone la porta 443 per accesso esterno.

Il comando **compose up** avvia i servizi di frontend e di backend, crea le reti ed il volume, inietta la configurazione ed il certificato nel frontend.

Per visualizzare lo stato dei servizi si usa il comando **docker compose ps**

NAME	IMAGE	COMMAND	SERVICE	STATUS	PORTS
frontend-1	example/webapp	"nginx ..."	frontend	Up 2 min	0.0.0.0:443->8043/tcp
backend-1	example/database	"docker-entrypoint..."	backend	Up 2 min	

```
1  services:
2      > Run Service
3      frontend:
4          image: example/webapp
5          ports:
6              - "443:8043"
7          networks:
8              - front-tier
9              - back-tier
10         configs:
11             - httpd-config
12         secrets:
13             - server-certificate
14
15     > Run Service
16     backend:
17         image: example/database
18         volumes:
19             - db-data:/etc/data
20         networks:
21             - back-tier
22
23 volumes:
24     db-data:
25         driver: flocker
26         driver_opts:
27             size: "10GiB"
28
29 configs:
30     httpd-config:
31         external: true
32
33 secrets:
34     server-certificate:
35         external: true
36
37 networks:
38     front-tier: {}
39     back-tier: {}
```

Ma perché docker compose?

- Si può dire che Docker Compose è una "facility" (uno strumento di comodità) che ti evita di scrivere a mano tutti i comandi docker build, docker run, con le loro mille opzioni.
- In pratica:
- Con docker build / run puoi ottenere esattamente lo stesso risultato, ma dovresti scrivere comandi lunghi e ripeterli ogni volta (porte, volumi, variabili d'ambiente, network, dipendenze, ecc.).
- Con docker-compose.yml dichiarare in un file tutto quello che serve (servizi, immagini, build, volumi, network, environment, dipendenze) e poi lo lanci con un semplice:
- **I comandi da lanciare sono in verde**
- Come vedi sono **tanti comandi** e bisogna ricordarsi tutti i dettagli (volumi, porte, variabili d'ambiente, network, dipendenze...).

- Senza docker compose devo lanciare a console

```
# network
docker network create mynet

# database
docker run -d \
  --name db \
  --network mynet \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=mydb \
  -v db_data:/var/lib/mysql \
  mysql:8.0

# app flask
docker run -d \
  --name app \
  --network mynet \
  -p 5000:5000 \
  -e DB_HOST=db \
  -e DB_NAME=mydb \
  -e DB_USER=root \
  -e DB_PASS=secret \
  myflask:latest

# nginx
docker run -d \
  --name web \
  --network mynet \
  -p 80:80 \
  -v ./nginx.conf:/etc/nginx/nginx.conf \
  nginx:latest
```

Con docker compose devo lanciare a console

docker compose up -d

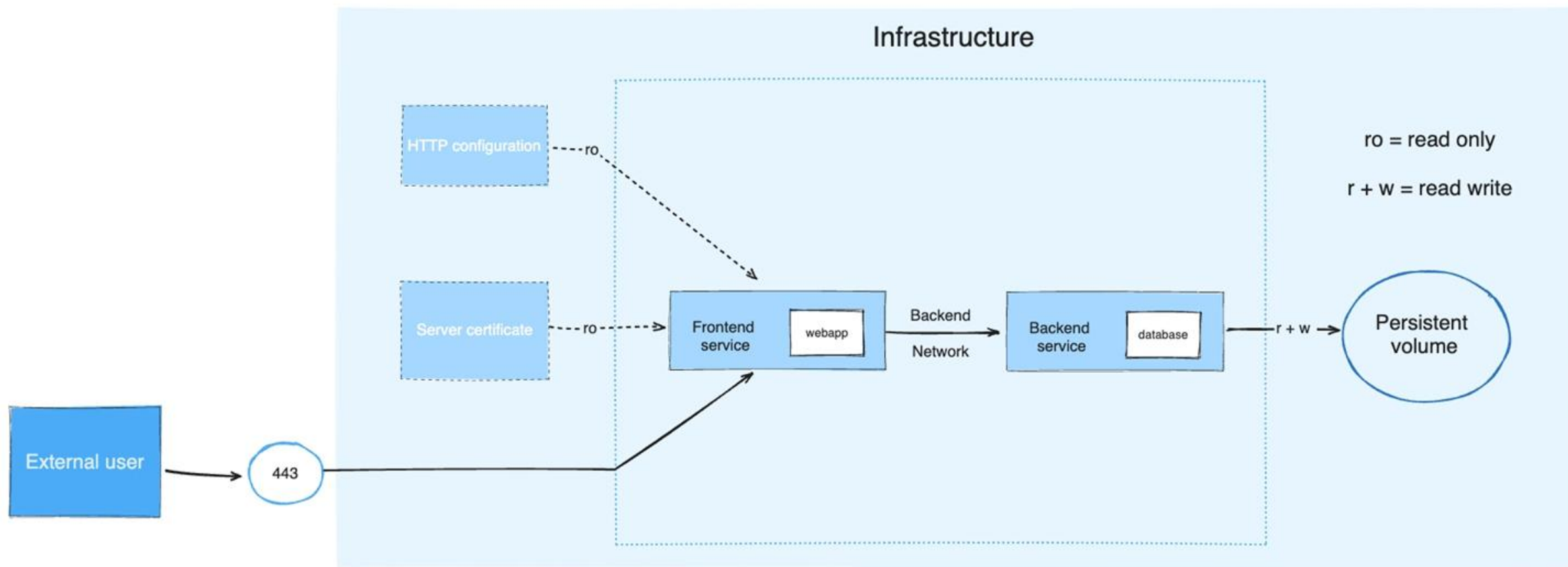
Docker compose.yml

```
version: "3.9"
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: mydb
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - mynet

  app:
    build: ./app # oppure image: myflask:latest
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_NAME: mydb
      DB_USER: root
      DB_PASS: secret
    depends_on:
      - db
    networks:
      - mynet

  web:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - app
    networks:
      - mynet

volumes:
  db_data:
```



COMANDI PRINCIPALI DOCKER COMPOSE

[Lista completa comandi docker compose](#)

FIATRE CONTAINER CONSULTING DOCKER COMPOSE

1. Docker compose up

- Avvia tutti i servizi definiti nel file docker-compose.yml (o docker-compose.yaml)
- Crea le reti, i volumi e i secret necessari se non esistono già
- Crea e avvia i container corrispondenti ai servizi

In pratica, con docker compose up l'intero ambiente descritto nel file Compose viene avviato in modo automatico e coordinato.

Modalità di esecuzione:

- **Foreground (default):** i messaggi di log dei container appaiono nel terminale. Il comando resta attivo finché non lo interrompi con Ctrl+C
- **Detached mode (-d):** I container partono in background, senza mostrare i log nel terminale. Il comando torna subito la shell, così si può usare il terminale mentre l'app gira "silenziosamente".

COSA FA ESATTAMENTE:

- I. **Legge il file docker-compose.yml** (e altri file opzionali come .env)
- II. **Crea tutte le risorse necessarie:** volumi, se specificati e non esistenti; reti definite; secret e config (se usati).
- III. **Costruisce le immagini** se non esiste un'immagine richiesta o se è stato specificato --build.
- IV. **Crea i container per ciascun servizio:** se il container esiste già, lo riusa (a meno che non sia stato rimosso).
- V. **Avvia i container:** i servizi partono secondo le loro dipendenze e configurazioni.
- VI. **Aggancia i log di output in tempo reale:** mostra tutto quello che stampano i container in esecuzione.

PRINCIPALI OPZIONI DEL COMANDO COMPOSE.UP

--abort-on-container-exit
--abort-on-container-failure
--always-recreate-deps
--attach
--attach-dependencies
--build
-d, --detach
--force-recreate
--no-attach
--no-build
--no-deps
--no-recreate
--no-start
--pull
--quiet-build
-V, --renew-anon-volumes
--scale
-t, --timeout
--timestamps
--wait-timeout
-w, --watch
-y, --yes

Ferma tutti i container se uno qualsiasi si ferma. Non compatibile con -d.

Ferma tutti i container se uno qualsiasi termina con errore. Non compatibile con -d.

Ricrea i container dipendenti. Non compatibile con --no-recreate.

Si collega solo ai servizi specificati. Non compatibile con --attach-dependencies.

Si collega automaticamente ai log dei servizi dipendenti.

Costruisce le immagini prima di avviare i container.

Modalità distaccata: esegue i container in background.

Ricrea i container anche se non ci sono cambiamenti nella configurazione o immagine.

Non si collega (non mostra i log) ai servizi specificati.

Non costruisce l'immagine, anche se la policy lo richiede.

Non avvia i servizi collegati.

Se i container esistono già, non li ricrea. Non compatibile con --force-recreate.

Crea i container ma non li avvia.

Scarica l'immagine prima di eseguire ("always", "missing", "never").

Sopprime l'output della build.

Ricrea i volumi anonimi invece di usare quelli esistenti.

Scala il servizio specificato a NUM istanze. Sovrascrive la configurazione nel file Compose.

Timeout per lo spegnimento dei container quando attaccati o già in esecuzione.

Mostra i timestamp nei log.

Tempo massimo per attendere che il progetto sia running/healthy.

Monitora il codice sorgente e ricostruisce/aggiorna i container al cambiamento dei file.

Assume sempre "sì" alle domande e esegue in modo non interattivo.

2. Docker compose down

Ferma i container e rimuove container, network, volumi e immagini create da up.

Per impostazione predefinita, vengono rimossi solo:

- I container dei servizi definiti nel file Compose.
- Le reti definite nella sezione networks del file Compose.
- La rete di default, se utilizzata.

Le reti e i volumi definiti come esterni non vengono mai rimossi.

I volumi anonimi non vengono rimossi di default.

Tuttavia, poiché non hanno un nome stabile, non vengono montati automaticamente da un successivo up.

Per dati che devono persistere tra aggiornamenti, si consiglia di usare percorsi espliciti come bind mount o volumi nominati.

PRINCIPALI OPZIONI DI compose down

--remove-orphans

Rimuove i container di **servizi non definiti** nel file Compose.

--rmi

Rimuove le immagini usate dai servizi. "local" rimuove solo le immagini senza tag personalizzato ("local" o "all").

-t, --timeout

Specifica il **timeout per lo spegnimento dei container**, in secondi.

-v, --volumes

Rimuove i volumi nominati dichiarati nella sezione volumes del file Compose e i volumi anonimi collegati ai container.

3. Docker compose logs

Mostra i logs prodotti dai servizi

PRINCIPALI OPZIONI COMANDO `compose logs`

-f, --follow	Segue i log in tempo reale (stream continuo).
--index	Specifica l'indice del container se il servizio ha più repliche.
--no-color	Produce output senza colori (monocromatico).
--no-log-prefix	Non stampa il prefisso (nome container, timestamp) nei log.
--since	Mostra i log a partire da un timestamp (es. 2013-01-02T13:23:37Z) o da un tempo relativo (es. 42m per 42 minuti fa).
-n, --tail	Numero di righe da mostrare dalla fine dei log per ogni container.
-t, --timestamps	Mostra i timestamp nei log.
--until	Mostra i log fino a un timestamp (es. 2013-01-02T13:23:37Z) o tempo relativo (es. 42m per 42 minuti fa).

4. Docker compose exec

Permette di eseguire comandi arbitrari all'interno dei container dei servizi.

Quando si usa `docker compose exec`, si sta dicendo a Docker di entrare in uno dei container dei servizi per eseguire un comando.

Può essere lanciato praticamente qualsiasi comando dentro il container, come se si fosse dentro il sistema operativo di quel container.

Docker mette a disposizione una console o un terminale “vero”, che permette di interagire in modo diretto, digitando comandi e vedendo risposte in tempo reale.

Ad esempio, il comando

```
docker compose exec -it flask sh
```

apre una shell interattiva (un terminale) dentro il container chiamato flask.

Ricorda: flask è il nome del servizio nel file `docker-compose.yml`, non il nome del container (`test_flask_nginx_flask_app`).

Occorre essere nella cartella dove si trova il `compose.yml`

Si possono così scrivere comandi manualmente, come se si fosse dentro una vera macchina Linux (o altro sistema). In pratica: è come “entrare” dentro il container per vedere cosa succede o per fare modifiche manuali.

PRINCIPALI OPZIONI

-d, --detach

Modalità detached: esegue il comando **in background**

-e, --env

Imposta **variabili d'ambiente per il comando**

--index

Indice del container se il servizio ha più repliche

-T, --no-TTY

Disabilita l'allocazione del pseudo-TTY (di default è attivo)

--privileged

Concede **privilegi estesi al processo**

-u, --user

Esegue il comando come l'utente specificato

-w, --workdir

Percorso della directory di lavoro per il comando

Nel terminale, entrare nella directory del progetto

```
PS C:\Users\PC> cd nginx-flask-mysql
PS C:\Users\PC\nginx-flask-mysql>
```

Lanciare il comando `docker compose up -d`

La lista dei containers dovrebbe contenere tre container attivi e la porta mappata.

`docker compose ps`

Risultati da terminali a fondo pagina

Andare all'URL <http://localhost:80>

Oppure lanciare il comando `curl`:

`curl localhost:80`

```
Hello Blog post #1
Hello Blog post #2
Hello Blog post #3
Hello Blog post #4
```

```
Terminal

PowerShell 7.5.2
PS C:\Users\PC> cd
PS C:\Users\PC> cd nginx-flask-mysql
PS C:\Users\PC\nginx-flask-mysql> docker compose up -d
[+] Running 5/5
 ✓ Network nginx-flask-mysql_backnet      Created
 ✓ Network nginx-flask-mysql_frontnet     Created
 ✓ Container nginx-flask-mysql-db-1       Healthy
 ✓ Container nginx-flask-mysql-backend-1  Started
 ✓ Container nginx-flask-mysql-proxy-1    Started
PS C:\Users\PC\nginx-flask-mysql>
```

```
PS C:\Users\PC\nginx-flask-mysql> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
nginx-flask-mysql-backend-1	nginx-flask-mysql-backend	"flask run"	backend	4 minutes ago	Restarting (1) 10 seconds ago	
nginx-flask-mysql-db-1	mariadb:10-focal	"docker-entrypoint.s..."	db	4 minutes ago	Up 4 minutes (healthy)	3306/tcp, 33060/tcp
nginx-flask-mysql-proxy-1	nginx-flask-mysql-proxy	"nginx -g 'daemon of..."	proxy	4 minutes ago	Up 4 minutes	0.0.0.0:80->80/tcp, [::]:80->80/tcp

DOCKERFILE BEST PRACTICES

- Scegliere la giusta immagine di partenza, assicurarsi che provenga da **una fonte affidabile** e che sia **leggera**.
Migliori fonti pubbliche:

- **Immagini ufficiali Docker**: collezione curata e con documentazione chiara, regolarmente aggiornata.
- **Fonti verificate** che offrono immagini di alta qualità pubblicate e mantenute da compagnie partner di Docker

– Fonti libere sponsorizzate da Docker

Quando si costruisce la propria immagine partendo da un Dockerfile, scegliere **l'immagine base minima** che soddisfa i requisiti richiesti. Un'immagine più piccola offre maggiore portabilità e downloads veloci, oltre a ridurre anche significativamente le dimensioni finali dell'immagine.

- Considerare anche l'utilizzo **di due immagini base**: una per lo sviluppo e test, l'altra (solitamente più snella) per la produzione. Nelle ultime fasi dello sviluppo, potrebbero infatti non essere necessari tools come compilatori, build systems, ecc.

The screenshot displays the Docker Hub interface. On the left, a sidebar contains navigation links: Ask Gordon (BETA), Containers, Images, Volumes, Builds, Models (BETA), MCP Toolkit (BETA), Docker Hub (selected), Docker Scout, and Extensions. The main area shows search results for 'ros' (Robot Operating System) with 10M+ downloads and 695 stars. Other results include 'opensearchproject/opensearch', 'bitnami/spark', 'bitnami/neo4j', 'apache/airflow', and 'bitnamicharts/minio'.

- **RICOSTRUIRE LE IMMAGINI SPESSO** al fine di mantenerle **aggiornate e sicure**, aggiornando le dipendenze. Le immagini Docker sono immutabili, cioè rappresentano un'istantanea del momento in cui sono state create, comprese le immagini base, le librerie ed i software inclusi.

Per essere sicuri di usare le versioni più recenti delle dipendenze, si può costruire l'immagine usando l'opzione `--no-cache` che evita di utilizzare la cache di build

```
docker build --no-cache -t my-image:my-tag .
```

ESEMPIO

```
FROM ubuntu:24.04
RUN apt-get -y update && apt-get install -y --no-install-recommends python3
```

versione aggiornata di Ubuntu con nuove patch di sicurezza. Usando `--no-cache` forzi il download e l'installazione freschi di base image e dipendenze, assicurandoti di avere le versioni più aggiornate.

- **ESCLUDERE FILE CON .dockerignore**

Per evitare di copiare nel contesto di build file non rilevanti, usare un file `.dockerignore`.

Questo file funziona in modo simile a `.gitignore`, permettendo di **specificare pattern per escludere file o cartelle (es .git, file temporanei)**.

- **CREA CONTAINER EFFIMERI**

Le immagini Docker dovrebbero generare container effimeri, cioè che **possono essere fermati, distrutti e ricreati facilmente**, con minime configurazioni e setup necessari.

Questa pratica segue il principio dei processi stateless della metodologia Twelve-Factor App, che facilita la scalabilità e la manutenzione.

- **NON INSTALLARE PACCHETTI NON NECESSARI**

Evitare di installare software extra o non indispensabile solo perché “potrebbe servire”. Questo porta a immagini con minor complessità, meno dipendenze, dimensioni più piccole e tempi di build più rapidi

- **SCOMPORRE LE APPLICAZIONI (DECOUPLING)**

Ogni container dovrebbe avere una sola responsabilità o funzionalità.

Separare le app in più container indipendenti permette di:

- Scalare orizzontalmente con più facilità
- Riutilizzare container in contesti diversi

Per esempio, un'app web può essere composta da tre container separati per:

Web server

Database

Cache in memoria

- **COMUNICAZIONE TRA CONTAINER**

Se i container devono collaborare, usare le **reti Docker (container networks)** per farli comunicare in modo sicuro e ordinato.

- **USARE UTENTI NON-ROOT**

creare utenti con permessi limitati per ridurre rischi di sicurezza

```
RUN addgroup app && adduser -S -G app app  
USER app
```

- **SPRUTTARE LA BUILD CACHE**

Quando Docker costruisce un'immagine, esegue le istruzioni del Dockerfile in ordine.

Per ogni istruzione, verifica se può riutilizzare il risultato da una build precedente tramite la cache.

Capire come funzionano i layer e la cache e quando viene invalidata è fondamentale per ottenere build più veloci.

Per ottimizzare:

- Inserire le istruzioni che cambiano raramente (es. installazione dipendenze) prima di quelle che cambiano spesso (es. copia del codice).
- Evitare modifiche inutili ai file che causano invalidazione della cache.

- **GESTIRE LE VARIABILI SENSIBILI E LE CREDENZIALI**

Non mettere password o token direttamente nel Dockerfile.

Usa **build args** o variabili d'ambiente gestite in modo sicuro.

Evita di committare nel repo file con dati sensibili.

Per creare un Dockerfile efficiente e manutenibile, si consiglia di seguire alcune raccomandazioni:

- **FROM:** preferire, quando possibile, **immagini ufficiali come base per le proprie immagini**. Preferire inoltre immagini minimaliste come alpine o versioni slim per ridurre peso e superficie d'attacco. Evitare immagini non ufficiali e non mantenute.
- **LABEL:** Si possono aggiungere delle labels all'immagine per organizzare le immagini per progetto, per registrare informazioni su licenze, o altro. Per ogni label, creare una nuova riga che inizia con **LABEL** seguita da una o più **coppie chiave-valore**.

```
docker inspect nome_immagine
```

e troverai un blocco tipo:

```
json
{
  "Labels": {
    "maintainer": "Mario Rossi <mario@example.com>",
    "project": "CRM-Finance",
    "version": "1.0",
    "license": "MIT"
  }
}
```

```
# Set one or more individual labels
LABEL com.example.version="0.0.1-beta"
LABEL vendor1="ACME Incorporated"
LABEL vendor2="ZENITH\ Incorporated"
LABEL com.example.release-date="2015-02-12"
LABEL com.example.version.is-production=""
```

- **RUN:** **separare lunghe o complesse istruzioni RUN in più righe separate da backslash** per rendere il Dockerfile più leggibile e comprensibile.

```
RUN apt-get update && apt-get install -y --no-install-recommends \
package-bar \
package-baz \
package-foo
```

- **VOLUME:** l'istruzione VOLUME dovrebbe essere usata per **indicare una cartella come persistente e condivisibile tra il container e l'host o altri container**. Consigliato per qualsiasi contenuto modificabile come dati del database, configurazioni o file generati durante l'uso. Questo separa chiaramente codice statico da dati dinamici.
- **WORKDIR:** imposta la directory di lavoro per tutte le istruzioni successive. È fortemente consigliato usare **percorsi assoluti** per chiarezza e manutenzione, e preferife **WORKDIR** invece di concatenare cd in comandi RUN. Questo rende il Dockerfile più leggibile e meno soggetto ad errori.

- **USARE COPY DI DEFAULT, ADD SOLO SE SERVE**
COPY è semplice e veloce, ideale per copiare file locali nel contesto di build.
ADD è utile solo se si ha necessità di **scaricare file remoti direttamente durante la build** (es. file da URL HTTPS, da GIT) o di estrarre automaticamente archivi tar nel container.
Prediligere COPY dove possibile perché migliora cache di build
- **CONTROLLARE PORTE E RISORSE ESPOSTE**
Esporre solo le porte strettamente necessarie con EXPOSE. Non lasciare porte o servizi aperti inutilmente.

- **Usa ENTRYPOINT + CMD per definire il comportamento del container**
Usa **ENTRYPOINT** per il comando principale del container (es. un servizio, un tool CLI).
Usa **CMD** per passare i parametri di default, così l'utente può sovrascriverli al momento dell'esecuzione.
Vantaggi:
Il container si comporta come un comando eseguibile, chiaro e riutilizzabile.
Facilità d'uso: puoi passare parametri senza dover scrivere tutto il comando da zero.
Puoi esporre funzionalità multiple usando uno script helper come ENTRYPOINT (es. inizializzazione + avvio servizio).
- **MINIMIZZARE I LAYER E ORDINARE LE ISTRUZIONI**
Ogni comando crea un layer. Cercare, quando possibile, di unire più comandi in uno solo con &&. Inoltre eliminare i file temporanei subito dopo l'uso per non lasciarli nel layer

```
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```

USO DI DOCKER HUB

Docker Hub è il principale registry pubblico per immagini Docker. Serve per archiviare, gestire e condividere container. Semplifica lo sviluppo fornendo immagini preconfigurate (database, linguaggi, web server, ecc.), riducendo il tempo di setup. Supporta automazione tramite webhooks, build automatizzate e integrazione con GitHub/Bitbucket. Garantisce affidabilità con contenuti verificati (trusted content) e immagini sicure. Offre controllo grazie a repository pubblici illimitati e privati per progetti interni. Offre interfaccia grafica, Docker Hub API e CLI. Permette di scaricare immagini, modificarle e condividerle.

PREREQUISITI

- Docker installato
- Account Docker Hub (necessario per pubblicare o usare repository privati)

1. TROVARE UN'IMMAGINE NELLA LIBRERIA DI DOCKER HUB

È possibile cercare contenuti in Docker Hub stesso, nella dashboard di Docker Desktop o usando la CLI.

– DOCKER HUB

È possibile navigare tra le risorse e filtrarle per categorie o in base alla fonte. Tra i filtri, c'è la possibilità di scegliere "Docker Official Image".

Selezionare l'immagine desiderata. Si aprirà una pagina dedicata all'immagini con informazioni sul suo utilizzo. Presente anche il comando docker pull per eseguire il pull dell'immagine.

Filter by (1) [Clear All](#)

Products ▼

- ☐ Images
- ☐ Extensions
- ☐ Plugins

Trusted content ▼

- ☒ Docker Official Image ⓘ
- ☐ Verified Publisher ⓘ
- ☐ Sponsored OSS ⓘ

Categories ▼

- ☐ Networking
- ☐ Security
- ☐ Languages & frameworks
- ☐ Integration & delivery
- ☐ Message queues

1 - 28 of 178 available results.

Docker Official Image ✕

IMAGE

memcached

Docker Official Images

Free & open source, high-performance, distributed memory object caching system.

Pulls1B+

Stars2388

Last Updated10 days

IMAGE

nginx

Docker Official Images

Official build of Nginx.

Pulls1B+

Stars20927

Last Updated16 days

IMAGE

redis

Docker Official Images

Redis is the world's fastest data platform for caching, vector search, and NoSQL database.

Pulls

Stars

Last Updated

Explore / Official Images / nginx

nginx Docker Official Image · 1B+ · 10K+

WEB SERVERS

Overview

Tags

[Explore](#) [My Hub](#)

Quick reference

- Maintained by:
[the NGINX Docker Maintainers](#)
- Where to get help:
[the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Supported tags and respective Dockerfile links

- [1.29.0](#) [mainline](#) [1](#) [1.29](#) [latest](#) [1.29.0-bookworm](#) [mainline-bookworm](#) [1-bookworm](#) [1.29-bookworm](#) [bookworm](#)
- [1.29.0-perl](#) [mainline-perl](#) [1-perl](#) [1.29-perl](#) [perl](#) [1.29.0-bookworm-perl](#) [mainline-bookworm-perl](#) [1-bookworm-perl](#) [1.29-bookworm-perl](#) [bookworm-perl](#)
- [1.29.0-otel](#) [mainline-otel](#) [1-otel](#) [1.29-otel](#) [otel](#) [1.29.0-bookworm-otel](#) [mainline-bookworm-otel](#) [1-bookworm-otel](#) [1.29-bookworm-otel](#) [bookworm-otel](#)
- [1.29.0-alpine](#) [mainline-alpine](#) [1-alpine](#) [1.29-alpine](#) [alpine](#) [1.29.0-alpine3.22](#) [mainline-alpine3.22](#) [1-alpine3.22](#) [1.29-alpine3.22](#) [alpine3.22](#)

Tag summary

Recent tags
stable-perl

Content type

Image

Digest

sha256:47ef54778...

>_ Docker pull command

```
docker pull nginx:stable-perl
```

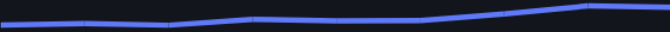
Run in Docker Desktop

Requires Docker Desktop 4.37.1 or later.

This weeks pulls

Pulls: 10,572,727

Last week



Tag summary

Recent tags
stable-perl

Content type

Image

Digest

sha256:47ef54778...

Size

80.1 MB

Last updated

16 days ago

Run in Docker Desktop

Requires Docker Desktop 4.37.1 or later.

DOCKER HUB

- **DOCKER DESKTOP**

Aprire la dashboard e selezionare Docker Hub. Si può navigare per categoria o usare la ricerca.

Una volta trovata l'immagine, cliccare per avere maggiori informazioni.

Nella pagina di dettaglio dell'immagine, in alto a destra, si trovano i comandi per eseguire il pull ed il run dell'immagine.

Il menù a tendina Tag permette di selezionare la versione desiderata dell'immagine.

The screenshot displays the Docker Hub interface for the **memcached** Docker Official Image. The page includes a description, download statistics (2.4K stars, 1B+ downloads), and a list of supported tags. A dropdown menu for tags is open, showing options like **latest**, **bookworm**, **alpine3.22**, and **alpine**. The Docker Desktop sidebar on the right shows the **Docker Hub** extension selected.

memcached Docker Official Image

Free & open source, high-performance, distributed memory object caching system.

DATABASES & STORAGE

☆ 2.4K ↓ 1B+

Overview Tags

Quick reference

- Maintained by: [the Docker Community](#)
- Where to get help: [the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Supported tags and respective Dockerfile links

- [1.6.39](#) , [1.6](#) , [1](#) , [latest](#) , [1.6.39-bookworm](#) , [1.6-bookworm](#) , [1-bookworm](#) , [bookworm](#)
- [1.6.39-alpine](#) , [1.6-alpine](#) , [1-alpine](#) , [alpine](#) , [1.6.39-alpine3.22](#) , [1.6-alpine3.22](#) , [1-alpine3.22](#) , [alpine3.22](#)

Quick reference (cont.)

- Where to file issues: <https://github.com/docker-library/memcached/issues>
- Supported architectures: ([more info](#))
[amd64](#) , [arm32v5](#) , [arm32v6](#) , [arm32v7](#) , [arm64v8](#) , [i386](#) , [mips64le](#) , [ppc64le](#) , [riscv64](#) , [s390x](#)
- Published image artifact details:
[repo-info](#) [repo's](#) [repos/memcached/](#) [directory](#) ([history](#))
(image metadata, transfer size, etc)

Recent Tags

LATEST BOOKWORM ALPINE3.22 ALPINE 1.6.39-BOOKWORM 1.6.39-ALPINE3.22 1.6.39-ALPINE 1.6.39 1.6-BOOKWORM 1.6-ALPINE3.22

Tag: latest ▼ Pull Run

latest
bookworm
alpine3.22
alpine
1.6.39-bookworm
1.6.39-alpine3.22
1.6.39-alpine
1.6.39
1.6-bookworm
1.6-alpine3.22
1.6-alpine
1.6
1-bookworm
1-alpine3.22
1-alpine
1
1.6.38-alpine3.22
1.6.38-alpine

Docker Desktop sidebar: Ask Gordon BETA, Containers, Images, Volumes, Builds, Models BETA, MCP Toolkit BETA, Docker Hub, Docker Scout, Extensions.

Search: Docker Hub / Search, Search resources, Machine Learning & AI, 1 - 24 of 2500 results, ros, The Robot Operating System for building robot applications, 10M+ 695.

- CLI

nel terminale, lanciare il comando:

```
$ docker search --filter is-official=true nginx
```

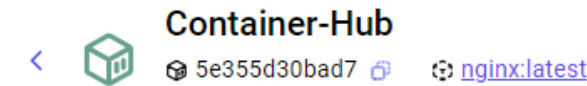
L'uso della CLI non consente di navigare tra categorie.

2. PULL E RUN DI UN'IMMAGINE DA DOCKER HUB

– Docker Desktop

Aprire la pagina dell'immagine desiderata e selezionare Run.

Se l'immagine non esiste localmente, viene automaticamente eseguito il pull da Docker Hub. Dopo che l'immagine è stata recuperata, apparirà una schermata su Docker Desktop per specificare le opzioni di run, come nome container e porta. Una volta lanciato il run, appare il container dei logs.



```
Logs Inspect Bind mounts Exec Files Stats

/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/08/08 13:19:59 [notice] 1#1: using the "epoll" event method
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
2025/08/08 13:19:59 [notice]
```

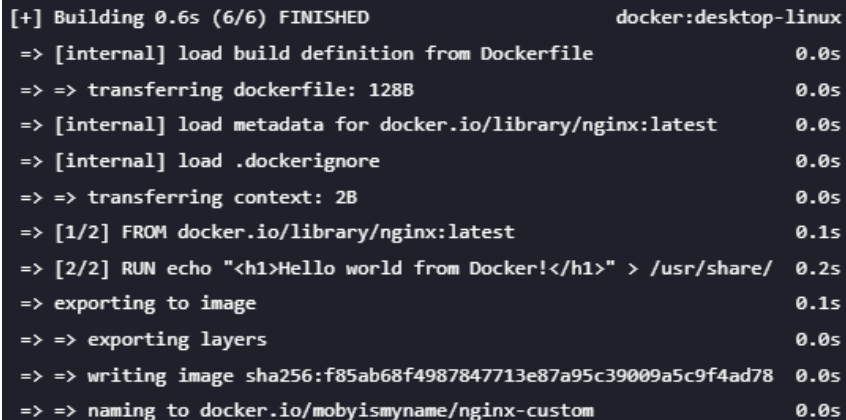
- **CLI**

Nel terminale inserire il seguente codice per eseguire pull e run dell'immagine.

```
docker run -p 8080:80 --rm nginx
```

Il comando run automaticamente eseguirà il pull e avvierà l'immagine senza aver bisogno di dichiarare docker pull.

Visitare localhost:8080 per vedere la pagina di default di Nginx.



```
[+] Building 0.6s (6/6) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 128B                    0.0s
=> [internal] load metadata for docker.io/library/nginx:latest 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                          0.0s
=> [1/2] FROM docker.io/library/nginx:latest           0.1s
=> [2/2] RUN echo "<h1>Hello world from Docker!</h1>" > /usr/share/ 0.2s
=> exporting to image                                   0.1s
=> => exporting layers                                  0.0s
=> => writing image sha256:f85ab68f4987847713e87a95c39009a5c9f4ad78 0.0s
=> => naming to docker.io/mobyismyname/nginx-custom    0.0s
```

3. BUILD E PUSH DI UN'IMMAGINE SU DOCKER HUB

Dopo aver creato un Dockerfile, lanciare il comando per costruire l'immagine. Sostituire <YOUR-USERNAME> con l'ID Docker.

```
docker build -t <YOUR-USERNAME>/nginx-custom .
```

Questo comando **costruisce l'immagine** e le **tagga** in modo che Docker capisca **in quale repository Docker Hub eseguire il push**.

Prima di eseguire il push, **bisogna assicurarsi di aver effettuato l'accesso a Docker Desktop**.

Lanciare il seguente comando per eseguire il push dell'immagine su Docker Hub.

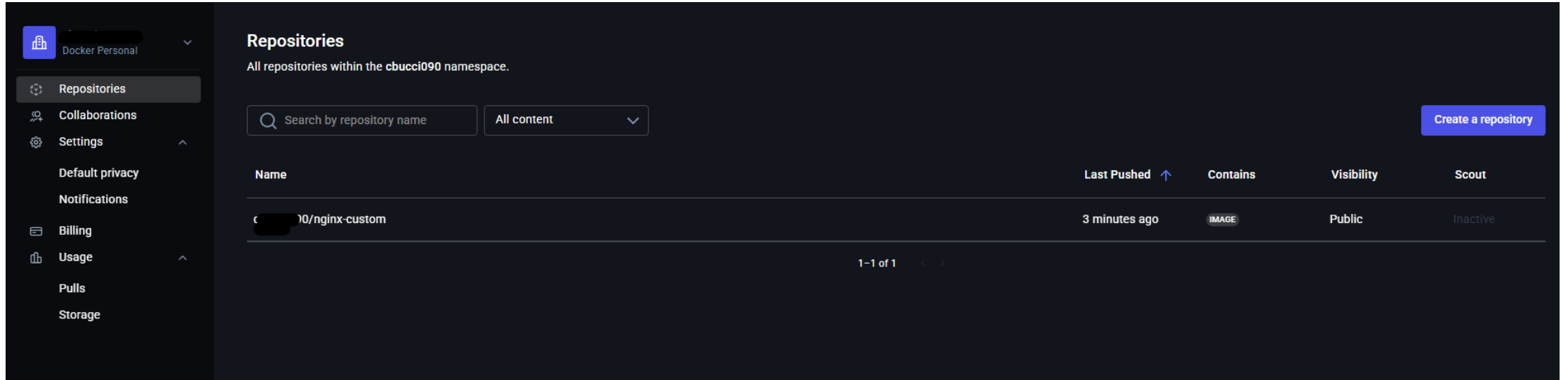
```
docker push <YOUR-USERNAME>/nginx-custom
```

Il comando esegue il push dell'immagine su DockerHub ed **automaticamente crea il repository se non esiste**.

```
Using default tag: latest
The push refers to repository [docker.io/cbucci090/nginx-custom]
36702289344a: Pushed
140da4f89dcb: Mounted from library/nginx
1d9f51194194: Mounted from library/nginx
2ef442a3816e: Mounted from library/nginx
4b1e45a9989f: Mounted from library/nginx
f30ffbee4c54: Mounted from library/nginx
7be4d3bff2c8: Pushed
96e47e70491e: Mounted from library/nginx
59e22667830b: Mounted from library/nextcloud
latest: digest: sha256:636af97c0d906aef13e26718770ff03a0d8b00efe0ea2cd40b00f9cf14e13ee0 size: 856
```

4. VISUALIZZARE IL REPOSITORY SU DOCKER HUB ED OPZIONI

Su Docker Hub si potrà verificare l'avvenuta creazione del repository



The screenshot shows the Docker Hub interface for the 'cbucci090' namespace. The left sidebar contains navigation links: Repositories (selected), Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main area is titled 'Repositories' and shows 'All repositories within the cbucci090 namespace.' There is a search bar and a dropdown for 'All content'. A 'Create a repository' button is in the top right. A table lists the repositories:

Name	Last Pushed	Contains	Visibility	Scout
cbucci090/nginx-custom	3 minutes ago	IMAGE	Public	Inactive

At the bottom of the table, it says '1-1 of 1'.

Su Docker Desktop, Images ed, in altro, My Hub

Images [Give feedback](#)

View and manage your local and Docker Hub images. [Learn more](#)

Local **My Hub**

cbucci090

[View Docker Scout dashboard](#)

	Tags	OS	Vulnerabilities	Last pushed	Size
 cbucci090/nginx-custom	latest		 Inactive	23 minutes ago	72.2 MB

INFORMAZIONI SUL REPOSITORY

Ogni repository può includere:

- **Descrizione** → breve testo visibile nei risultati di ricerca. Massimo 100 caratteri.
- **Overview** → spiegazione dettagliata dell'immagine e di come usarla.
- **Categorie** → etichette per facilitare la ricerca.

Queste informazioni sono modificabili solo se il repository non è archiviato.

Repositories / [nginx-custom](#) / General

cbucci090/nginx-custom

Last pushed about 1 hour ago · Repository size: 68.9 MB

[Add a description](#)

[Add a category](#)

General Tags Image Management **BETA** Collaborators Webhooks Settings

Tags DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	about 1 hour

[See all](#)

Repository overview INCOMPLETE

An overview describes what your image does and how to run it. It displays in [the public view of your repository](#) once you have pushed some content.

[Add overview](#)

DESCRIZIONE

- Riassumere chiaramente lo scopo dell'immagine (a cosa serve, per quale app o piattaforma).
- Evidenziare funzionalità o benefici principali (es. leggero, veloce, compatibile con X).
- Usare parole chiave rilevanti (stack tecnologici, use case, ambienti).
- Restare brevi e diretti (1-2 frasi).
- Scrivere pensando al target (dev, sysadmin, ecc.).

OVERVIEW (dettagli)

- Cosa fa l'immagine, funzionalità e perché usarla.
- Esempio minimo di utilizzo (anche in Dockerfile).
- Lista di varianti e tag disponibili con relativi use case.
- Link a documentazione, supporto o community.
- Contatti dei maintainer
- Licenza

CATEGORIE

- Servono per far trovare più facilmente il repository.
- Esempi: *Databases & Storage, Machine Learning & AI, Security, Web Servers...*
- Se ne possono assegnare fino a **3**.

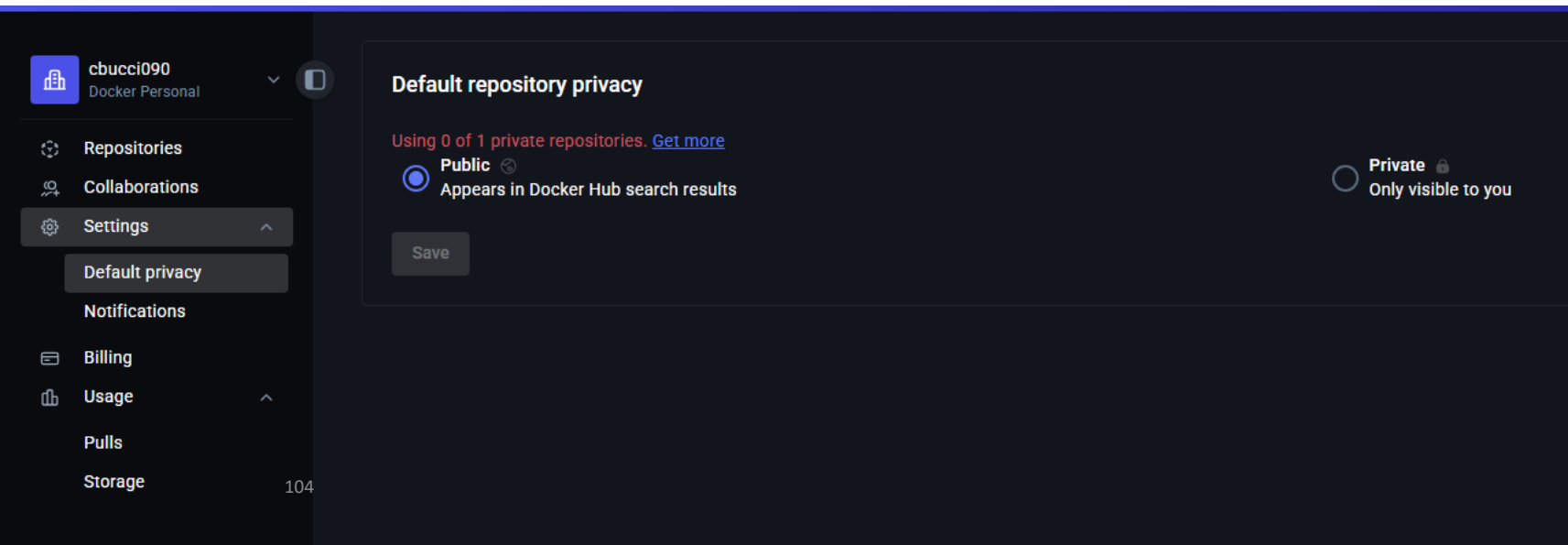
GESTIONE ACCESSI IN DOCKER HUB

VISIBILITÀ REPOSITORY

- **Pubblico** → visibile nei risultati di ricerca e scaricabile da tutti.
 - **Push**: tramite **collaboratori** (per repository personali) o **ruoli/teams/tokens** (per organizzazioni).
- **Privato** → non visibile pubblicamente, **accesso solo agli autorizzati**.
 - **Push/Pull**: tramite **collaboratori** (personali) o **ruoli/teams/tokens** (organizzazioni).

CAMBIO VISIBILITÀ

- Accedere a Docker Hub → **My Hub > Repositories** → scegli repository.
- **Settings** → sezione *Default privacy* → scegli *Public* o *Private*.
- Selezionare e salvare
- Modificabile anche da tab Settings > Visibility Settings > Make private (o Make public). Digitare il nome del repository per salvare



COLLABORATORI

Hanno **accesso push e pull su repository personali**.

Non possono gestire impostazioni o aggiungere altri collaboratori.

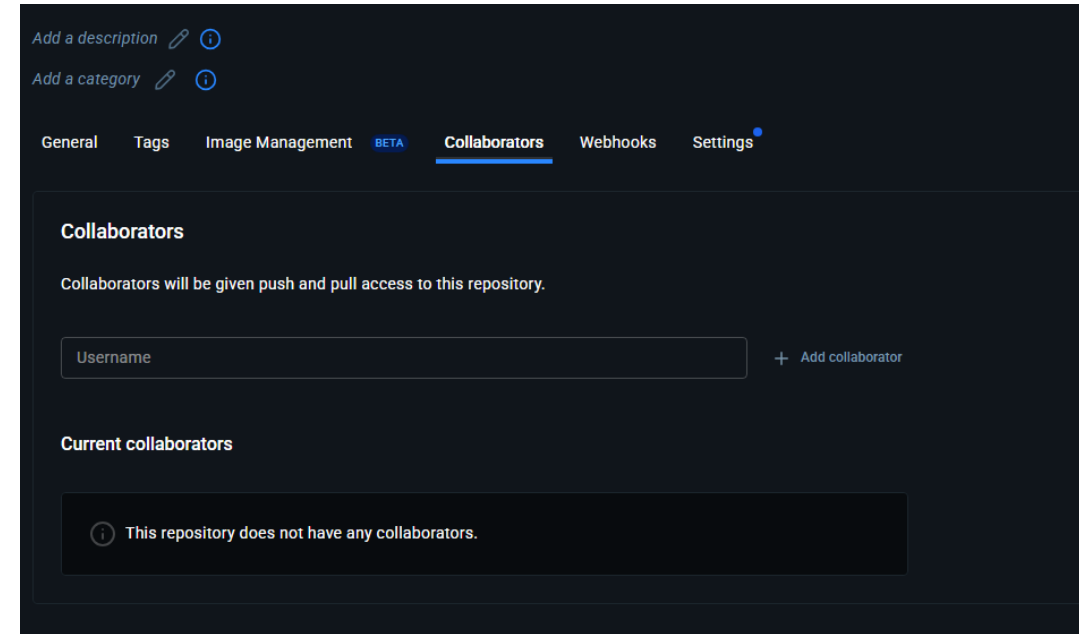
LIMITI:

- Pubblici → collaboratori illimitati.
- Privati → massimo 1 collaboratore con account *Docker Pro*.

Non sono disponibili per repository di organizzazioni.

GESTIONE:

- Andare su **Collaborators tab** del repository.
- Aggiungi/rimuovi collaboratori usando nome utente Docker.



RUOLI ORGANIZZAZIONE (disponibile solo per organizzazioni)

- Permettono di assegnare permessi differenti agli utenti.
- Gestibili in **Roles and permissions**.

TEAMS (disponibile solo per organizzazioni)

- Gruppi di utenti con permessi granulari.
- Configurazione: Creare un team.
 - In Permissions tab del repository → aggiungi/modifica/rimuovi permessi per il team (read, write, ecc.).

ORGANIZATION ACCESS TOKENS (OATS)

Token con permessi specifici sui repository. Ideali per automazioni e CI/CD.6.

GATED DISTRIBUTION (ACCESSO ANTICIPATO)

- Condivisione sicura di immagini private con clienti o partner senza dare accesso all'organizzazione.
- Caratteristiche:
 - Repository privati con invito esplicito.
 - Accesso pull-only per utenti esterni.
 - Invito via email tramite API (no UI).
- Passi:
 - Genera bearer token (API autenticazione).
 - Crea team e assegna permessi read-only.
 - Invia inviti con API Bulk create invites impostando role=distributor_member.

GESTIONE DELLE IMMAGINI

• Tagging

I tag servono a **versionare** e **organizzare** diverse versioni di un'immagine all'interno dello stesso repository. Si possono creare, visualizzare e cancellare tag per tenere tutto ordinato.

• Gestione immagini

Possibilità di gestire immagini e indici di immagini **per ottimizzare lo spazio** e la **struttura** del repository.

• Software artifacts (OCI artifacts)

Docker Hub supporta anche artefatti OCI, cioè non solo immagini Docker classiche, ma anche altri contenuti come Helm charts, report di vulnerabilità, ecc. Questo permette di conservare e distribuire vari tipi di contenuti containerizzati.

• Push immagini

Caricamento (push) di immagini locali su Docker Hub usando il comando **docker push**, rendendole disponibili per il team o per la community.

• Spostamento immagini tra repository

Possibilità di spostarle tra repository diversi,

1. TAGS

I tag permettono di gestire più versioni di un'immagine dentro lo stesso repository.

Esempio: docs/base:testing dove testing è il tag.

Se non specificato, per default si usa il tag latest.

COME TAGGARE UN'IMMAGINE LOCALE

Al momento della build

```
docker build -t <org-o-utente>/<repo>[:<tag>] .
```

Ritaggarne immagine esistente

```
docker tag <immagine-esistente> <org-o-utente>/<repo>[:<tag>]
```

PUSH DELL'IMMAGINE TAGGATA SU DOCKER HUB

Dopo aver taggato l'immagine, si carica su Docker Hub con

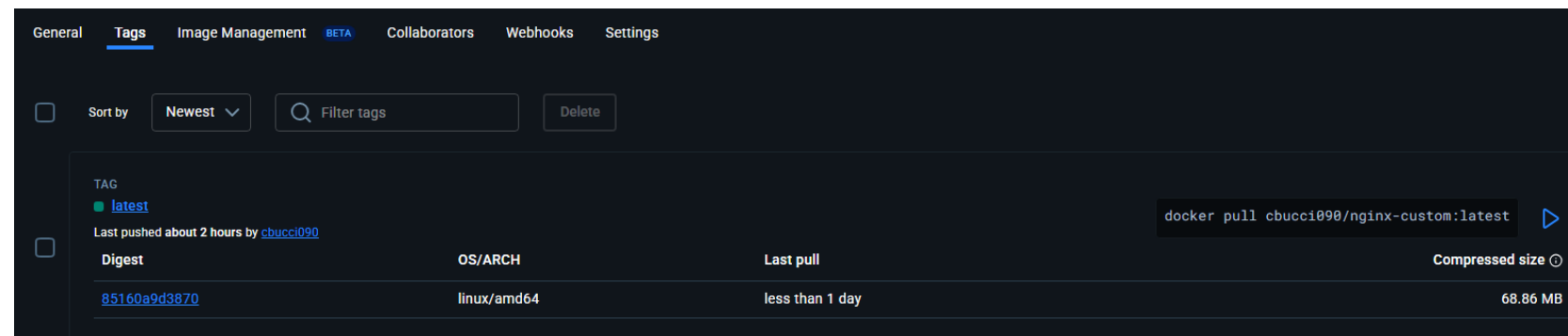
```
docker push <org-o-utente>/<repo>:<tag>
```

VISUALIZZARE I TAG DI UN REPOSITORY

- Accedi a Docker Hub.
- Vai su **My Hub > Repositories**.
- Seleziona il repository desiderato.
- Apri la scheda **Tags** per vedere i tag disponibili e la dimensione delle immagini.
- Puoi cliccare sul digest del tag per vedere dettagli aggiuntivi.

ELIMINARE TAG

Solo il proprietario del repository o membri con permessi possono eliminare tag.



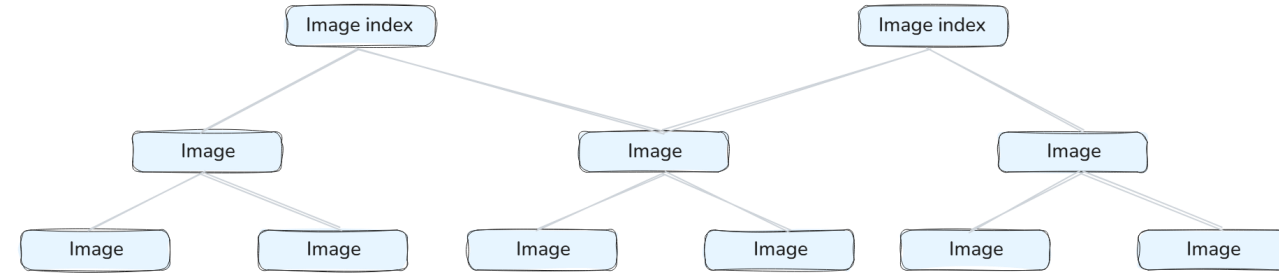
2. IMAGE MANAGEMENT (beta)

Concetti chiave

- Immagine (Image): Contiene la configurazione e i layer specifici per un'architettura e sistema operativo (es. AMD64, ARM).
- Indice immagine (Image Index): Un'immagine speciale che punta a più immagini specifiche per architetture diverse. Permette di usare un singolo riferimento per diverse piattaforme (multi-arch).

Come gestire immagini e indici immagine nel repository

- Accedere a Docker Hub.
- Andare su My Hub > Repositories.
- Selezionare il repository di interesse.
- Andare alla sezione Image Management.



Funzionalità disponibili

- **Cerca:**
Usa la casella di ricerca per trovare immagini o indici.
- **Filtra:**
Filtra per:
 - Tagged (immagini con tag)
 - Image index (indici multi-architettura)
 - Image (immagini singole)
- **Ordina**
Puoi ordinare la lista cliccando sulle colonne:
 - Size (dimensione)
 - Last pushed (ultimo caricamento)
 - Last pulled (ultimo download)

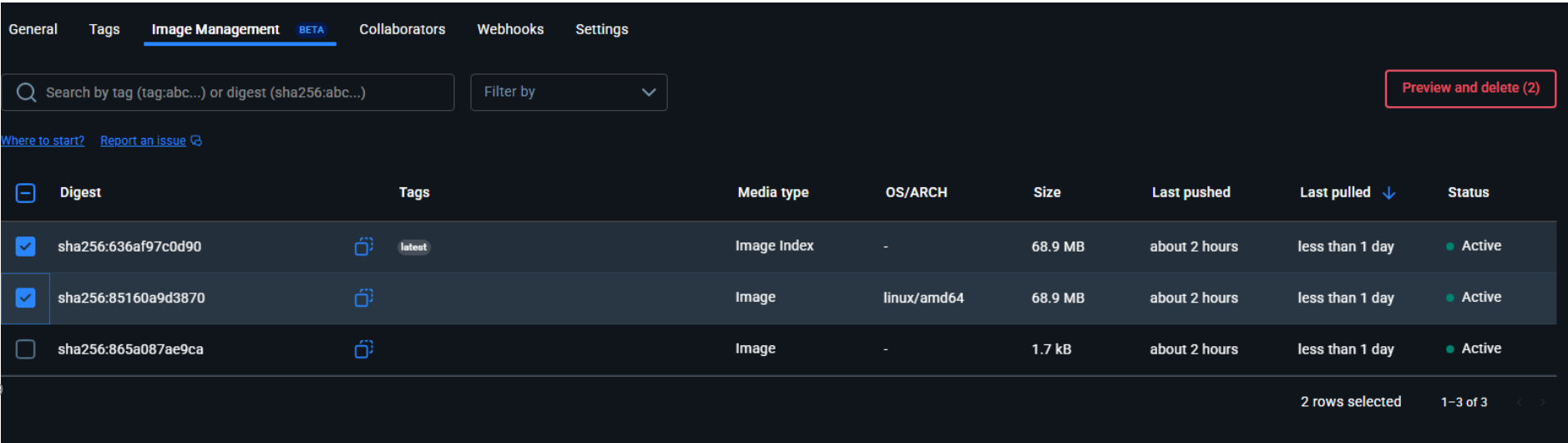
STATO OBSOLETO

Le immagini non scaricate (pull) negli ultimi 6 mesi, vengono indicate come STALE nella colonna stato.

ELIMINARE IMMAGINI O INDICI

- Selezionare le checkbox accanto agli elementi da eliminare.
- Se si seleziona un indice immagine, si eliminano anche le immagini sottostanti non referenziate altrove.
- Cliccare su **Preview and delete**.
- Verificare l'elenco e lo spazio che sarà liberato.
- Confermare con **Delete forever**.

Per eliminare molte immagini in modo automatizzato, si può usare l'API di cancellazione.



The screenshot shows the Docker Image Management interface. At the top, there are tabs: General, Tags, Image Management (selected), Collaborators, Webhooks, and Settings. Below the tabs is a search bar with the text "Search by tag (tag:abc...) or digest (sha256:abc...)" and a "Filter by" dropdown. A red button labeled "Preview and delete (2)" is in the top right. Below the search bar, there are links for "Where to start?" and "Report an issue". The main table has columns: Digest, Tags, Media type, OS/ARCH, Size, Last pushed, Last pulled, and Status. Three rows are visible, each with a checkbox in the Digest column. The first two rows are selected. The third row is not selected.

☐	Digest	Tags	Media type	OS/ARCH	Size	Last pushed	Last pulled	Status
☒	sha256:636af97c0d90	latest	Image Index	-	68.9 MB	about 2 hours	less than 1 day	Active
☒	sha256:85160a9d3870		Image	linux/amd64	68.9 MB	about 2 hours	less than 1 day	Active
☐	sha256:865a087ae9ca		Image	-	1.7 kB	about 2 hours	less than 1 day	Active

2 rows selected 1-3 of 3

Render.com

Render.com PAAS

COSA È RENDER.COM

- Piattaforma cloud (**PaaS** – platform as a service) che permette di distribuire applicazioni, API, siti statici e database senza dover gestire direttamente server o infrastrutture complesse.
- Supporta **diversi tipi di deployment**:
 - Web services (app con Dockerfile, Node.js, Python, Go, ecc.)
 - Cron jobs
 - Database
 - Siti statici (da repo GitHub/GitLab)
 - Background workers (job schedulati o code consumer)
- Offre:
 - **Scalabilità automatica** verticale
 - **Ridistribuzione automatica** in caso di crash
 - Integrazione con **logging** e **metrics**
 - **Certificati SSL** automatici

COME FUNZIONA

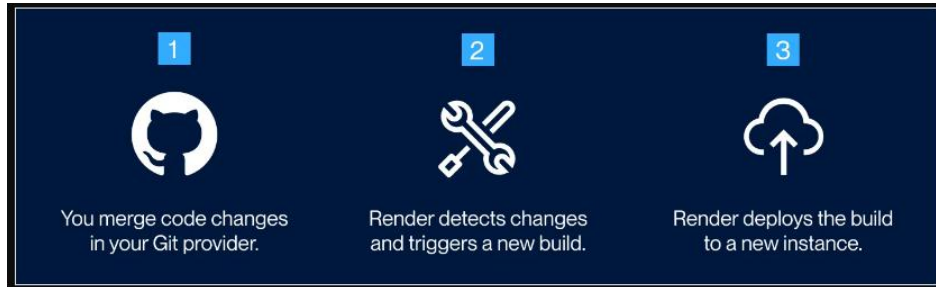
In relazione a Docker, consente di lavorare direttamente con container e immagini Docker già pronte.

3 modi principali per pubblicare progetti:

- **Collegamento di un repository** GitHub/GitLab: render esegue build e deploy ad ogni push (per progetti non Docker, riconosce automaticamente linguaggio/framework e costruisce l'ambiente)
- Utilizzo di un **Dockerfile**: render fa il build dell'immagine e avvia il container.
- Usando **un'immagine Docker Hub**: render prende direttamente l'immagine già pronta e la esegue come container.

COME FUNZIONANO I DEPLOYS

Render può automaticamente eseguire il deploy dell'applicazione **ogni volta che si effettua il merge di una modifica al codice di base.**



Si possono anche lanciare **deploy manuali**, sia in maniera programmatica che dalla dashboard.

Di default, i services hanno un **filesystem effimero**: ogni modifica che il service in esecuzione apporta al suo filesystem, è persa con i nuovi deploy.

Per rendere i dati persistenti tra deploys si può:

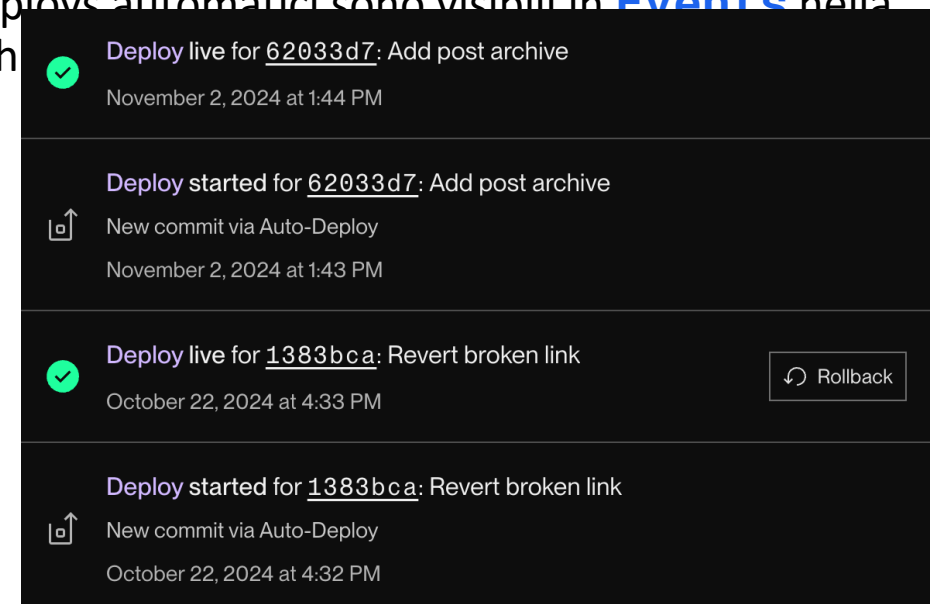
- **Creare e connettersi ad un Render-managed datastore** (Render Postgres o Key Value) o ad un **datastore personalizzato** (MySQL o MongoDB)
- **Attaccare un disco persistente** al service

DEPLOYS AUTOMATICI

SOLO Se hai un piano **Team o superiore** su Render, puoi usare la funzione

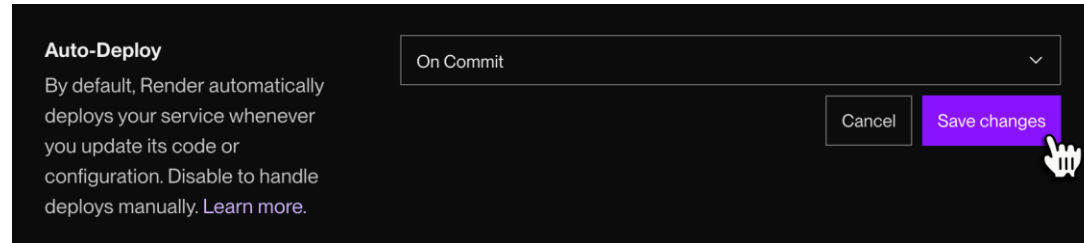
Per creare servizi su Render, spesso ci si collega al branch di un repository GitHub/GitLab/Bitbucket (tipo main o production). Di default, **ogni volta che si esegue il push o merge** di una modifica a quel branch, Render automaticamente **ricostruisce ed esegue nuovamente il deploy** del service.

I deploys automatici sono visibili in **Events** nella dash



CONFIGURARE I DEPLOYS AUTOMATICI

Nella sezione Settings del service.



Opzioni disponibili:

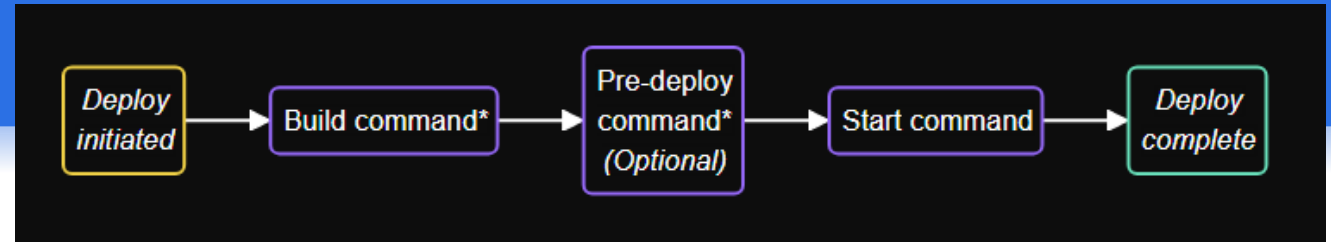
- **On Commit:** appena si esegue il push o merge di modifiche. Comportamento di default
- **After CI checks Pass:** Render lancia un nuovo deploy SOLO DOPO CHE tutti i controlli sono superati (success, neutral, skipped)
- **OFF:** auto-deploys disattivati

DEPLOYS MANUALI

Dalla dashboard selezionare **Manual Deploy dropdown:**

- **DEPLOY LATEST COMMIT:** esegue deploy dell'ultima commit
- **DEPLOY A SPECIFIC COMMIT:** esegue il deploy di una commit specificata dal suo SHA o selezionata da una lista di commits recenti. Disabilita il deploy automatico
- **CLEAR BUILD CACHE & DEPLOY:** simile a "deploy latest commit" ma prima pulisce la cache del build del service.
- **RESTART SERVICE:** esegue il deploy dello stesso commit del servizio.

DEPLOY STEPS



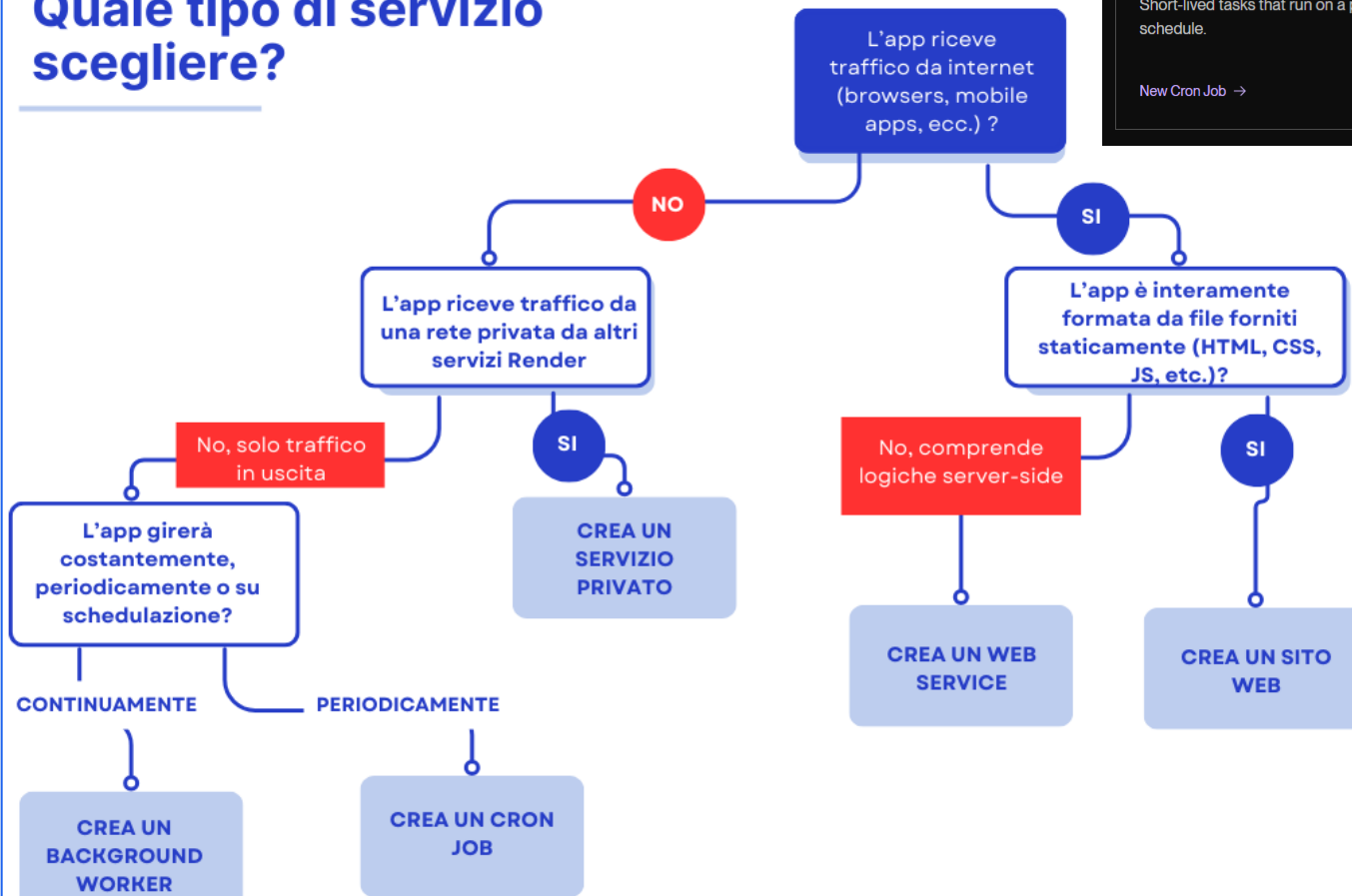
- **Build command, pre-deploy command e start command** possono essere opzionalmente specificati in fase di creazione del service.
- **Se qualche comando fallisce, l'intero processo di deploy fallirà.** Il service continuerà a funzionare usando il più recente deploy andato a buon fine, assicurando zero downtime.
- **BUILD COMMAND:** esegue la **compilazione e l'installazione delle dipendenze** necessarie al service per funzionare.
Per quanto riguarda Docker, **NON si può impostare un comando build.** Render costruisce un'immagine personalizzata basata su un Dockerfile o carica una specifica immagine da un registry.

- **PRE-DEPLOY COMMAND:** se definito, viene eseguito dopo il build ma prima che venga avviato il deploy. Raccomandato per operazioni che dovrebbero precedere il deploy ma non sono legate al codice come:
 - Migrazioni di database
 - Caricamento di assets su un CDN
- **START COMMAND:** comando lanciato per avviare il service. Per quanto riguarda Docker, di default Render lancia il CMD definito nel Dockerfile. Si può specificare un comando diverso.

CREAZIONE DI UN NUOVO SERVICE

Per prima cosa selezionare tipo di servizio che si desidera creare

Quale tipo di servizio scegliere?



Create a new Static Site

1 Choose service > 2 Configure > 3 Deploy

[Skip](#)

[Which to use?](#)

Static Sites

Static content served over a global CDN. Ideal for frontend, blogs, and content sites.

[New Static Site →](#)

Web Services

Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.

[New Web Service →](#)

Private Services

Web app hosted on a private network, accessible only from your other Render services.

[New Private Service →](#)

Background Workers

Long-lived services that process async tasks, usually from a job queue.

[New Worker →](#)

Cron Jobs

Short-lived tasks that run on a periodic schedule.

[New Cron Job →](#)

Postgres

Relational data storage. Supports point-in-time recovery, read replicas, and high availability.

[New Postgres →](#)

Key Value

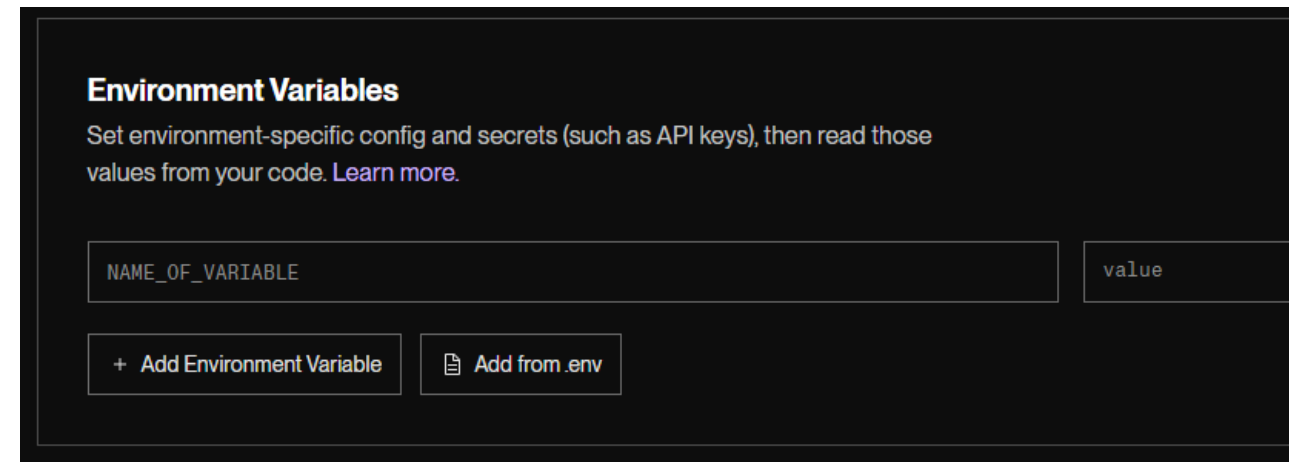
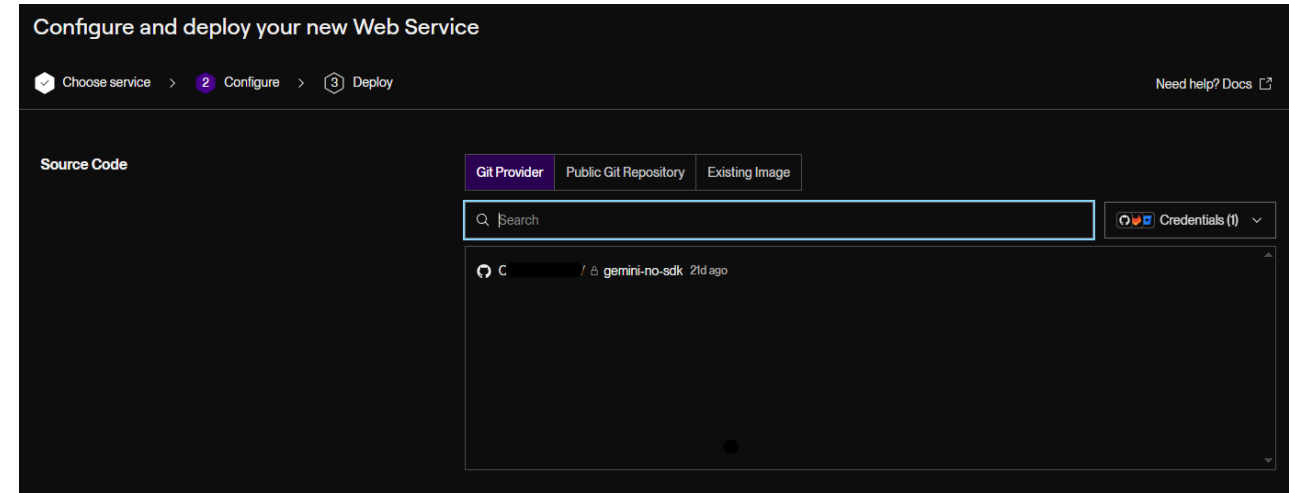
Managed Redis®-compatible storage. Ideal for use as a shared cache, message broker, or job queue.

[New Key Value Instance →](#)

Render.com con GitHub

Render prende il codice **direttamente dal repo GitHub** e costruisce l'immagine.

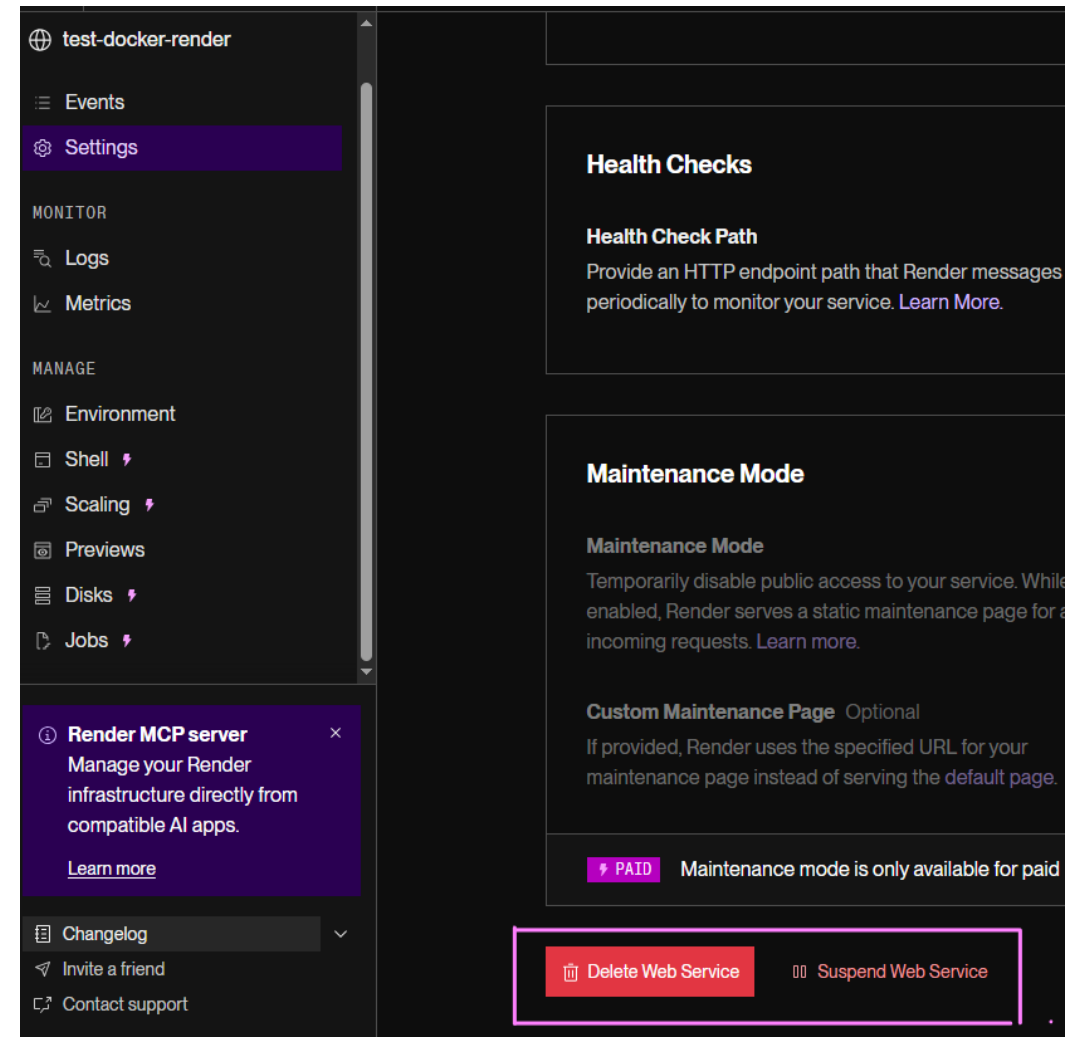
1. **Collegare repository GitHub** a Render in modo che quest'ultimo abbia accesso al codice
2. Una volta collegati, selezionare linguaggio, branch, region, root directory ed il path del dockerfile
3. È inoltre possibile impostare variabili d'ambiente o caricare un file .env
4. Impostazioni avanzate consentono di
 - caricare secret files,
 - montare dei dischi ssd,
 - specificare i path del Docker build context, del dockerfile
 - Di sovrascrivere le istruzioni CMD e ENTRYPOINT
 - Impostare l'auto-deploy, comandi pre-deploy o dei filtri per includere/escludere specifici path .



- Nella sezione "**Settings**" dell'applicazione, è possibile **eliminare o sospendere il web service**.
- Nella stessa sezione sarà possibile riavviare i servizi eventualmente sospesi.

COSA AVVIENE QUANDO SI LANCI IL DEPLOY

- I. Render **clona** il repository dal branch configurato (o root directory)
- II. **Cerca il Dockerfile** nella root (o path indicato nelle impostazioni)
- III. Esegue il **build** dell'immagine sui server di Render
- IV. L'immagine creata **viene avviata come container**
- V. Render applica eventuali variabili d'ambiente, networking e porte
- VI. Il **servizio web viene avviato**: il container gira online, accessibile attraverso URL pubblico
- VII. Aggiornamenti automatici: ad ogni push, Render ricostruisce e ridistribuisce app



STRUTTURA MINIMA DEL REPOSITORY GITHUB

`backend-emotion-api/`

|—— `Dockerfile`

|—— `app/` (o i file `.py`)

|—— `requirements.txt` (se usi Python)

|—— `altri file`

ESEMPIO

Abbiamo il repository `test-render-docker` che contiene i seguenti file:

- `app.py`
- `requirements.txt`
- `Dockerfile`
- `Templates`
 - `Index.html`

app.py

```
from flask import Flask, render_template, jsonify

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/hello")
def hello():
    return jsonify(message="Hello from Render API with Gunicorn!")

if __name__ == "__main__":
    # Solo per debug locale
    app.run(host="0.0.0.0", port=5000, debug=True)
```

requirements.txt

```
flask
gunicorn
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:app"]
```

Templates/index.html

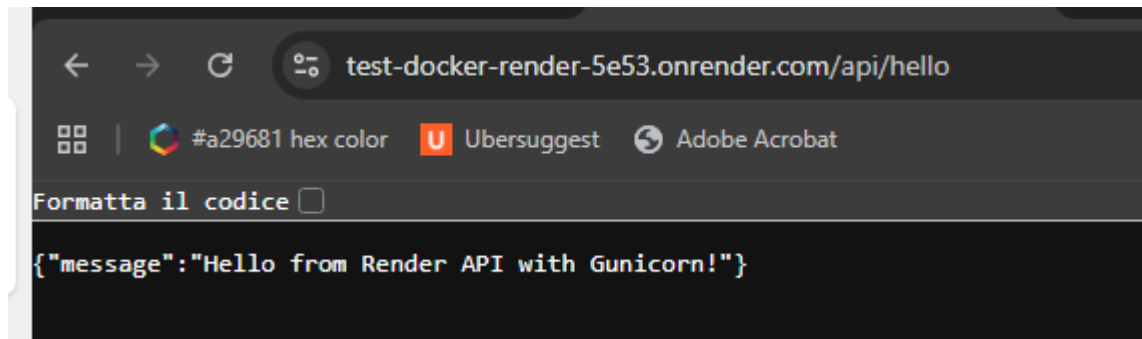
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Flask API</title>
</head>
<body>
    <h2>✅ API is running!</h2>
    <p>Vai a <a href="/api/hello">/api/hello</a> per testare l'endpoint.</p>
</body>
</html>
```

Una volta eseguito il deploy, all'URL del service apparirà una semplice pagina HTML contenente un link ad un endpoint

✓ **API is running!**

Vai a </api/hello> per testare l'endpoint.

Che restituirà un contenuto json



The screenshot shows a web browser window with the address bar displaying `test-docker-render-5e53.onrender.com/api/hello`. Below the address bar, there are several tabs: "#a29681 hex color", "U Ubersuggest", and "Adobe Acrobat". The main content area shows a JSON response: `{"message": "Hello from Render API with Gunicorn!"}`. Above the JSON, there is a button labeled "Formatta il codice" with a checkbox next to it.

YAML FILE – RENDER BLUEPRINTS (IaC – Infrastructure as Code)

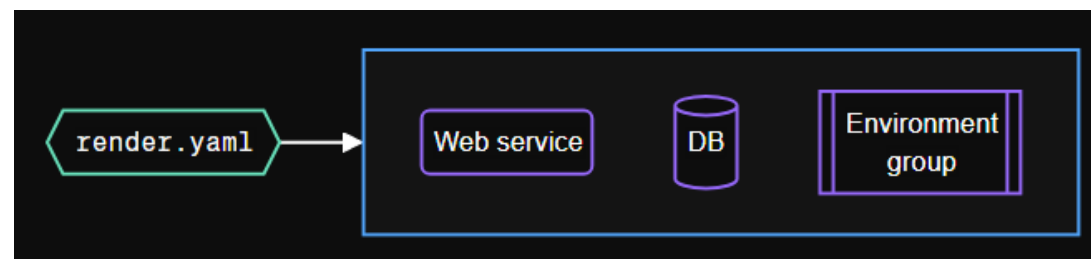
Il file `render.yaml` è il "**manifest**" dell'infrastruttura su Render.com: **descrive l'intera infrastruttura così come deve essere creata e mantenuta da Render.**

La creazione di un file `yaml` è utile nel caso si abbia la necessità di **gestire più servizi, le loro interconnessioni** e la **loro infrastruttura** senza doverli configurare uno per uno tramite UI.

Utile inoltre **per versionare anche l'infrastruttura** insieme al codice – chiunque clona il repository, ha già il setup pronto.

Il file `render.yaml` viene **letto da Render quando si fa il deploy da un repository Git collegato.** Vengono automaticamente creati i servizi e le risorse definiti nel file.

Non serve per deploy da immagine Docker già pronta.



ESEMPIO DI FILE render.yaml

```
services:
  - type: web                //tipo di servizio creato
    plan: free               //piano di Render utilizzato
    name: django-app        //nome leggibile del servizio (visibile in dashboard e URL)
    runtime: python          // Render userà un ambiente Python preconfigurato (non serve Dockerfile in questo caso)
    repo: https://github.com/render-examples/django.git //repo clonato da Render per costruire app
    buildCommand: './build.sh' //comando eseguito per installare dipendenze e preparare progetto
    startCommand: 'python -m gunicorn mysite.asgi:application -k uvicorn.workers.UvicornWorker'
                        // comando di avvio del server Django/ASGI: usa Gunicorn con worker Uvicorn (per gestione richieste asincrone)
  envVars:                  //variabili d'ambiente iniettate al runtime
    - key: DATABASE_URL      //verrà impostata automaticamente con la connection string del db Postgres chiamato django-app-db. Evita di dover scrivere manualmente postgres://user:pass@host:5432/dbname
      fromDatabase:
        name: django-app-db
        property: connectionString

databases:                 //Definizione di un database Postgres gestito da Render
  - name: django-app-db    //Nome db, usato anche per collegamento ai servizi
    plan: free              //Piano gratuito di Render per Postgres
```

RENDER.YAML REFERENCE: <https://render.com/docs/blueprint-spec>

Il file `render.yaml` definisce un'app Django con un database PostgreSQL su Render. Quando si applica questo file come Blueprint, Render:

- crea un db Postgres gratuito
- Crea un web service Python
 - Clona il repo indicato
 - Esegue `./build.sh`
 - Avvia il server con Gunicorn/Uvicorn
- Inietta la variabile `DATABASE_URL` al servizio, collegandolo al Postgres creato
- App Django ha già connessione al DB pronta.

DOCKER-COMPOSE E RENDER

Il file `docker-compose.yaml` descrive più container da avviare insieme e da collegare tra loro.

Render, però, **non supporta direttamente Docker Compose** perché non si occupa di avviare più container coordinati, ma di gestire più servizi indipendenti: ogni container deve diventare un servizio separato.

Si devono quindi **scomporre i container definiti in Compose in singoli servizi Render**: il container "web" diventa un Web Service, il container "worker" diventa un Background Worker, il container "db" non serve più perché Render fornisce un db Postgres gestito.

Il file `render.yaml`, sostituisce il ruolo del `docker-compose.yml` in chiave Render.

QUINDI:

- Docker Compose è perfetto in locale, perché coordina più container su un'unica macchina.
- Render lavora per servizi indipendenti: per questo bisogna spezzare il `docker-compose.yml` nei singoli componenti.
- **`render.yaml` diventa il ponte tra il mondo di Docker Compose e Render**, trasformando la descrizione dei container in un modello di infrastruttura "cloud-native" che Render può comprendere e gestire in automatico.

```
version: "3.9"

services:
  web:
    build: .
    container_name: django-app
    command: >
      python -m gunicorn mysite.asgi:application -k uvicorn.workers.UvicornWorker
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/django_db
    depends_on:
      - db

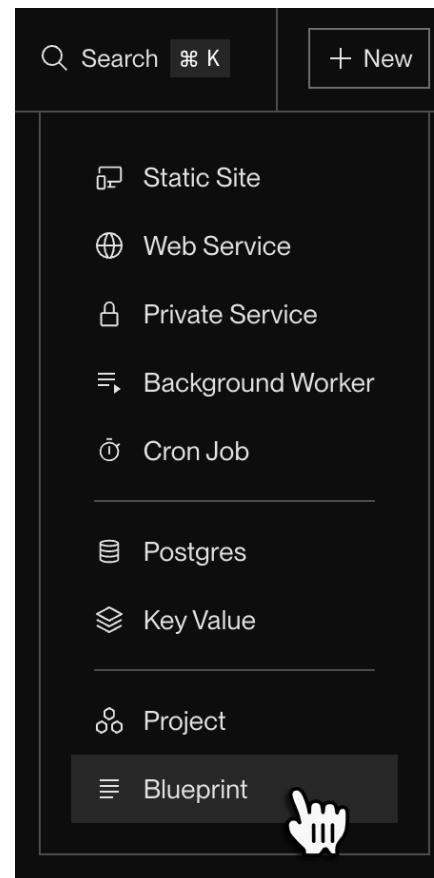
  db:
    image: postgres:15
    container_name: django-app-db
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=django_db
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Questo il file docker-compose.yml relativo all'infrastruttura Render descritta dal file render.yaml visto in precedenza.

CONFIGURAZIONE BLUEPRINT

1. Nella root del repository Git, creare un file chiamato **render.yaml**
2. Inserire nel file i dettagli delle risorse che si vogliono creare e gestire
3. Eseguire il **commit** e **push** delle modifiche al repository
4. Aprire la dashboard Render e cliccare su **New > Blueprint**
5. Nella lista, cliccare il tasto **Connect** al repository che contiene il file render.yaml
6. Inserire un **nome** per il Blueprint e specificare il **branch** del repo al quale ci si collega.
7. Controllare la **lista delle modifiche** che Render applicherà basandosi sul Blueprint collegato.
8. Se tutto è corretto, cliccare su **Apply**



Please review the changes to apply below. All future updates to **render.yaml** will be synced automatically. You can learn more about Render's infrastructure-as-code [here](#).

- ➔ Create database **django-app-db**
- ➔ Create web service **django-app**

Apply

Cancel

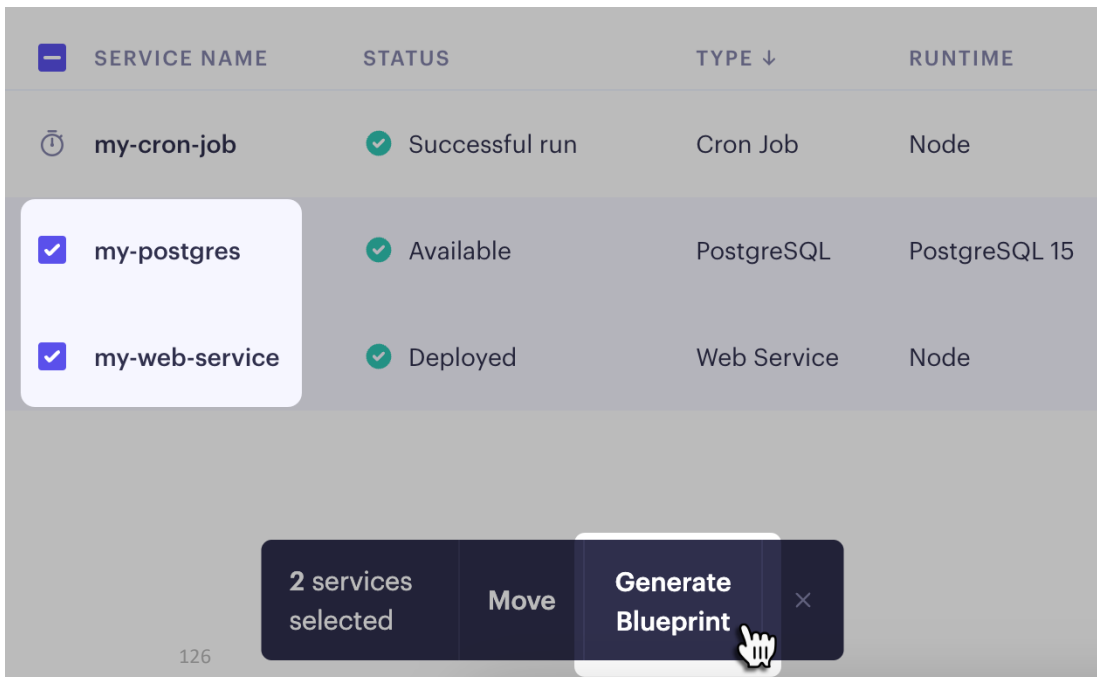
- ✔ Create database **django-app-db**
- 🔄 Create web service **django-app**

GENERARE UN BLUEPRINT DA SERVIZI ESISTENTI

Si può generare un file render.yaml usando una combinazione di servizi esistenti.

Nella Dashboard di Render, selezionare i servizi che ci interessano e cliccare su Generate Blueprint.

Si aprirà una pagina dove poter scaricare il file render.yaml generato.



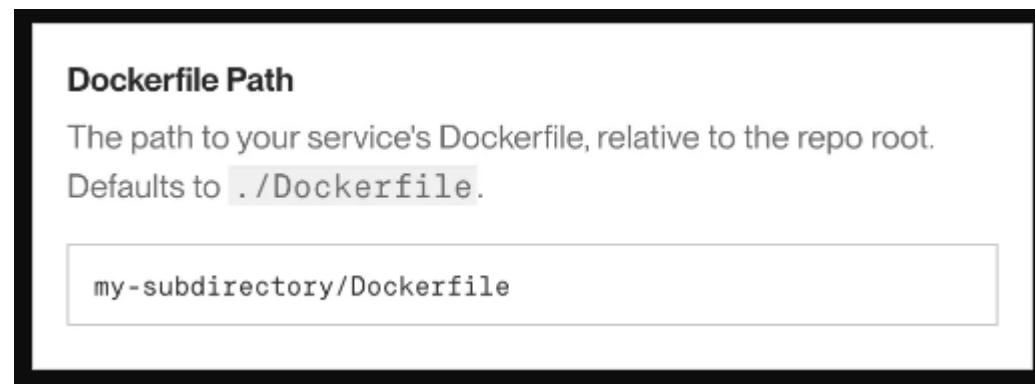
The screenshot shows the Render dashboard interface. At the top, there's a table with columns: SERVICE NAME, STATUS, TYPE, and RUNTIME. The table lists three services: my-cron-job (Successful run, Cron Job, Node), my-postgres (Available, PostgreSQL, PostgreSQL 15), and my-web-service (Deployed, Web Service, Node). Below the table, there's a dark blue bar with the text '2 services selected', a 'Move' button, and a 'Generate Blueprint' button which is being clicked by a mouse cursor. The number '126' is visible in the bottom left corner.

SERVICE NAME	STATUS	TYPE	RUNTIME
my-cron-job	Successful run	Cron Job	Node
my-postgres	Available	PostgreSQL	PostgreSQL 15
my-web-service	Deployed	Web Service	Node

2 services selected Move Generate Blueprint

Render.com con Dockerfile

Se il Dockerfile non si trova nella directory principale del repository, specificare il suo percorso (e.g., my-subdirectory/Dockerfile) nel campo Dockerfile Path



The screenshot shows a form field labeled 'Dockerfile Path'. The text below the label reads: 'The path to your service's Dockerfile, relative to the repo root. Defaults to ./Dockerfile.' The input field contains the text 'my-subdirectory/Dockerfile'.

Dockerfile Path

The path to your service's Dockerfile, relative to the repo root.
Defaults to `./Dockerfile`.

my-subdirectory/Dockerfile

Render.com e Docker Hub

- Si può eseguire il **deploy di un'immagine Docker già compilata** ad uno di questi servizi Render:
 - Web services
 - Private services
 - Background workers
 - Cron jobs
- Si può eseguire il deploy di **immagini Docker pubbliche** che si trovano su **registry raggiungibili da Render**.
- Il deploy di **immagini private** può avvenire con immagini che si trovano su questi **registri**:
 - Docker Hub
 - GitHub Container Registry
 - GitLab Container Registry
 - Goolge Artifact Registry
 - AWS Elastic Contrainer Registry) ECI

Deploy

Image

The image URL and credential used for your Web Service.

REQUISITI DELLE IMMAGINI PER IL DEPLOY

- **Linux/amd64 platform:** l'immagine Docker deve essere costruita per linux/amd64.
- L'immagine compressa **non può eccedere i 10GB**
- Per eseguire con successo il rollback di un deploy, l'immagine ed il digest al quale si vuole tornare, devono essere presenti nel registry.

1. Nella dashboard di Render, selezionare **Existing Image** come fonte del codice

Image URL

docker.io/soluzionesoftware/test_node_app1_image:latest

Credential (Optional)

No credential

Edit

2. Fornire **l'URL dell'immagine** insieme alle **credenziali** richieste per accedere all'immagine, **se privata**.

L'URL dell'immagine è quello che appare in alto a destra nella schermata del repository su Docker Hub

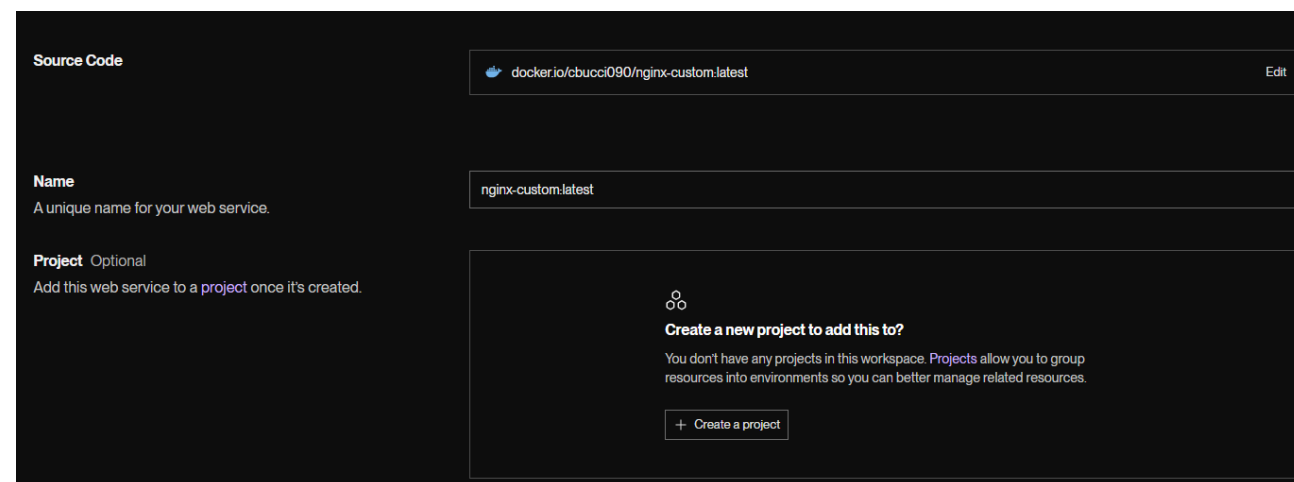
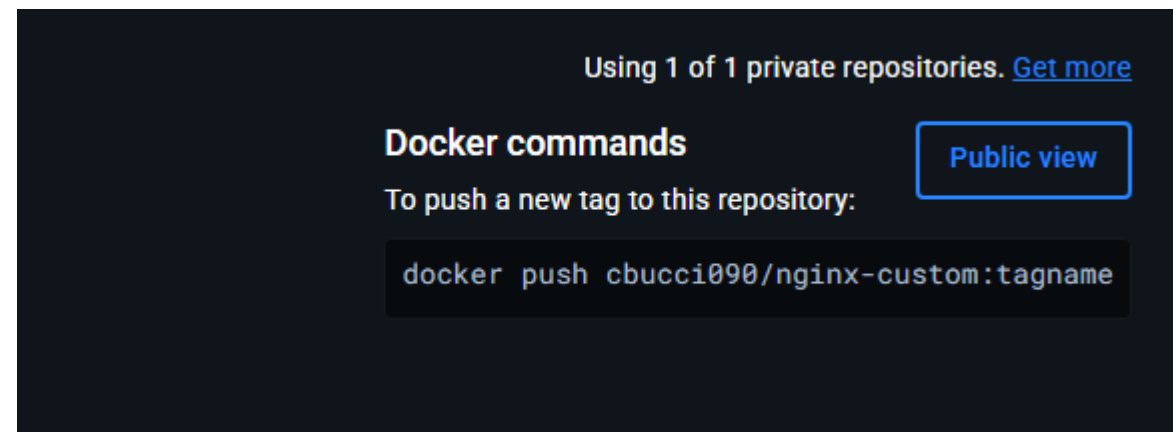
3. Se non forniti, i campi dell'URL dell'immagine usano **valori di default** per **host** dell'immagine (docker.io), **namespace** (library) e **tag** (latest).

I seguenti valori andranno tutti allo stesso URL:

- docker.io/library/alpine:latest
- docker.io/library/alpine
- library/alpine
- alpine

Si può specificare un **digest** invece di un tag:

```
docker.io/library/alpine@sha256:c0669ef34c  
dc14332c0f1ab0c2c01acb91d96014b172f1a76f3a  
39e63d1f0bda
```



- Se **l'immagine è privata**, inserire **credenziali del registry** per accedere all'immagine. Per recuperare il **Personal Access Token**, andare in **Settings > Personal access tokens** nelle impostazioni dell'account di Docker Hub

[Personal access tokens](#) / New access token

Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description
render login

Expires on
Never

Access permissions
Read-only

To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u cbucci090
```

[Copy](#)

2. At the password prompt, enter the personal access token.

```
dcKr_pat_dxkU7qLsqGNF65r30o10woDvuDc
```

[Copy](#)

[Back to access tokens](#)

DOCKER HUB

Git Provider Public Git Repository **Existing Image**

Image URL

Deploy an image from a Docker registry

 docker.io/library/nginx:latest

Credential (Optional)

No credential 

[+ Add credential](#)

 MyCredential

 MyOtherCredential

Compilare i campi:

- **Name:** un nome identificativo per le credenziali.
- **Registry:** il registry dal quale eseguire il pull
- **Username:** username dell'account del registry usato per autenticarsi
- **Personal Access Token:** il token generato dal registry che consente l'accesso all'immagine. Solo per Docker Hub, è possibile fornire la password invece del token.

DEPLOY DEI SERVIZI IMAGE-BACKED

Nel caso di servizi basati su un'immagine (image-backed services), **il deploy non viene automaticamente eseguito** ogni volta che una nuova immagine è associata al tag assegnato.

Si può eseguire nuovamente il deploy usando altri metodi.

×

Add Credential

Learn more about storing external [registry credentials](#).

Name

My credential

Registry

Select a registry ▾

Username

Personal Access Token

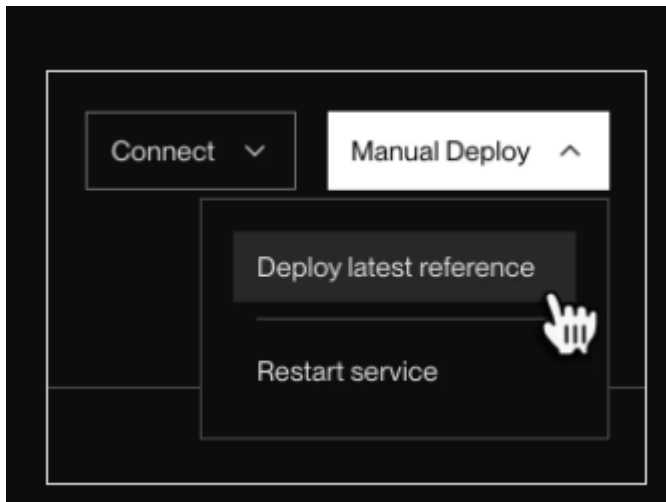
Cancel

Add Credential

1. DEPLOY DALLA DASHBOARD DI RENDER:

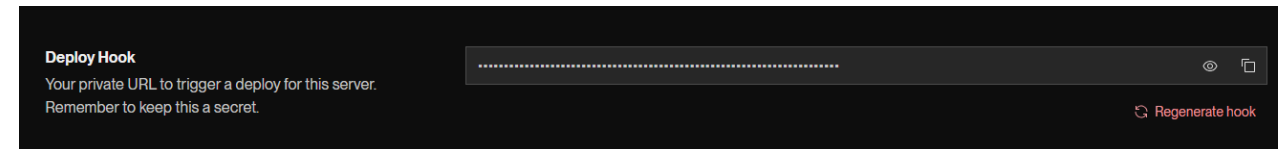
Dalla dashboard, selezionare il servizio e poi cliccare su **Manual Deploy** > **Deploy latest reference**.

Questo avvia un deploy che esegue il pull dell'immagine attualmente associata al tag assegnato al servizio.



2. DEPLOY VIA WEBHOOK:

Ogni servizio Render ha un **deploy hook URL** che può essere usato per avviare un **deploy via GET o POST request**. L'URL è disponibile nella pagina **Settings** nella dashboard di Render.



È possibile specificare un tag o digest aggiungendo un parametro **imgURL** alla query all'hook URL.

```
# Append a string with this format to your deploy hook URL.  
# This example deploys the image 'nginx:1.26' from Docker Hub.  
# Note the URL-encoding.  
&imgURL=docker.io%2Flibrary%2Fnginx%401.26
```

Render esegue il pull ed il deploy dell'immagine per il tag specificato o il digest INVECE DI usare il tag o il digest nelle impostazioni del servizio.

URL DI ESEMPIO (URL encoding richiesto):

```
https://api.render.com/deploy/srv-  
XXYYZZ?key=AABBCC&imgURL=docker.io  
%2Flibrary%2Fnginx%401.26
```

ESEMPIO CON DOCKER HUB

Prendiamo lo stesso progetto di prima.

Eseguiamo il **build** in locale:

```
docker build -t tuo-username/nome-  
progetto:latest .
```

Eseguiamo il **push** dell'immagine su **Docker Hub**

Su Render ci colleghiamo all'immagine su Docker Hub
tramite **l'URL dell'immagine**

Al termine del deploy, all'URL che viene creato per il
service, troveremo la stessa schermata di prima



PROGETTI ED AMBIENTI

È possibile organizzare i services ed impostare controlli a seconda dell'ambiente.

Un **progetto** Render è come un **contenitore logico** che permette di **raggruppare e gestire più servizi legati alla stessa applicazione**.

Un project può avere:

- **Web services:** API, backend, frontend
- **Static Sites:** siti statici (es. React/Vue buildati)
- **Background Workers:** job che girano in background (code, schedulazioni)
- **Cron jobs**
- **Databases**
- **Private Services:** microservizi visibili solo agli altri servizi del project

I progetti hanno molteplici **vantaggi**:

- **Organizzare e raccogliere** tutto ciò che riguarda una singola app
- **Networking interno:** i servizi all'interno dello stesso project possono **comunicare tra loro con hostnames privati**
- **Gestione centralizzata** con log, deploy e stati dei vari servizi.

Ogni progetto ha **uno o più ambienti** (ad es. Produzione). Se si gestiscono versioni staging e produzione, si possono aggiungere i services di ogni versione in un ambiente diverso.

CONTROLLI SPECIFICI PER AMBIENTE

Possibilità di configurare regole legate ad uno specifico ambiente:

- **Definizione di variabili d'ambiente** e **file segreti** accessibili solo dai servizi all'interno di un singolo ambiente
- Evitare che i servizi di staging usino per errore credenziali della produzione (e viceversa)
- Designare un ambiente come **protetto** in modo che solo gli amministratori possano apportare modifiche potenzialmente distruttive
- Impedire che i servizi di staging accedano ad un database di produzione (e viceversa)

CREAZIONE DI UN PROGETTO

The image shows the Pulumi CLI interface. On the left is a sidebar with a search bar and a list of resource types: Static Site, Web Service, Private Service, Background Worker, Cron Job, Postgres, Key Value, Project (highlighted with a pink bar), and Blueprint. The main area displays a 'Manual Deploy' dropdown and an 'Upgrade now' button. On the right is a 'Release' section with links like 'Deploy', 'How do I...', 'Support', 'Build pipeline', 'Deployment', 'Troubleshooting', and 'Git provider'. Overlaid on the bottom right is a 'Create a project' dialog box. The dialog has a title bar with a close button. Below the title is a description: 'Projects organize your services to make app development easier.' It contains two input fields: 'Project name' with the text 'Example project' and 'Environment name' with the text 'Production'. Below the inputs are 'Cancel' and 'Create a project' buttons.

Search ^ K

+ New Upgrade ? B

Static Site

Web Service

Private Service

Background Worker

Cron Job

Postgres

Key Value

Project

Blueprint

Manual Deploy

Upgrade now

Release

Deploy

How do I...

Support

Build pipeline

Deployment

Troubleshooting

Git provider

GitHub

GitLab

Create a project

Projects organize your services to make app development easier.

Project name

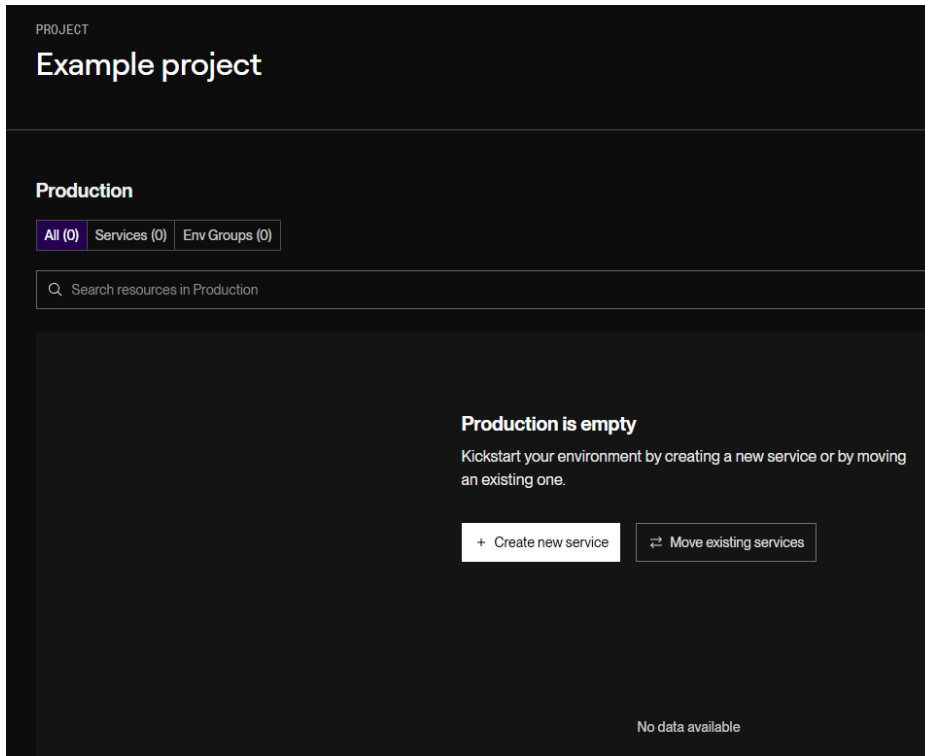
Example project

Environment name

Set up an initial environment. You can add a new environment at any time.

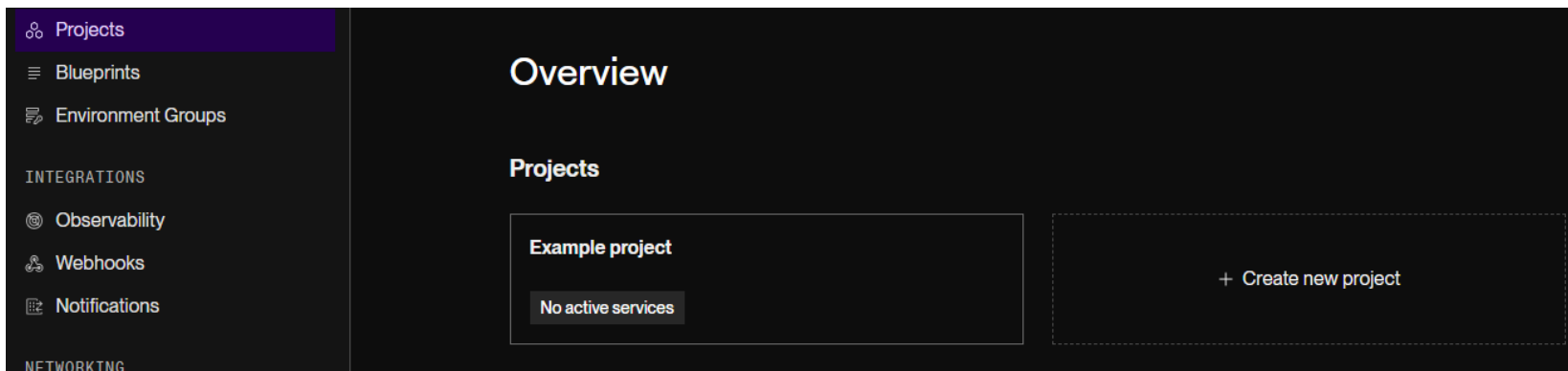
Production

Cancel Create a project



AGGIUNGERE SERVIZI AD UN PROGETTO/AMBIENTE

- Se un ambiente in un progetto è vuoto, appariranno pulsanti per la creazione di nuovi servizi o per spostarne di esistenti.
- Durante la creazione di un servizio, si può specificare a quale progetto e ambiente appartiene.
- Si possono **spostare più servizi** insieme selezionandoli nella lista dei servizi del workspace e cliccando **Move**.
- Nella homepage del workspace, i progetti appaiono in alto. I servizi di un progetto compaiono nella sua pagina, non nella homepage del workspace



Caricare un'immagine su Docker Hub

❶ Costruisci l'immagine con un tag che include il tuo nome utente Docker Hub

`docker build -t soluzionesoftware/test_jwt_node_numrnd:1.0 .`

- ◆ `soluzionesoftware` deve corrispondere al tuo username su Docker Hub

- ◆ `:1.0` è la versione dell'immagine (puoi usare anche `latest`)

Effettua il login (solo se non sei già autenticato)

`docker login`

```
PS C:\Software\Docker\test_jwt_node_numrnd> docker images --filter reference=soluzionesoftware/*
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
soluzionesoftware/test_jwt_node_numrnd    1.0      2f259ac44cee  5 minutes ago  1.09GB
soluzionesoftware/alpine-test             dev      0dc57054d60d  2 weeks ago   8.31MB
```

🔒 Ti verrà chiesto username e password (o token)

❷ Pusha l'immagine su Docker Hub

`docker push soluzionesoftware/test_jwt_node_numrnd:1.0`

📁 L'immagine viene caricata nel tuo repository Docker Hub

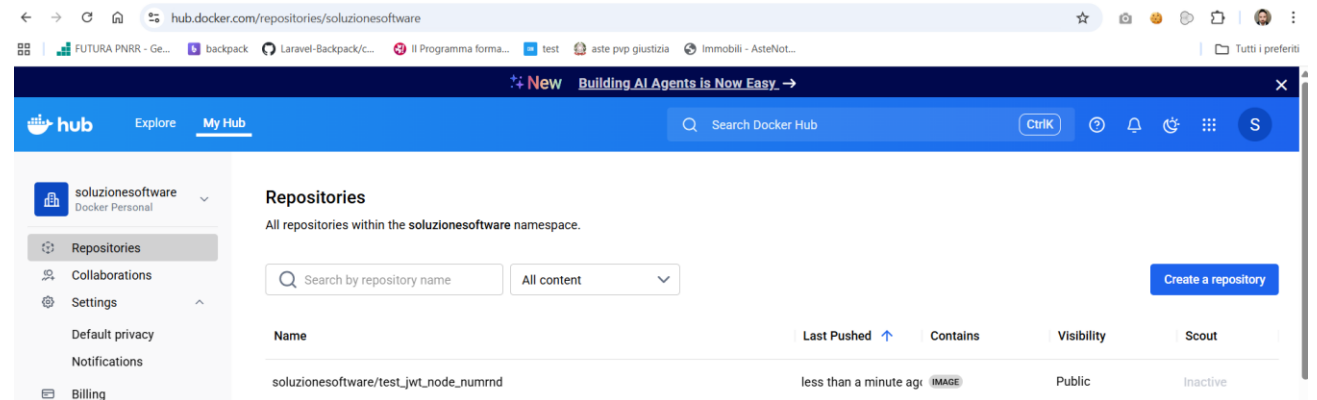
⚠️ Cosa succede se non sei loggato?

Il comando `docker push` restituirà un errore:

`denied: requested access to the resource is denied`

Questo succede perché Docker non può verificare che sei il proprietario del namespace `soluzionesoftware`

✅ Soluzione: eseguire `docker login` prima del push



Come utilizzare l'immagine pushata su docker hub - DOCKER CLI

-  Eseguire l'immagine Docker dal CLI, con Dockerfile o Docker Compose
-  Da Docker CLI
- `docker run --name test_jwt_node_numrnd_cont1 -it -p 3000:3000
soluzione software/test_jwt_node_numrnd:1.0`
-  Avvia il container interattivamente sulla porta 3000
-  Utile per test e debug veloce

Come utilizzare l'immagine pushata su docker hub - DOCKER FILE

-  Dockerfile?
-  Puoi creare un nuovo Dockerfile solo se vuoi estendere l'immagine:
- FROM soluzionesoftware/test_jwt_node_numrnd:1.0
- # Aggiunte personalizzate qui...
-  Non necessario se l'immagine è già pronta e lancia già server.js
-  Non serve solo per eseguire

Come utilizzare l'immagine pushata su docker hub - DOCKER COMPOSE

-  Docker Compose?
- Usalo quando vuoi:
- Gestire più servizi (es. DB, Redis, Nginx, ecc.)
- Replicare l'ambiente facilmente
-  docker-compose.yml:
-  Avvia tutto con: `docker-compose up -d`
-  Ideale per sviluppo, test, e deployment

Appunti finali

♦ Immagine

Un'immagine è un modello (template) da cui si creano i container.

Può essere:

scaricata da Docker Hub

costruita localmente da un Dockerfile usando:

`docker build -t nome_immagine .`

♦ Container

Si crea (e si avvia) un container da un'immagine con:

`docker run [opzioni] nome_immagine`

puoi:

fermarlo con `docker stop nome_container`

rimuoverlo con `docker rm nome_container`

♦ Rete Docker

Puoi creare una rete virtuale con:

`docker network create nome_rete`

quando avvii i container, puoi collegarli alla rete con:

`docker run --network nome_rete ...`

container nella stessa rete possono comunicare tra loro tramite:

hostname = nome del container

porte esposte

♦ Docker Compose

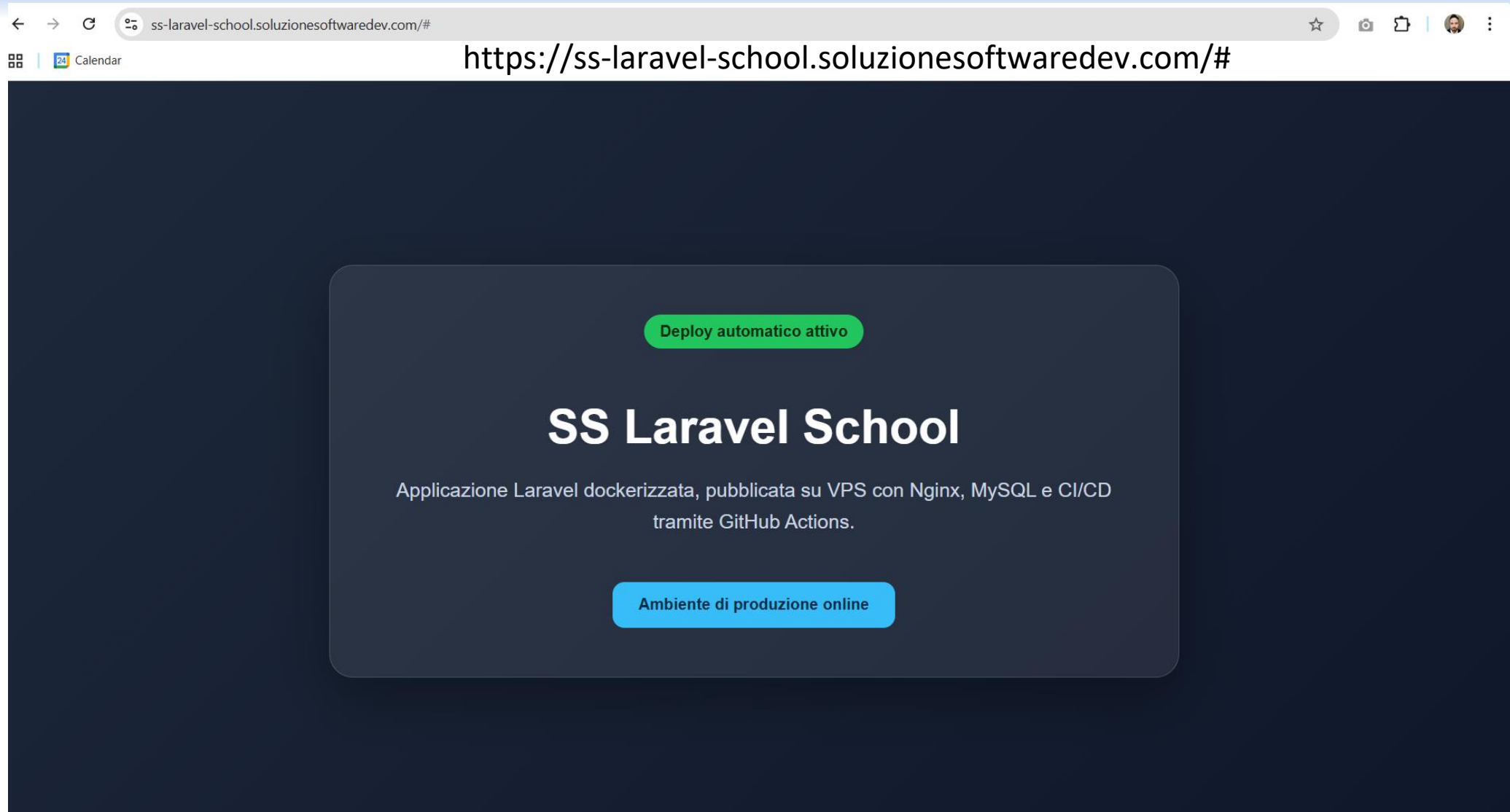
In alternativa a lanciare tutto a mano, puoi usare Docker Compose, creando un file docker-compose.yml.

Avvii tutto con:

docker-compose up

Compose gestisce:
container
reti
volumi
dipendenze tra container

Applicazione Laravel con docker



Laravel in Produzione Dockerfile

- progetto
- school/
 - └─ Dockerfile
 - └─ docker-compose.yml
 - └─ docker/
 - | └─ nginx/
 - | └─ default.conf
 - └─ app/
 - └─ public/
 - └─ routes/
 - └─ composer.json
 - └─ ...
- host:8080 → nginx → php-fpm → Laravel
- ↓
- mysql
- Nginx serve public/
PHP-FPM interpreta Laravel
MySQL ha volume persistente

- Dockerfile
- FROM php:8.2-fpm
- WORKDIR /var/www/html
- RUN apt-get update && apt-get install -y \
- git unzip libzip-dev vim \
- && docker-php-ext-install pdo pdo_mysql zip \
- && rm -rf /var/lib/apt/lists/*
- COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
- COPY . .
- RUN rm -f bootstrap/cache/*.php
- RUN composer install --optimize-autoloader --no-interaction
- RUN chown -R www-data:www-data storage bootstrap/cache

Laravel in Produzione docker-compose.yml

- **3 servizi**

- app → Laravel PHP-FPM
- nginx → web server
- mysql → database persistent

- docker-compose
- services:
- app:
- build: .
- container_name: ss-laravel-school-app
- restart: unless-stopped
- working_dir: /var/www/html
- env_file:
- - .env.docker
- volumes:
- - ./var/www/html
- - ./env.docker:/var/www/html/.env
- - /var/www/html/vendor
- networks:
- - ss_laravel_school_network
-

-
- nginx:
- image: nginx:alpine
- container_name: ss-laravel-school-nginx
- restart: unless-stopped
- ports:
- - "8080:80"
- volumes:
- - ./var/www/html
- - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf
- networks:
- - ss_laravel_school_network
- depends_on:
- - app
- mysql:
- image: mysql:8.0
- container_name: ss-laravel-school-mysql
- restart: unless-stopped
- environment:
- MYSQL_DATABASE: ss_laravel_school
- MYSQL_ROOT_PASSWORD: root
- MYSQL_USER: ss_laravel_school_user
- MYSQL_PASSWORD: myPassword!school
- volumes:
- - ss_laravel_school_data:/var/lib/mysql
- networks:
- - ss_laravel_school_network
-
- volumes:
- ss_laravel_school_data:
-
- networks:
- ss_laravel_school_network:
- driver: bridge

per creare i container

- `docker compose up -d --build`
- `docker exec -it ss-laravel-school-app php artisan key:generate`
- `docker exec -it ss-laravel-school-app php artisan migrate --force`
- `docker exec -it ss-laravel-school-app php artisan config:cache`

- se si vuole fare seed:
- `docker exec -it ss-laravel-school-app php artisan db:seed`

copiare i logs nella macchina locale

- `docker cp ss-laravel-school-app:/var/www/html/storage/logs .`
- cancellare i files dentro logs del container
- `docker exec -it ss-laravel-school-app sh -c "rm -f storage/logs/*.log"`

GitHub Actions: Automazione e CI/CD per lo sviluppo moderno

- GitHub Actions è la piattaforma integrata di GitHub che permette di automatizzare processi software direttamente dal repository.
- Consente di eseguire workflow automatici quando avvengono eventi come:
 - push di codice
 - pull request
 - rilascio versioni
 - schedulazioni temporizzate
 - eventi personalizzati
- **Cosa permette di fare**
- **Continuous Integration (CI)**
 - Ad ogni modifica può:
 - installare dipendenze
 - eseguire test automatici
 - controllare qualità codice
 - buildare applicazioni Docker
- **Continuous Deployment (CD)**
 - Dopo il push può:
 - collegarsi a server remoti
 - aggiornare container Docker
 - eseguire migrazioni database
 - pubblicare siti web o API

github actions

- .github/workflows/deploy.yml
- name: Deploy ss-laravel-school
-
- on:
- push:
- branches:
- - main
-
- jobs:
- deploy:
- runs-on: ubuntu-latest
-
- steps:
- - name: Deploy on VPS
- uses: appleboy/ssh-action@v1.2.0
- with:
- host: \${ secrets.VPS_HOST }
- username: \${ secrets.VPS_USER }
- password: \${ secrets.VPS_PASSWORD }
- port: \${ secrets.VPS_PORT }
- script: |
- /opt/apps/ss-laravel-school/update.sh

```
"email": "link@soluzioneSoftware.com",  
"password": "admin!pwd@2"
```

Access

- Collaborators
- Moderation options

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security and quality

- Advanced Security
- Deploy keys
- Secrets and variables**
- Actions
- Codespaces
- Dependabot

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets Variables

Environment secrets

This environment has no secrets.

Manage environment secrets

Repository secrets

New repository secret

Name	Last updated
VPS_HOST	now
VPS_PASSWORD	now
VPS_PORT	now
VPS_USER	now

docentmaurocasadei / ss-laravel-school

Code Issues Pull requests **Actions** Projects Wiki Security and quality Insights Settings

New workflow

hool

All workflows

Showing runs from all workflows

Filter workflow runs

2 workflow runs

Workflow	Event	Status	Branch	Actor
github actions	Deploy ss-laravel-school #2: Commit b854ac9 pushed by soluzione-software	now In progress	main	
github actions	Deploy ss-laravel-school #1: Commit 45413a2 pushed by soluzione-software	4 minutes ago 23s	main	

● Ubuntu Server 24.04 LTS 64bit - Docker

[vedi dettagli](#)



● VPS O2A4

a partire da
6,29 Euro/ mese

2 CPU Virtuale
4 GB di RAM
40 GB di Hard Disk
25 TB/mese di
traffico

• Ubuntu 24.04 + Docker

- progetto/
 - | docker-compose.yml
 - | .env
 - | nginx/
 - | | conf.d/
 - | | | app.conf
 - | scripts/
 - | | init-letsencrypt.sh
 - | | renew-letsencrypt.sh
 - | app/
 - | | Dockerfile
 - | | ...

A www.test-docenze.soluzionesoftwaredev.com

punta a 31.14.139.233

A test-docenze.soluzionesoftwaredev.com

punta a 31.14.139.233

Cosa mettere su GitHub Repo

- **Nel progetto GitHub**
- Dockerfile
- docker-compose.yml
- .github/workflows/deploy.yml
- nginx/conf.d/app.conf
- sql/hamburgeria_2.sql
- src/
package.json
- **Sul server**
- Cartella progetto:
 - mkdir -p /opt/test-docenze
 - cd /opt/ test-docenze
- File segreto:
 - nano .env.production

Configurare Github Secrets

- Settings → Secrets and variables → Actions
- VPS_HOST=test-docenze.soluzionesoftwaredev.com
VPS_USER=root
VPS_PASSWORD=la_tua_password
VPS_PATH=/opt/test-docenze

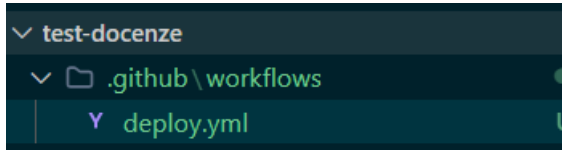
Repository secrets

Name 
 VPS_HOST
 VPS_PASSWORD
 VPS_PATH
 VPS_USER

- GitHub
 - ↓ push
- GitHub Actions
 - ↓ ssh
- VPS Ubuntu 24.04
 - ↓
- Docker Compose
 - ├── nginx
 - ├── certbot
 - ├── backend NestJS
 - └─ mysql

deploy.yml

- deploy.yml serve a dire a **GitHub Actions** cosa deve fare quando fai un push.
- Tu fai push su GitHub
↓
GitHub legge deploy.yml
↓
si collega alla VPS
↓
copia i file del progetto
↓
esegue docker compose
↓
aggiorna l'app online



```
• name: Deploy test-docenze
•
• on:
•   push:
•     branches:
•       - main
•
• jobs:
•   deploy:
•     runs-on: ubuntu-latest
•
•   steps:
•     - name: Checkout repository
•       uses: actions/checkout@v4
•
•     - name: Copy project to VPS
•       uses: appleboy/scp-action@v0.1.7
•       with:
•         host: ${ secrets.VPS_HOST }
•         username: ${ secrets.VPS_USER }
•         password: ${ secrets.VPS_PASSWORD }
•         source: "."
•         target: ${ secrets.VPS_PATH }
•         overwrite: true
•
•     - name: Deploy with Docker Compose
•       uses: appleboy/ssh-action@v1.0.3
•       with:
•         host: ${ secrets.VPS_HOST }
•         username: ${ secrets.VPS_USER }
•         password: ${ secrets.VPS_PASSWORD }
•         script: |
•           cd ${ secrets.VPS_PATH }
•           chmod +x update.sh
•           ./update.sh
```

Configurare server via SSH

- Creazione cartella dedicata al progetto
- Creazione file di configurazione:
- nano /opt/test-docenze/.env.production

```
root@TEST-DOCENZE:/opt# mkdir -p /opt/test-docenze
cd /opt/test-docenze
nano .env.production
root@TEST-DOCENZE:/opt/test-docenze# docker --version
docker compose version
Docker version 28.1.1, build 4eba377
Docker Compose version v2.35.1
root@TEST-DOCENZE:/opt/test-docenze#
```

Dockerfile

- Il Dockerfile definisce **come costruire il container del backend NestJS**.
 - In pratica dice a Docker:
 - come preparare l'ambiente Node.js, quali file copiare, quali dipendenze installare, come compilare il progetto, quale comando eseguire all'avvio.
- **Dockerfile**
 - FROM node:22-alpine
 - WORKDIR /usr/src/app
 - COPY package*.json ./
 - RUN npm install --legacy-peer-deps
 - COPY . .
 - RUN npm run build
 - EXPOSE 3000
 - CMD ["npm", "run", "start:prod"]

Docker Compose – Architettura dell'Applicazione

- **Obiettivo**
Docker Compose consente di avviare e gestire tutti i componenti dell'applicazione tramite un unico comando.
- **Servizi Configurati**
 -  **Database MySQL (db)**
 - Utilizza l'immagine ufficiale MySQL 8.0.
 - Salva i dati in un volume persistente (mysql_data).
 - Importa automaticamente il database iniziale da uno script SQL.
 - Verifica periodicamente il corretto funzionamento tramite Health Check.
 -  **Backend NestJS (backend)**
 - Costruito dal Dockerfile del progetto.
 - Espone internamente la porta 3000.
 - Utilizza le variabili di ambiente di produzione.
 - Salva i log applicativi in un volume dedicato.
 - Viene avviato solo quando il database è operativo.
 -  **Nginx (nginx)**
 - Espone le porte pubbliche 80 (HTTP) e 443 (HTTPS).
 - Riceve le richieste provenienti dagli utenti.
 - Gestisce i certificati SSL/TLS.
 - Inoltra le richieste al backend NestJS.
 -  **Certbot (certbot)**
 - Richiede e rinnova automaticamente i certificati Let's Encrypt.
 - Condivide i certificati con Nginx tramite volumi Docker.

- Internet
 - |
 - ▼
 - Nginx (80/443)
 - |
 - ▼
 - NestJS (3000)
 - |
 - ▼
 - MySQL (3306)
- Certbot
 - |
 - └─ Gestione certificati SSL

update.sh

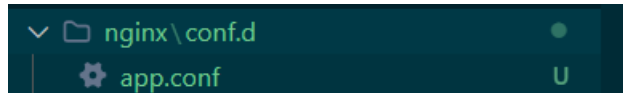
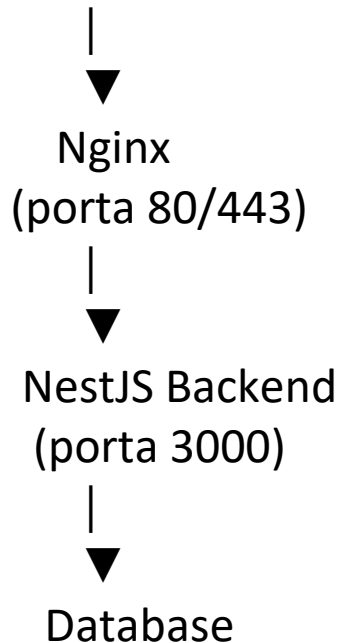
- **Script update.sh – Deploy Automatico Applicazione NestJS**
- **Scopo dello script**
Automatizzare il rilascio (deploy) dell'applicazione NestJS in ambiente Docker, configurando anche HTTPS tramite Let's Encrypt.
- **Operazioni eseguite**
 - Accede alla cartella del progetto sul server.
 - Crea le directory necessarie per Nginx e il database.
 - Avvia e aggiorna i container Docker del database e del backend NestJS.
 - Verifica la presenza di un certificato SSL valido.
 - Se il certificato esiste:
 - rinnova automaticamente il certificato;
 - aggiorna i container;
 - ricarica la configurazione di Nginx senza interrompere il servizio.
 - Se il certificato non esiste:
 - configura temporaneamente Nginx in HTTP;
 - richiede un nuovo certificato Let's Encrypt;
 - configura Nginx per l'utilizzo di HTTPS;
 - avvia l'intera infrastruttura in modalità sicura.
- Elimina le immagini Docker inutilizzate per liberare spazio sul server.

- **Risultato**
L'applicazione NestJS viene pubblicata o aggiornata automaticamente con:
 - Backend operativo in Docker.
 - Reverse Proxy Nginx.
 - Certificato SSL/TLS Let's Encrypt.
 - Accesso sicuro tramite HTTPS.
 - Minimo downtime durante gli aggiornamenti.

Ruolo di Nginx nell'architettura NestJS

- Nginx è un **Web Server** e **Reverse Proxy** che si posiziona davanti all'applicazione NestJS per gestire le richieste provenienti da Internet.
- **Perché viene utilizzato?**
- Riceve tutte le richieste degli utenti.
- Gestisce il protocollo HTTPS e i certificati SSL/TLS.
- Reindirizza il traffico HTTP verso HTTPS.
- Inoltra le richieste all'applicazione NestJS in esecuzione sulla porta interna 3000.
- Aumenta sicurezza, affidabilità e prestazioni del sistema.
- **Flusso delle richieste**

- **Utente Browser**



- **Vantaggi principali**
 - Certificati SSL gestiti centralmente.
 - Nessuna esposizione diretta del backend su Internet.
 - Possibilità di gestire più applicazioni sullo stesso server.
 - Supporto a bilanciamento del carico (Load Balancing).
 - Maggiore sicurezza e controllo del traffico.
-
- **Nel progetto**
 - Nginx riceve le richieste su:
 - `https://test-docenze.soluzioneoftwaredev.com`
 - e le inoltra al container Docker:
 - `proxy_pass http://backend:3000;`
 - dove è in esecuzione l'applicazione NestJS.

alla fine parte il deploy

- `git add .`
- `git commit -m "Add nginx ssl deploy"`
- `git push origin main`

